

ZLang HDL

Standard Library Guide

Community Release

v0.1.0a24

Source APIs, numerical and stream contracts, examples and recipes

Apache-2.0 Community distribution

Contents

1 Standard library . . . 4
1.1 Shared floating-point library . . . 5
1.2 Buffered stream composition, CRC and FIR . . . 6
1.3 Oscillators, decimation and coding . . . 6
1.4 Fractional delay and framed error correction . . . 7
1.5 Fixed-point math . . . 8
1.6 Complex values, streams, storage, and coding . . . 8
1.7 Target and architecture descriptions . . . 10
2 Standard buses . . . 11
2.1 Bounded ZTPU AXI burst subset . . . 12
2.2 AXI4-Lite, APB, and RegBus . . . 12
3 Basic GPIO, timer and serial cores . . . 14
3.1 Clock, reset and pins . . . 14
3.2 GPIO, timer and PWM . . . 14
3.3 Rising-edge capture . . . 15
3.4 UART 8N1 . . . 15
3.5 SPI mode 0 . . . 15
3.6 I²C single-byte write . . . 15
3.7 Source example . . . 15
3.8 RegBus32 wrappers . . . 16
4 Stream composition, CRC and fixed-point FIR . . . 18
4.1 Passive taps for vendor debug tools . . . 18
4.2 Broadcast and join . . . 18
4.3 CRC-32 byte frames . . . 18
4.4 Parallel FIR . . . 19
4.5 Community hardware observation boundary . . . 20
5 Reloadable and folded FIR . . . 21
5.1 Explicit one-MAC variants . . . 21
6 Oscillators, CORDIC, multirate filters and coding . . . 22
6.1 Phase accumulator and ROM sine/cosine . . . 22
6.2 Rotation-only CORDIC sine/cosine . . . 22
6.3 Vectoring, atan2 and magnitude . . . 23
6.4 CIC decimator . . . 23
6.5 Polyphase FIR decimator . . . 23
6.6 Extracted 24-bit coding blocks . . . 24
6.7 Polyphase FIR interpolation . . . 24
6.8 Fractionally interpolated NCO . . . 24
6.9 CIC interpolation . . . 25
6.10 Halfband ×2 . . . 25
7 Extended-Hamming SECDED . . . 26
7.1 Run the 39/32 example . . . 26
8 Farrow fractional delay . . . 27
8.1 Lagrange profiles . . . 27
8.2 Run . . . 27
9 DVB Reed–Solomon coding . . . 28
9.1 Encoder . . . 28
9.2 Decoder . . . 28
9.3 Run . . . 29
10 Framed K=7 Viterbi decoding . . . 30
10.1 Inputs and decision rule . . . 30
10.2 Frames, storage and timing . . . 30
10.3 Existing 48-bit encoder words . . . 30
10.4 Run a hard-decision example . . . 30
11 Standard library recipes . . . 31
11.1 Stream composition and CRC . . . 31
11.2 Choosing a FIR . . . 31
11.3 Changing the sample rate or delay . . . 31
11.4 Choosing an error-control component . . . 31

11.5 Reproducing an example . . . 31
12 Stdlib formal coverage . . . 32
12.1 New stream, DSP and coding sources . . . 33
13 Standard library API inventory . . . 34
13.1 std.arch.xilinx7_bilinear . . . 34
13.2 std.arch.xilinx7_fir . . . 36
13.3 std.arch.xilinx7_memory . . . 37
13.4 std.arch.xilinx7_multiply . . . 38
13.5 std.arch.xilinx7_multiply_add . . . 39
13.6 std.arch.xilinx7_signed_product . . . 40
13.7 std.bus.ahb_lite . . . 42
13.8 std.bus.apb . . . 42
13.9 std.bus.axi4 . . . 43
13.10 std.bus.axi4_exclusive . . . 50
13.11 std.bus.axi4_pins . . . 51
13.12 std.bus.axi4_subordinate . . . 55
13.13 std.bus.axi4_user_subordinate . . . 57
13.14 std.bus.axi_burst . . . 59
13.15 std.bus.axi_lite . . . 63
13.16 std.bus.axi_stream . . . 64
13.17 std.bus.reg . . . 65
13.18 std.bus.wishbone . . . 66
13.19 std.coding.convolutional . . . 67
13.20 std.coding.core . . . 68
13.21 std.coding.crc . . . 69
13.22 std.coding.gf256 . . . 70
13.23 std.coding.rs_dvb . . . 70
13.24 std.coding.rs_dvb_arithmetic . . . 70
13.25 std.coding.rs_dvb_decode . . . 71
13.26 std.coding.rs_dvb_decoder_inverse . . . 72
13.27 std.coding.rs_dvb_decoder_kernels . . . 72
13.28 std.coding.rs_dvb_decoder_math . . . 73
13.29 std.coding.rs_dvb_encode . . . 74
13.30 std.coding.rs_dvb_framing . . . 74
13.31 std.coding.scrambler . . . 75
13.32 std.coding.secded . . . 76
13.33 std.coding.viterbi_k7 . . . 77
13.34 std.coding.viterbi_k7_core . . . 77
13.35 std.coding.viterbi_k7_math . . . 79
13.36 std.coding.viterbi_k7_storage . . . 80
13.37 std.coding.viterbi_k7_word . . . 81
13.38 std.coding . . . 81
13.39 std.dsp.cic . . . 81
13.40 std.dsp.cordic . . . 82
13.41 std.dsp.cordic_constants . . . 83
13.42 std.dsp.cordic_vectoring . . . 84
13.43 std.dsp.farrow . . . 85
13.44 std.dsp.fft . . . 86
13.45 std.dsp.fir . . . 87
13.46 std.dsp.fir_coefficients . . . 88
13.47 std.dsp.fir_folded . . . 88
13.48 std.dsp.fir_reloadable . . . 90
13.49 std.dsp.halfband . . . 91
13.50 std.dsp.nco . . . 92
13.51 std.dsp.nco_linear . . . 93
13.52 std.dsp.polyphase . . . 94
13.53 std.math.complex . . . 95
13.54 std.math.complex_fixed_18_16 . . . 96
13.55 std.math.fixed . . . 97
13.56 std.math.fp . . . 99
13.57 std.math.fp_arithmetic . . . 100

13.58 std.math.fp_convert . . . 100
13.59 std.math.fp_format . . . 101
13.60 std.math.fp_round . . . 103
13.61 std.peripheral.capture . . . 103
13.62 std.peripheral.gpio . . . 104
13.63 std.peripheral.i2c . . . 104
13.64 std.peripheral.pwm . . . 105
13.65 std.peripheral.regbus . . . 106
13.66 std.peripheral.spi . . . 107
13.67 std.peripheral.timer . . . 107
13.68 std.peripheral.uart . . . 108
13.69 std.storage.core . . . 109
13.70 std.storage . . . 113
13.71 std.stream.complex_fixed . . . 113
13.72 std.stream.composition . . . 113
13.73 std.stream.core . . . 114
13.74 std.stream.serialization . . . 116
13.75 std.target.asic.generic . . . 116
13.76 std.target.asic.sky130 . . . 117
13.77 std.target.generic . . . 119
13.78 std.target.intel.cyclone_v . . . 122
13.79 std.target.xilinx.series7 . . . 124
13.80 std.target.xilinx.xc7z030 . . . 128
Font license notice . . . 129

Source APIs, numerical and stream contracts, examples and recipes.

1 Standard library

`std` is the compiler-shipped logical namespace. It maps to tracked ordinary ZLang HDL sources under `stdlib/`; it is intentionally shorter than the physical directory name and is not a Python or user-package import.

Import	Purpose
`std.math.fixed`	Pure exact-width/inferred fixed-point abs, clamp, min/max, MAC and explicit quantization helpers, plus compatibility components
`std.math.complex`	Generic `Complex<T>`/`Butterfly<S,D>`, nominal operators, explicit complex quantization
`std.math.complex_fixed_18_16`	Compatibility Q2.16 complex multiply profile
`std.math.fp`	Shared binary32/binary64 ready/valid FPU, independent of a CPU
`std.math.fp_format` / `std.math.fp_round` / `std.math.fp_arithmetic` / `std.math.fp_convert`	Raw-bit formats, explicit rounding, arithmetic and numerical conversion helpers
`std.stream.core`	Generic `FrameBeat<T,M>`, ready/valid register slice, skid buffer, FIFO wrapper, and explicit two-way mux/demux
`std.stream.composition`	Passive `Tee<T>` / `RvTee<T>`, buffered broadcast `RvFork2<T>` and ordered `RvJoin2<A,B>` with `StreamPair<A,B>`
`std.stream.serialization`	Bounded power-of-two vector serializer and raw bit collector
`std.stream.complex_fixed`	Compatibility Q5.32-to-Q2.16 complex stream quantizer
`std.dsp.fft`	Exact FFT butterfly, bit reversal, and compile-time twiddle helpers
`std.dsp.fir`	Parallel II=1 immutable-coefficient fixed-point FIR, exact or final nearest-even/saturating output
`std.dsp.fir_reloadable` / `std.dsp.fir_folded`	Atomic two-bank reload and explicit one-MAC FIR variants
`std.dsp.nco`	Binary-turn phase accumulator and synchronous paired sine/cosine ROM
`std.dsp.nco_linear`	Two-edge II=1 linear interpolation between paired ROM endpoints
`std.dsp.cordic`	Rotation-only sin/cos, signed binary turns, global-stall pipeline
`std.dsp.cordic_vectoring`	Normalized vectoring, atan2 and unsigned magnitude
`std.dsp.cic`	Explicit-width modular CIC decimation/interpolation
`std.dsp.polyphase`	Exact or explicitly quantized polyphase FIR decimation/interpolation
`std.dsp.halfband`	Checked symmetric ×2 decimation/interpolation with exact pre-adders
`std.dsp.farrow`	Exact stored-coefficient fractional delay, linear/cubic Lagrange and final conversion
`std.storage` / `std.storage.core`	Bounded delay/FIFO plus raw-bit reorder and ordered ping-pong storage wrappers
`std.coding` / `std.coding.core`	Parity, bit reversal, checked polynomial-tap LFSR step, and exact convolution helpers
`std.coding.crc`	Exact reflected polynomial step and framed byte-stream CRC-32
`std.coding.scrambler`	x^7+x^4+1 24-bit kernel with explicit restart/clear controls
`std.coding.convolutional`	Rate-1/2 K=7 24-bit kernel with explicit history controls
`std.coding.secded`	Extended-Hamming functions and one-stage ready/valid encode/decode, K=1…64
`std.coding.gf256`	Exact polynomial-basis GF(256) arithmetic, polynomial 0x11D
`std.coding.rs_dvb` / `std.coding.rs_dvb_decode`	Framed systematic DVB RS(204,188) encoder / errors-only decoder
`std.coding.viterbi_k7`	Framed K=7 hard/3-bit-soft Viterbi core and word adapter

Import	Purpose
`std.peripheral.*`	Fixed-width GPIO, timer, PWM, rising capture, UART 8N1, SPI mode 0 and single-byte I²C write cores, each with a CSR-backed RegBus32 wrapper; contracts
`std.bus.reg`	RegBus and the source-authoritative CSR target
`std.bus.axi_lite`	AXI4-Lite protocol and RegBus frontend
`std.bus.axi_burst`	Bounded no-ID AXI burst profile, read/write views, and single-outstanding burst/single-beat helpers
`std.bus.apb`	APB protocol and RegBus frontend
`std.bus.ahb_lite`	Standards-correct bounded AHB-Lite protocol and RegBus frontend
`std.bus.axi_stream`	AXI4-Stream beat/profile and backpressure-preserving pipe
`std.bus.wishbone`	Wishbone B4 Classic and RegBus frontend
`std.target.generic`	Resource-free generic target identity
`std.target.asic.generic`	Generic ASIC cell/resource capability profile
`std.target.asic.sky130`	SKY130 `sky130_fd_sc_hd` standard-cell synthesis profile
`std.target.intel.cyclone_v`	Bounded Cyclone-V target/resource inventory
`std.target.xilinx.series7`	Bounded source-described Series-7 DSP48E1 profile
`std.target.xilinx.xc7z030`	XC7Z030 part and bounded resource inventory
`std.arch.xilinx7_fir`	Manual four-resource symmetric-FIR cascade template
`std.arch.xilinx7_memory`	Bounded Series-7 storage/resource mapping descriptions
`std.arch.xilinx7_multiply`	Series-7 hard-multiplier architecture descriptions
`std.arch.xilinx7_signed_product`	Signed product-reduction architecture descriptions

1.1 Shared floating-point library

Import `std.math.fp` for `RvBinaryFloat<E,F>`. Its supported interchange profiles are E=8/F=23 (binary32) and E=11/F=52 (binary64). `FpRequest<W>` carries raw operands, an explicit operation and rounding mode; `FpResult` carries the result, exception flags and illegal-operation indication. These declarations and helpers are ordinary stdlib sources for standalone hardware and composition into larger designs. Library availability does not establish support in any particular processor or application.

The core holds one transaction and one buffered result. Simple operations take one edge; finite division takes F+6 and square root F+5 edges. Output stalls preserve the result and flags; cancel/reset discards pending work. There is no hidden CSR or overlapping transaction execution. See the API inventory and source contract for exact signatures and operation selection. Source availability and specialization tests do not establish a universal IEEE arithmetic proof.

`FpRequest<W>` carries `a`, `b` and `c` as `bits<W>`, an `integer:bits<64>`, `operation:u5`, `rounding:u3`, and the `word`/`unsigned_integer` control bits. Operation values follow the source contract:

Value	Operation
0…4	Add, subtract, multiply, divide, square root
5…8	Fused multiply-add, multiply-subtract, negated multiply-subtract, negated multiply-add
9…11	Copy, negate, or XOR the operand sign
12…17	Minimum, maximum, equality, less-than, less-or-equal, classify
18…20	Float-to-integer, integer-to-float, conversion from the other supported float format
21	Move raw floating bits to the integer result
22	Move raw integer bits to the floating result

Rounding values 0…4 mean nearest-even, toward zero, toward negative infinity, toward positive infinity, and nearest with ties away from zero. Operations that do not round ignore this field. `FpResult.value` is always `bits<64>`; floating results occupy the low W bits. Integer conversion uses `word` and `unsigned_integer`; raw moves follow their operation/format contract. Operation 20

reads the other-format raw bits from `request.integer` (its low 32 bits for a binary64 destination), rather than from `a`. `flags[4:0]` is invalid, divide-by-zero, overflow, underflow, inexact. Operation values 23…31 are illegal. Inspect `illegal` as well as flags.

Use the stream module for the complete finite division/square-root datapath; `fp_compute` alone does not implement those iterative cases. Hold each input until transfer, consume each result only on output transfer, and use `cancel` only to discard work. See ready/valid semantics for the common handshake contract.

1.2 Buffered stream composition, CRC and FIR

`Tee<T>` duplicates a wire value onto `output` and `tap`. `RvTee<T>` forwards the functional ready/-valid stream unchanged and exposes payload, valid, ready and transfer as ordinary wire outputs. Neither adds state, latency, buffering or CDC; the tap has no `ready` and cannot stall the stream. They do not retain samples or preserve nets against downstream optimization. See passive tap contracts and the ILA/Signal Tap wrapper example.

`RvFork2<T>` stores one accepted item and tracks its delivery to `output0` and `output1` separately. Each consumer receives it exactly once even if the other stalls. The last delivery may retire on the edge accepting a replacement. `RvJoin2<A,B>` stores one item per input and pairs their acceptance order in `StreamPair<A,B> { left:A, right:B }`. Both lanes can refill on pair retirement. Reset discards pending items; neither component performs CDC or type conversion.

`crc_step_reflected<W>` takes W-bit raw state, one data bit and W-bit reflected polynomial, and returns W-bit raw state without initialization/finalization. `RvCrc32Byte` accepts `CrcByteBeat { data:bits<8>, last:bit }` through `input` and emits one 32-bit checksum on `output` for each accepted last byte. It uses LSB-first byte order, reflected polynomial `0xEDB88320`, raw init and final XOR `0xFFFFFFFF`. A pending result blocks the next frame except on simultaneous retire/accept. Reset drops a partial frame and pending result; empty frames have no representation.

`RvFirFixedExact<SW,SF,CW,CF,N,coefficients>` takes signed fixed-point samples and immutable compile-time `vec<N,fixed<CW,CF>>` coefficients. Pass named arguments with a typed local vector, as in `coefficients=taps`. N must be at least one; the checked derived dimension H defaults to N-1. Coefficient zero multiplies the current accepted sample. Later coefficients multiply previous accepted samples; history advances only on `input.transfer`. Products and balanced `dot` accumulation remain exact, giving `fixed<SW+CW+ceil_log2(N),SF+CF>`. N=1 has no history storage.

The FIR has one buffered output stage: unstalled latency one, II=1, and same-edge retire/refill. Backpressure holds output and history; reset clears validity and zeroes history. `RvFirFixed NearestEvenSaturate` additionally takes OW/OF and converts only the final exact result to `fixed<OW,OF>` using nearest-even rounding and saturation, without an extra pipeline stage. Coefficient reload, decimation/interpolation, folded scheduling and automatic symmetry detection are not part of these modules.

Separate `std.dsp.fir_reloadable` modules expose serial inactive-bank loading and atomic `commit/commit_ready`. A full bank activates only after computation and retirement of the old result; the switch edge accepts no sample or coefficient. History survives commit, while reset restores initial coefficients and clears history/loading/results. Separate `std.dsp.fir_folded` modules use one logical MAC for N=1…32, with unstalled latency=II=N and no overlapping sample computations. Both families provide exact and single-final-conversion variants. See their complete contracts and commands.

The stream/DSP guide gives complete commands for `framed_crc.zhl` and `stream_fir8.zhl` with checked-in stimulus events.

1.3 Oscillators, decimation and coding

`std.dsp.nco` provides a modular accepted-sample phase accumulator and paired synchronous-ROM sine/cosine. `std.dsp.cic` makes decimation rate, stage count, differential delay and modular internal/output width explicit. `std.dsp.polyphase` groups immutable FIR coefficients by phase, retaining exact products and accumulation before an optional single final quantization. `std.coding.scrambler` and `std.coding.convolutional` expose the same 24-bit kernels used by

the 802.11a project, plus one-result stream wrappers with explicit restart/tail controls. No compiler dispatch recognizes these names. The multirate/coding guide defines phase units, coefficient/bit order, latency and reset/stall contracts with runnable stimuli. `std.dsp.cordic` provides `RvCordicSinCos<P=16,N=P-2>`: input `rv<sint<P>>` encodes signed binary turns, output `rv<SinCos<fixed<16,14>>>`. P is 8/12/16, 1<=N<=P-2, guard bits G=4. Unstalled latency is N+2 rising edges, II=1; output backpressure freezes every stage and reset clears all in-flight work. Only the final coordinate conversion rounds nearest-even/saturates. This is an approximation, not correctly rounded sin/cos; the guide gives error bounds and the runnable `examples/cordic.zhl` schedule. Vectoring is a separate module: `std.dsp.cordic_vectoring` provides `RvCordicVectoring` and `RvAtan2`, with SW=P=8/12/16, SF=SW-2 and N=P-2. The former returns phase, unsigned magnitude and `angle_defined`; the latter omits magnitude. Unstalled latency is N+3 edges, II=1, with global stall. Zero input has phase/magnitude zero and undefined angle; the negative x axis is the negative half-turn code. See the guide for normalization, nearest-even conversion and universal profile error bounds.

`std.dsp.polyphase` also provides exact/final-quantized interpolation: `y[n*R+p] = sum_k h[p+R*k]*x[n-k]`, R ordered outputs per accepted input, first output after one edge, input II=R. Exact width is SW+CW+ceil_log2(K), scale SF+CF. Coefficients define gain without hidden R scaling. `std.dsp.nco_linear` provides two-edge, II=1 linear ROM endpoint interpolation. `std.dsp.cic` additionally provides modular interpolation, with explicit IW, R/N/M and nominal gain (R*M)^N/R. `std.dsp.halfband` supplies checked symmetric ×2 decimation/interpolation, exact SW+1 pre-adders and optional final conversion. Their numerical and stream contracts define phase order, zero insertion, stalls, reset and runnable examples. None adds automatic scheduling, coefficient reload to multirate filters, implicit CDC, or a new `implement` feature.

1.4 Fractional delay and framed error correction

`std.dsp.farrow` provides `RvFarrowFixedExact`, linear/cubic Lagrange profiles and final nearest-even/saturating conversion. Each accepted beat carries sample and `mu_raw:uint<MF>`, representing μ=mu_raw/2^MF in [0,1). M=0…3 and K=1…8; one registered subfilter stage plus M Horner stages give latency M+1, II=1 under global stall. History advances only on acceptance. With B=SW+CW+ceil_log2(K), F=SF+CF, exact output is `fixed<B+M*(MF+2),F+M*MF>`. Exactness is relative to the stored binary coefficients; linear delay μ and cubic delay 1+μ are separate from pipeline latency. This is not a resampling controller. See the coefficient/error contracts and runnable example.

`std.coding.secded` provides pure encode/decode and elastic one-stage wrappers for K=1…64. Hamming parity occupies power-of-two positions, data fills the remaining positions from the low bit, and overall parity is last. Decode reports clean, corrected or detected-uncorrectable, with a separate data-valid flag. Every single error is corrected and every double error detected; larger patterns have no guarantee. See layout, status and example.

`std.coding.gf256` uses polynomial basis with primitive polynomial 0x11D. `std.coding.rs_dvb` encodes complete 188-byte frames as systematic 204-byte codewords; `std.coding.rs_dvb_decode` accepts complete codewords before checking/correcting and emitting 188 payload bytes. Status precedes payload; detected failures emit no trusted payload. The fixed DVB profile is RS(255,239) shortened by 51 leading zero symbols, with roots α^0…α^{15} and primitive element 0x02. Errors-only decoding corrects up to eight erroneous symbols; beyond that radius even zero post-correction syndromes do not rule out miscorrection. No erasures or interleaving are implicit. See byte order, framing and runnable examples.

`std.coding.viterbi_k7` exposes one K=7 rate-1/2 ACS engine for hard or 3-bit soft received pairs, using the existing encoder recurrence and G0/G1 order. It buffers complete survivor history for 1…4096 decoded bits, then performs traceback and ordered output. Ties prefer the smaller predecessor and final state. Initial state is zero; final state minimizes metric and is not assumed zero. The word adapter preserves transmission order without inserting tail bits. Minimum-metric decoding is not universal error detection. See pair/word interfaces, metric bounds and examples.

These source contracts are not claims of universal formal coverage or of refreshed installed packages. Consult the formal coverage report for executed evidence; declarations without associated evidence remain unverified.

Imports are resolved transitively and cycles are rejected. External path/Git packages use the

pinned zlang.toml/zlang.lock project resolver. Filesystem-relative source imports and implicit network lookup are not part of the std resolver, and ordinary compilation never fetches dependencies.

Reusable stateful modules across the standard-bus packages inherit their enclosing module's one exact clock/reset contract. Concrete application tops and the source-authored AXI flat-pin adapters own external clock/reset pins. The inherited contract is preserved in specialization identity, simulation, formal products, and Direct-SV; absent or ambiguous direct-top ownership and incompatible hierarchy fail closed rather than selecting a default reset.

std.bus.axi_burst is the source-owned ZTPU interoperability profile, not a compiler-recognized bus. It publishes the combined five-channel AXI4BurstSubset<AW,DW> plus read-only and write-only views matching ZTPU's separate physical masters. AXI4BurstReader and AXI4BurstWriter implement one full-width incrementing transaction at a time, with 1-256-beat counting, independent channel backpressure, checked RLAST, counted WLAST, and deterministic boolean error latching for nonzero RRESP/BRESP. The validated AW=64,DW=32 witness passes semantic/canonical restoration, deterministic backend emission, simulator traces, and Direct-SV/Verilator. IDs, write strobes, burst-kind and other full-AXI sidebands, multiple outstanding transactions, UB-DMA chunking, and fences remain outside this bounded profile.

For one full-width beat, axi_single_beat_address<AW,DW> constructs AxiBurstAddress<AW> with len=0 and the matching size, plus a validity bit for geometry and alignment. AXI4SingleBeatReader preserves the burst reader's one-beat RLAST/RRESP checks; AXI4SingleBeatWriter snapshots AW and W payloads and allows either channel to transfer first. Both retain the one-outstanding busy/done/error convention. See examples/axi_single_beat.zhl. These helpers do not add IDs, WSTRB, or implicit bus adaptation.

std.bus.ahb_lite follows the AHB-Lite address/data pipeline and two-cycle ERROR response rather than the historical ZTPU model's same-cycle AHB-like shortcut. AHBLiteToRegBus<AW,DW> accepts one aligned full-width beat at a time for byte-addressable power-of-two DW from 8 through 1024, supplies an all-byte RegBus write mask, and holds the AHB data phase through RegBus request/response stalls. Subword transfers, arbitration, SPLIT/RETRY and compiler/backend AHB dispatch are not part of this profile. Its hresetn example domain uses active-low asynchronous assertion and two-edge synchronized release. The bridge itself inherits the enclosing module's one exact physical clock/reset contract, so the same source also composes with a synchronous active-high system. Ambiguous multi-domain inheritance and mismatched connected state owners remain errors; this is not an implicit reset or CDC conversion.

std.math.complex is ordinary source-authoritative ZLang. It has no bus or stream dependency and uses only generic structs/functions and nominal operator declarations; semantic analysis and the Direct-SV backend have no Complex special case. Mixed fixed-point multiplication retains its full exact width and scale, and the caller places every quantization boundary explicitly. In particular, the core contains no implicit fixed<18,16> butterfly. The historical Q2.16 component and stream profiles live in separate compatibility imports std.math.complex_fixed_18_16 and std.stream.complex_fixed; importing numerical Complex arithmetic does not select either profile or import a bus.

1.5 Fixed-point math

std.math.fixed provides pure generic fixed_abs, fixed_min, fixed_max, fixed_clamp, and exact fixed_mac functions plus explicit rounding/overflow helpers such as fixed_quantize_nearest_even_saturate<T>. fixed_abs widens according to ordinary unary-negation rules, so the most-negative input has an exact positive result; fixed_mac preserves the full product and addition widths. The library also retains the source-authored FixedAbs, FixedClamp, FixedMinMax, FixedMAC, FixedSaturatingAdd, and UFixedSaturatingAdd components. They operate on canonical fixed-point types; concise SF8.8, SF_Sat8.8, UF8.8, and UF_Sat8.8 forms are language aliases rather than library implementations. Encoding and conversion rules are specified in fixed-point-types.md.

1.6 Complex values, streams, storage, and coding

std.math.complex provides the ordinary parameterized value type Complex<T> with re and im fields. butterfly_quantized<T> requires the output numeric profile explicitly; there is no profile-

selecting short spelling in the core. Complex remains ordinary source, not a compiler or backend primitive.

`std.bus.axi_stream` keeps `AXIStream<DW>` as the byte-lane bus profile and also provides `AXIStreamOf<T>` for a semantic payload type:

```
input  : AXIStreamOf<Complex<fixed<18,16>>>.sink @clk
output : AXIStreamOf<Complex<fixed<18,16>>>.source @clk
```

`AXIStream<32>` has `data/keep/strb/last`; `AXIStreamOf<T>` transports exactly one typed `T` per transfer. It does not by itself specify an external AXI TDATA serialization for arbitrary `T`.

Generic stream composition uses `std.stream.core`, independently of AXI:

```
import std.stream.core

struct Meta { tag : u2 }
type Beat = FrameBeat<u8,Meta>

module Queue {
    clock clk
    reset rst
    in input : rv<Beat>
    out output : rv<Beat>

    inst storage : RvFifo<T=Beat,D=4>
    connect input -> storage.input
    connect storage.output -> output
}
```

`RvRegisterSlice<T>`, `RvSkidBuffer<T>`, and `RvFifo<T,D>` reuse the exact language FIFO semantics, including simultaneous push/pop and payload stability under stall. `RvMux2<T>` and `RvDemux2<T>` provide explicit runtime routing between statically elaborated ready/valid endpoints. They route backpressure only through the selected path, never create dynamic instances or clock gates, and can be composed as a bounded tree for wider routing. Scalar routing remains ordinary `mux` and explicit valid/enable gating. `std.stream.serialization` currently supports a power-of-two `RvVectorSerializer<T,N,IW>` and a raw `RvBitCollector<N,IW>`, where `IW = floor_log2(N)`. Their declarations enforce `N >= 2`, power-of-two `N`, and exact `IW` through compile-time `where` constraints.

`std.dsp.fft` keeps arithmetic intent explicit. Its butterfly does not quantize, twiddle generation uses compile-time `sin`/`cos` and quantizes only to its caller-selected type, and bit reversal is a deterministic sequence-order operation. Value parameters inferred through a `bits<N>` argument are not accepted, so bit-width helpers use explicit calls such as `fft_bit_reverse<N=8>(value)`.

`std.storage.core` supplies `StorageDelay1<T>`, `StorageDelay2<T>`, the generic `StorageQueue<T,D,CW>` (`CW = ceil_log2(D + 1)`), and bounded raw-bit `StorageReorderBits<W,N>`/`StoragePingPongBits<W,N>` banks. The latter use power-of-two depth, exact runtime indices, and explicit bitcast at typed/raw boundaries. Ping-pong publication is ordered: a second `commit` is blocked until the visible read bank retires, while simultaneous retire/commit is legal. The same file provides ordinary `StorageDualPortMemory<T,N,AW>`, `Storage2R1WMemory<T,N,AW>`, and `StorageAsyncMemory1W1R<T,N,AW>` wrappers over the shared typed memory IR. `StorageAsyncFifo<T,D>` wraps the existing explicit ready/valid `async_fifo(D)` crossing. These module names are not compiler intrinsics and do not authorize implicit CDC. It also provides immutable, source-authored generic ROM wrappers:

```
inst direct : StorageRom<T=u8,N=8,IW=3,image=image>
inst generated : StorageGeneratedRom<
    T=u8,N=8,IW=3,producer=fn make_image<T=u8,N=8>
>
```

Both wrappers elaborate to the existing typed `Rom` IR with concrete immutable contents and the

deterministic companion image used by Direct-SV. `StorageRom` accepts a fully evaluated exact `vec<N,T>` constant; `StorageGeneratedRom` invokes a statically selected pure zero-argument producer during elaboration. Constants and producers are specialization parameters, not runtime ports or backend callbacks. Fixed, struct, vector, and nested-vector word types retain their exact canonical layout.

The core language currently requires a literal in `delay<N>`, so a generic-depth delay wrapper would be dishonest; longer delays remain explicit source until parameterized delay depth is supported. `std.coding.core` supplies representation-level parity, bit reversal and one-step LFSR operations plus exact dot/convolution and `table_gather` helpers. `table_gather<T,N,IW>` is accepted only when the complete proven `uint<IW>` range fits the source vector. It preserves order but deliberately does not claim bijection: duplicate and omitted elements remain legal. The `CodingLfsrStep<N>` module enforces `N >= 2`; the compatibility pure function cannot carry a `where` clause because function constraints are not yet syntax.

The current language deliberately prevents several tempting but invalid ”generic” wrappers. A target-independent `reg vec<N,T>` requires an explicit initializer; `default<T>` is not supported. The shipped reusable reorder/ping-pong banks therefore use an explicit `bits<W>` representation boundary. A generic runtime gather now retains the conservative element-type range through a table-loaded index, but it does not prove that a table is a mathematical permutation. The negative semantic tests retain the bounded gather witness; no backend guesses an initializer or claims a permutation proof. Concrete projects may still use typed ROMs and permutations with known source-generated tables, as the Wi-Fi and FFT sources do.

The separated `std.stream.complex_fixed` compatibility kernel exposes raw `rv<Complex<...>>` ports. The former aggregate AXIStream profile cannot be forwarded transparently yet: hierarchical connections between `AXIStreamOf<Complex<...>>` specializations and member-level child ready/valid ports are rejected by the current aggregate binding model. The kernel is fully backend-validated, but restoring that aggregate ABI requires a generic aggregate-member binding compiler slice rather than bus-specific stdlib code.

Every shipped `.zhl` file is discovered recursively by the `std.*` resolver. Clean-wheel tests compare the complete recursive source tree with wheel contents; adding a library file without packaging it therefore fails the build rather than creating a checkout-only import.

1.7 Target and architecture descriptions

Target libraries use the same safe, hashed `std.*` resolver as bus and math sources. They describe resource capabilities and physical binding locators; they do not add functional primitives. See target-platform-architecture-description.md for the supported declarations, manual selection flow, and current bounded Series-7 implementation.

2 Standard buses

The production bus profiles are ordinary ZLang sources. The compiler contains no AHB, AXI, APB, AXI-Stream, Wishbone, or RegBus transaction dispatcher.

Import	Source-owned declarations	Initial profile
`std.bus.reg`	`RegBus`, CSR bank/target	In-order request/response CSR boundary
`std.bus.axi_lite`	`AXI4Lite`, `AXI4LiteToRegBus`	32/32-compatible, independent AW/W buffering
`std.bus.axi_burst`	`AXI4BurstSubset`, read/write views and helpers	No-ID, single-outstanding, full-width incrementing bursts
`std.bus.axi4`	Five-channel `AXI4`/`AXI4WithUser`, bounded read/write managers	IDs, bursts and optional USER payloads; under validation
`std.bus.axi4_subordinate`	Bounded read/write subordinate adapters	Backend transactions and responses; under validation
`std.bus.axi4_pins`	Flat-pin adapters	Source-owned five-channel pin projection
`std.bus.axi4_exclusive`	Reservation monitor and exclusive subordinate	Separate explicit same-edge commit contract; under validation
`std.bus.apb`	`APB`, `APBToRegBus`	APB setup/access sequencing
`std.bus.ahb_lite`	`AHBLite`, `AHBLiteToRegBus`	Single-manager, full-width AHB-Lite to RegBus
`std.bus.axi_stream`	`AXIStream`, `AXIStreamPipe`	Data/keep/strb/last ready/valid stream
`std.bus.wishbone`	`Wishbone`, `WishboneToRegBus`	B4 Classic single-beat, ack/err/stall

Error completion is part of the source profile rather than a backend policy. AXI4-Lite maps a failed RegBus write or read to `SLVERR` (`2'b10`) and holds the complete B/R payload while the corresponding ready signal is low. APB forwards the held RegBus error through `PSLVERR` on the access completion. Wishbone uses mutually exclusive normal `ACK` and abnormal `ERR` termination; either one retires the single outstanding request.

AHB-Lite preserves the protocol's pipelined address/data relationship. The bridge captures an accepted address/control phase, consumes write data in the following data phase, and holds `HREADYOUT` low while its one RegBus request is pending. Local size/alignment errors and RegBus failures both use the standard two-cycle ERROR response: low-ready/high-response followed by high-ready/high-response. The first profile accepts aligned, full-bus-width transfers for power-of-two byte-addressable data widths from 8 through 1024; subword transfer strobes are deliberately deferred.

Reusable stateful cores in `std.bus.reg`, `std.bus.ahb_lite`, `std.bus.axi_lite`, `std.bus.axi4`, `std.bus.axi4_subordinate`, `std.bus.axi4_user_subordinate`, `std.bus.axi_burst`, `std.bus.axi_stream`, `std.bus.apb`, and `std.bus.wishbone` do not impose a fixed physical reset spelling. Each inherits the enclosing module's one exact clock/reset contract. Native simulation, formal verification, and generated SystemVerilog all use that same contract. Source-owned flat AXI pin adapters and concrete application tops declare their physical domains explicitly rather than hiding reset policy inside reusable cores.

The runnable AHB example uses conventional active-low `hresetn` with asynchronous assertion and the language's two-edge synchronized release; the same bridge also composes with a synchronous active-high parent. The APB, AXI4-Lite, Wishbone, AXI-Stream, RegBus, and bounded AXI4 witnesses exercise the same inheritance rule. Ambiguous multi-domain inheritance, a stateful core compiled as a direct top without an explicit tooling-owned root domain, and mismatched connected sequential logic fail closed. In every supported AHB contract, `HREADYOUT` is high while reset is asserted.

`std` is a logical namespace mapped to the physical `stdlib/` source tree. Its `.zhl` modules are discovered by convention. Cyclic imports and unsafe paths are rejected.

The AXI4 source modules include IDs, bounded outstanding slots, five independent channels, and optional USER sidebands. Directed and randomized tests cover stalls, repeated IDs, slot reuse, error responses, native simulation against generated Direct-SV, and direct-SystemVerilog generation. These checks do **not** establish complete external endpoint compliance: bounded formal endpoint proofs remain outstanding. The ordinary subordinate never emits `EXOKAY`; exclusive commit requires the separate `axi4_exclusive` contract and same-edge `exclusive_grant` gating. The narrower `axi_burst` profile remains distinct and unchanged. AXI-Stream ID/dest/user, Wishbone burst/retry, automatic adapters, and CDC are not added by these profiles.

Runnable integrated examples are AXI4-Lite CSR, APB CSR, AHB-Lite CSR, Wishbone CSR, and the streaming packet engine. The bounded burst helpers are exposed by the ZTPU AXI burst witness. Python models in `tests/support/standard_bus.py` are independent test oracles, not compiler runtime implementations.

The AHB-Lite contract follows the Arm AMBA 3 AHB-Lite protocol rather than the older ZTPU model's simplified same-cycle "AHB-like" behavior. In particular, write address/control and `HWDATA` are not sampled in the same phase, and ERROR is not shortened to one cycle. The source profile and executable tests freeze that bounded contract; broader AHB features remain explicit exclusions below.

2.1 Bounded ZTPU AXI burst subset

`std.bus.axi_burst` is ordinary source-authoritative ZLang HDL. It defines a combined `AXI4BurstSubset<AW,DW>` with independent AR/R and AW/W/B ready/valid channels, together with read-only and write-only protocol views for ZTPU's separate physical master ports. Address payloads carry byte address, eight-bit beats-minus-one `len`, and three-bit beat `size`; read and write data carry `last`, and R/B carry the exact two-bit response value.

The reusable `AXI4BurstReader<AW,DW,LW>` and `AXI4BurstWriter<AW,DW,LW>` accept aligned 1-256-beat requests and permit one outstanding transaction each. AR and AW payloads are snapshotted and remain stable until accepted. Read consumer backpressure drives RREADY directly. The writer accepts AW before exposing W, while AW, W, and B remain independent handshakes; its ready/valid producer retains W data while stalled. Counted framing owns completion: the reader reports early or missing RLAST but drains the requested beat count, while the writer generates WLAST on its locally counted final beat. Any nonzero RRESP or BRESP sets a sticky boolean error for that transaction epoch.

The concrete `AW=64,DW=32` reader/writer witness passes semantic and canonical round trips, deterministic artifact checks, bounded simulator traces, and Direct-SV/Verilator. Validation includes 1- and 256-beat requests, invalid 0 and 257 lengths, misalignment, independent channel stalls, stable owned payloads, RLAST/RRESP/BRESP failures, reset in active phases, and ignored starts while busy. No compiler, simulator, or backend dispatches on AXI names.

This profile deliberately omits IDs, WSTRB, burst-kind, lock, cache, protection, QoS, region and user fields, multiple outstanding transactions, UB-DMA burst chunking, command-descriptor decoding, fences, CDC, or implicit adaptation. Its accepted semantic boundary is the bounded profile stated above.

2.2 AXI4-Lite, APB, and RegBus

`std.bus.reg`, `std.bus.axi_lite`, and `std.bus.apb` are ordinary imported library sources. The compiler checks structural protocol roles, typed ready/valid members, ownership, and domains; transaction state machines belong to library modules. `RegBus<32,32>` is the typed CSR boundary.

The AXI4-Lite and APB profiles use one clock/reset and one outstanding operation. `AXI4LiteToRegBus` buffers AW and W independently and joins them after both transfers; AR returns one held R response. `APBToRegBus` implements setup/access phases and holds controls stable while waiting for PREADY. Top-level aggregate endpoints expose typed public leaves (for example, `axi_aw_valid` and `apb_psel`); `connect axi -> frontend.axi` delegates a same-role endpoint to a child.

`RegBusCSRTarget` is ordinary source-authored state with RW, sticky W1C, and one-cycle pulse registers. Its response data and valid bit are held until a response transfer. Formal attachment

requires exact published observations: properties needing hidden or unsupported state are explicitly skipped or rejected under the selected policy. The AXI4-Lite/APB/RegBus profiles do not provide full AXI4 behavior. The separate `axi_burst` and `axi4` modules have their own bounded contracts and qualification limits, described above.

3 Basic GPIO, timer and serial cores

These components are ordinary ZLang sources under `std.peripheral.*`. They provide small, fixed-width interfaces. The API inventory contains exact declarations and parameter constraints.

Import	Component	Public contract
`std.peripheral.gpio`	`Gpio8`	Eight combinational input/output/OE bits; no registers or IRQ.
`std.peripheral.timer`	`PeriodicTimer32<PERIOD=1000>`	Fixed 32-bit counter, constant positive period, enable/pause and tick.
`std.peripheral.pwm`	`Pwm16<PERIOD=256,DUTY=128>`	One 16-bit edge-aligned channel with constant period and duty.
`std.peripheral.capture`	`RisingCapture32`	Rising edges, caller-supplied 32-bit timestamp, one result slot.
`std.peripheral.uart`	`UartTx8N1<DIVISOR=16>`, `UartRx8N1<DIVISOR=16>`	Eight data bits, no parity, one stop bit, LSB first, constant clock divisor.
`std.peripheral.spi`	`SpiMasterMode0<HALF_PERIOD=2>`	Eight-bit full-duplex master, MSB first, mode 0, one chip select.
`std.peripheral.i2c`	`I2cWriteByte<HALF_PERIOD=4>`	Seven-bit address, one write byte, automatic START/STOP.

Every core also has a small `RegBus<32,32>.responder` wrapper described below. There are no IRQ controllers, software drivers, DMA, vendor-compatible register profiles or runtime baud/mode configuration. The wrappers instantiate only the simple cores above; they do not hide a more capable controller.

3.1 Clock, reset and pins

Stateful cores use one rising-edge `clk` and synchronous active-high `rst`. Connect the separate ordinary input `reset_active` to the **same physical reset**. It suppresses acceptance, result validity and active pin controls during reset. Reset discards an active transfer and pending results. Test infrastructure and input vectors are not part of these source APIs.

Pin inputs named `*_sampled` and capture `level` must already belong to the sampling clock domain. Pad buffers, synchronization, setup/hold, pull-ups and board timing are caller-owned. No core adds CDC. `Gpio8` is combinational; its `reset_active` gates output enables, and its `sampled` output equals `pins_in` without sampling or synchronization.

Ready/valid payloads retain their declared binary types. Acceptance occurs on `valid && ready`; results remain stable while valid and stalled. Arithmetic and U/X diagnostics follow the language contract. No electrical, metastability, routed timing or board-capture qualification is implied by source availability or simulation. A proof of one specialization does not establish all parameter choices.

3.2 GPIO, timer and PWM

`Gpio8` drives `pins_out=data` and `pins_oe=reset_active ? 0 : oe`. The caller selects eight output-enable bits. There is no edge detector or interrupt.

`PeriodicTimer32` permits `1 <= PERIOD <= 0xffffffff`. `count` starts at zero. An enabled edge increments it, wrapping at `PERIOD-1`. `tick` is high during the enabled terminal cycle, so PERIOD=1 ticks every enabled edge. Disable pauses the count. There is no runtime period write, restart command or separate free-running timestamp.

`Pwm16` permits `1 <= PERIOD <= 65535` and `0 <= DUTY <= PERIOD`. `pin` is high while enabled and `phase < DUTY`. Disabled edges return phase to zero; reset does the same. Duty 0 is continuously low; duty equal to period is continuously high while enabled. There is no runtime configuration queue, polarity option or shadow register bank.

3.3 Rising-edge capture

`RisingCapture32` takes `enable`, `level` and `timestamp:u32`, and produces `output:rv<u32>` plus `overflow`. The first active sample after reset establishes history. Subsequent enabled low-to-high transitions capture the pre-edge timestamp. A full slot drops the new edge and sets sticky overflow; only reset clears overflow. A retiring result may be replaced on that same edge. Stalls do not change a pending timestamp. Falling edges and sequence counters are not provided.

3.4 UART 8N1

DIVISOR is a specialization constant in 4…65535, counting sampling-clock edges per bit. `UartTx8N1` takes `input:rv<bits<8>>` and emits `tx` and `busy`. Idle/reset TX is high. A byte is one low start bit, eight LSB-first data bits and one high stop bit. A new byte may be accepted on the last stop-bit edge.

`UartRx8N1` takes `rx_sampled` and produces `output:rv<UartRxBeat>`, `busy` and `overrun`. `UartRxBeat` has `data:bits<8>` and `framing_error:bit`. The receiver qualifies the start at floor(DIVISOR/2), then samples every DIVISOR clocks. A low stop reports framing error and waits for high before another frame, so a held break produces one error beat. One result slot is provided. An incoming byte cannot backpressure the pin: a full slot drops that byte and retains overrun until reset. No FIFO, flow control or alternate framing is included.

3.5 SPI mode 0

HALF_PERIOD is a constant in 1…65535. `SpiMasterMode0` takes `input:rv<bits<8>>`, returns `output:rv<bits<8>>`, and exposes `sck`, `mosi`, `cs_n`, `busy` and `miso_sampled`. CPOL=0 and CPHA=0. Each received bit is sampled on a leading edge; MOSI advances on trailing edges. A setup edge precedes chip-select assertion. Result availability is `2+16*HALF_PERIOD` edges after acceptance without reset. A result slot can retire and accept the next command on the same edge. Each command deasserts CS at completion; continuous-CS transactions and modes 1–3 are not available.

3.6 I²C single-byte write

HALF_PERIOD is a constant in 1…65535. Input payload is `I2cWriteRequest { address:u7, data:bits<8> }`. One accepted request generates START, `(address << 1) | 0`, address ACK, one data byte, data ACK and STOP. The output is one `rv<u3>` status:

Status	Meaning
0	Both bytes acknowledged and STOP completed.
1	Address NACK; STOP completed.
2	Data NACK; STOP completed.
3	Clock-stretch timeout; lines released.
4	Bus occupied at acceptance or released-data arbitration fault; lines released.

`scl_low` and `sda_low` are **low-drive enables**. A physical pad drives 0 when enabled and otherwise releases to an external pull-up. Never connect them as high-drive data values. Counting an observed high interval pauses while SCL is stretched. The fixed timeout is 1024 consecutive low samples during a requested high interval. These are sampled digital contracts, not I²C speed grade or multi-master qualification.

One command and one stable result are supported. The next command waits for result retirement; same-edge retirement/acceptance is allowed. Reads, repeated START, multi-byte transactions, target mode, runtime abort and recovery pulses are not implemented. Reset is the destructive abort mechanism.

3.7 Source example

`examples/peripherals_basic.zhl` connects the cores with explicit pin and ready/valid ports. Check it with:

```
zlang --check examples/peripherals_basic.zhl
```

The example is synthesizable source, not a host driver or automated testbench. Supply clocks, reset, pin models and traffic through the ordinary simulation or physical-wrapper workflows in the User Guide.

3.8 RegBus32 wrappers

All seven peripheral families have a responder wrapper. Registers are ordinary ZLang `csr` declarations. The shared `std.peripheral.regbus` transport captures one pre-edge response per accepted request. Effects occur on that accepting edge, once; holding a response does not repeat writes, transmit, or POP. While a response is pending, request ready is low. There is no same-edge response retirement/new-request acceptance. Response latency is one edge. Reset discards pending requests/results and resets RW words to zero.

Wrapper	32-bit aligned byte offsets
`Gpio8RegBus32`	0 INPUT (RO); 4 OUTPUT (RW); 8 OE (RW)
`PeriodicTimer32RegBus32<PERIOD=1000>`	0 COUNT (RO); 4 CONTROL (RW); 8 PERIOD (RO)
`Pwm16RegBus32<PERIOD=256, DUTY=128>`	0 PHASE (RO); 4 CONTROL (RW); 8 PERIOD (RO); 12 DUTY (RO)
`RisingCapture32RegBus32`	0 STATUS (RO); 4 CONTROL (RW); 8 DATA (RO); 12 POP (command)
`Uart8N1RegBus32<DIVISOR=16>`	0 STATUS (RO); 4 TX (command); 8 RX (RO); 12 POP (command); 16 DIVISOR (RO)
`SpiMode0RegBus32<HALF_PERIOD=2>`	0 STATUS (RO); 4 TX (command); 8 RX (RO); 12 POP (command); 16 HALF_PERIOD (RO)
`I2cWriteByteRegBus32<HALF_PERIOD=4>`	0 STATUS (RO); 4 REQUEST (command); 8 RESULT (RO); 12 POP (command); 16 HALF_PERIOD (RO); 20 STRETCH_LIMIT (RO, 1024)

CONTROL bit 0 enables timer, PWM or capture; other bits are stored and read back without effect. OUTPUT/OE store all 32 bits, but only their low eight bits connect to GPIO pins. Configuration constants remain immutable: write them and the response reports error. Counts keep their core widths and timing contracts.

RW writes merge the four WSTRB byte lanes. A zero-strobe write to a writable word succeeds without effect. Unknown/unaligned addresses and writes to RO words return error and zero data, even with zero strobes. Commands read as zero. Successful writes return the pre-write read value. All read responses are snapshots; live status may change while a response is held.

UART/SPI TX uses bits 7:0 and only WSTRB lane 0 starts a transfer. A write to other lanes has no effect. An effective TX while not ready returns error. UART RX packs data at 7:0 and framing_error at bit 8. SPI RX packs eight bits at 7:0. Reads only peek; write POP bit 0 with lane 0 enabled to consume the result. Empty effective POP returns error. A zero POP bit has no effect.

I²C REQUEST packs data at 7:0 and address at 14:8. Every nonzero-strobe REQUEST requires both low byte lanes; a partial low word returns error. Acceptance is atomic, with no intermediate address/data registers. Busy or an unconsumed result blocks the next command with an error response. RESULT contains the core status code. POP behaves as above.

Status bit assignments:

Peripheral	STATUS bits
Capture	0 valid; 1 sticky overflow
UART	0 TX ready; 1 RX valid; 2 framing error; 3 sticky overrun; 4 TX busy; 5 RX busy
SPI	0 TX ready; 1 result valid; 2 busy
I²C	0 command ready; 1 result valid; 2 busy; 5:3 result code

Payload/framing/result fields are meaningful when their valid bit is set. Overflow/overrun clear only on reset. There are no extra FIFOs or IRQs. The capture wrapper still takes a caller-owned `timestamp:u32` input.

`examples/peripherals_regbus.zhl` exposes seven independent responders and explicit pins. It does not add an address interconnect, a software driver or traffic generator. Check it with:

```
zlang --check examples/peripherals_regbus.zhl
```

4 Stream composition, CRC and fixed-point FIR

These are ordinary ZLang modules. Use `import std.stream.composition`, `import std.coding.crc` or `import std.dsp.fir`; no `implement` block or target-specific architecture is required. Stateful components use one `clock clk reset rst` domain. Connect only compatible clock/reset contracts; these components do not perform CDC.

4.1 Passive taps for vendor debug tools

Import `std.stream.composition` for `Tee<T>` and `RvTee<T>`. Both are combinational wiring modules with no declared clock/reset, registers or FIFOs. Instance ports retain the caller's clock-domain rules; no crossing is inserted.

Module	Inputs	Outputs	Contract
`Tee<type T>`	`input:T`	`output:T`, `tap:T`	Both outputs equal the input, with its exact type and stored representation.
`RvTee<type T>`	`input:rv<T>`	`output:rv<T>`, `tap_payload:T`, `tap_valid:bit`, `tap_ready:bit`, `tap_transfer:bit`	`output.payload=input.payload`, `output.valid=input.valid`, `input.ready=output.ready`; tap fields mirror the payload/handshake, and transfer is `valid & ready`.

T must be a supported ordinary wire/payload type. Width, signedness, scale, enum/union identity and aggregate layout are not converted. No reset gating, sampling, buffering or additional latency is introduced. A tap payload is the current value, including while valid is low; qualify accepted items with `tap_transfer`. U/X values retain the existing native simulation contract; `tap_transfer` is an ordinary Boolean computation, not a new diagnostic rule.

Connect `RvTee` through the functional stream and explicitly declare the debug outputs in its caller, as in the runnable example. There is no observing `ready`, so a debug consumer cannot introduce backpressure. For a single wire, `debug_value = value` is sufficient; `Tee` is optional.

Use a board-level HDL wrapper to connect those outputs to Vivado ILA or to internal preserved nets selected in Quartus Signal Tap. The vendor workflow explains bit order, clocks, preservation and qualification limits. The stdlib modules do not create a vendor core, keep a signal from synthesis optimization, infer CDC or guarantee routed timing. `RvFork2` below remains a buffered lossless broadcast; it is not a passive monitor.

4.2 Broadcast and join

`RvFork2<T>` has `input:rv<T>` and `output0/output1:rv<T>`. It stores one payload and remembers delivery to each consumer independently. A consumer that accepted early does not receive the item again while the other stalls. The last delivery can retire together with acceptance of a replacement item. There is no empty-slot combinational bypass.

`RvJoin2<A,B>` has `input0:rv<A>`, `input1:rv<B>` and `output:rv<StreamPair<A,B>>`, whose fields are `left:A` and `right:B`. Each input has its own slot. The k-th accepted element of input0 pairs with the k-th accepted element of input1, regardless of their arrival cycles. Both slots may refill on output retirement. Stalls hold the pair unchanged. Reset discards pending items. Payload types remain exact, including aggregates; there is no implicit `default<T>`, adapter or protocol conversion.

4.3 CRC-32 byte frames

`crc_step_reflected<W>(state:bits<W>, data_bit:bit, polynomial:bits<W>)` returns an exact W-bit raw state. Specify W explicitly, for example `crc_step_reflected<W=32>(state, data_bit, polynomial)`. W must be positive. The function does not initialize, finalize or reverse a word.

`RvCrc32Byte` accepts `input:rv<CrcByteBeat>`; `CrcByteBeat` contains `data:bits<8>` and `last:bit`. Its `output:rv<bits<32>>` produces one checksum for each accepted last byte. Input bytes are processed LSB-first with reflected polynomial `0xEDB88320`, raw initialization `0xFFFFFFFF` and final XOR `0xFFFFFFFF`. Thus `123456789` gives `0xCBF43926`. This is the CRC convention in RFC 1952's sample code.

Within a frame the engine can accept one byte per cycle. A pending result blocks the next frame, except when that result retires on the same edge as the next frame's first byte. Reset drops the partial frame and pending result. There is no empty-frame transaction, byte-enable field or automatic framing.

The framed broadcast example sends each byte both to the CRC engine and to an independently backpressured mirror:

```
.venv/bin/zlang sim examples/framed_crc.zhl --top FramedCrc \
  --events examples/framed_crc.events.jsonl --trace build/framed_crc.fst --json
.venv/bin/zlang examples/framed_crc.zhl --top FramedCrc \
  --systemverilog build/FramedCrc.sv --verilator-lint
```

Its checked-in events send `abc`; the checksum is `0x352441C2`.

4.4 Parallel FIR

`RvFirFixedExact<SW,SF,CW,CF,N,coefficients>` accepts `input:rv<fixed<SW,SF>>` and returns `output:rv<fixed<SW+CW+ceil_log2(N),SF+CF>>`. Use named specialization arguments. The checked derived parameter `H=N-1` is only the history dimension; leave it at its default.

Coefficients are an immutable compile-time `vec<N,fixed<CW,CF>>`, not runtime ports. Declare a typed local vector and pass its name as `coefficients=taps`. `coefficients[0]` multiplies the current accepted sample, and subsequent coefficients multiply earlier accepted samples. Products are full-width; the existing `dot` supplies its deterministic balanced exact reduction.

N must be at least one. N=1 has no history storage. Reset zeroes history and clears output validity. The output is one buffered stage: one-cycle unstalled latency and II=1. A full stalled output blocks acceptance and leaves history unchanged. Retirement and next acceptance can occur on the same edge.

`RvFirFixedNearestEvenSaturate<SW,SF,CW,CF,N,OW,OF,coefficients>` uses that exact core, then rounds once to `fixed<OW,OF>` with nearest-even rounding and saturation. It adds no pipeline cycle. Other numerical policies require an explicit caller-owned conversion after the exact core. There is no coefficient reload, decimation, interpolation, folded scheduler or implicit DSP mapping. For those explicit library variants, see reloadable/folded FIR and multirate filters; the immutable modules here retain their existing interfaces and one-stage behavior.

Run the eight-tap example:

```
.venv/bin/zlang sim examples/stream_fir8.zhl --top StreamFir8 \
  --events examples/stream_fir8.events.jsonl --trace build/stream_fir8.fst --json
.venv/bin/zlang examples/stream_fir8.zhl --top StreamFir8 \
  --systemverilog build/StreamFir8.sv --verilator-lint
```

Simulation integer values are raw fixed-point bits: with SF=12, `4096` represents 1.0. The example sends that impulse followed by zero samples. Accepted output raw values are `128,384,640,896,896,640,384,128,0`. When adapting events to introduce stalls, hold each valid input payload until its transfer; one JSONL record is a simulation event, not necessarily a transfer.

Source availability and runnable examples do not establish numerical or formal qualification for every specialization. The public formal coverage inventory distinguishes available public checks from components without published qualification evidence. Any proof claim must identify its specialization, property and evidence. Synthesis cell counts are not routed timing evidence.

4.5 Community hardware observation boundary

`Tee<T>` and `RvTee<T>` in `std.stream.composition` provide explicit passive observation. Debug outputs connect to vendor-owned ILA/Signal Tap in a board-level SystemVerilog wrapper; they are internal nets, not extra FPGA pins when wired inside that wrapper. A plain `debug_value = value` assignment is also sufficient. See the vendor example. Automatic probe insertion, capture generation and scan/ATPG flows are not Community capabilities. No new syntax or arbitrary sequential `extern` contract is implied.

5 Reloadable and folded FIR

These source components use the ordinary single-domain ready/valid contract. The immutable FIR keeps its existing interface; loading and commit are exposed only by the separate reloadable variants. Source availability does not imply that a previously built wheel or extension contains this update.

Import `std.dsp.fir_reloadable`. `RvFirReloadableExact` keeps full signed products and the balanced dot result. Its output is `fixed<SW+CW+ceil_log2(N),SF+CF>`. `RvFirReloadableNearestEvenSaturate` adds `OW,OF` and one final nearest-even/saturating conversion, without another stage.

Both use ready/valid sample input/output and a ready/valid coefficient `reload` input. Load exactly N coefficients, tap 0 first, into the inactive bank. Hold `commit` until `commit_ready` is high at a clock edge. A full requested commit blocks new samples until the old pending output retires. The switch edge accepts neither a sample nor a coefficient. Loading the last coefficient and committing require separate edges. Processing may continue while loading. Every result uses one complete coefficient bank, never a mixture.

Reset restores both banks to `initial_coefficients`, clears sample history, the pending result and partial reload. Commit preserves sample history. Unstalled sample latency is one edge, II=1; output backpressure stalls input acceptance and history. There is one pending result, not an unbounded queue.

Run the example from the repository root after `make install`:

```
.venv/bin/zlang sim examples/fir_reload.zhl --top ReloadableFir \
  --events examples/fir_reload.events.jsonl --json
```

Event payloads are raw fixed-point integers: sample 8 means 1.0 for SF=3; coefficient 4 means 1.0 for CF=2. The example loads `[1,-1,0.5]`, stalls an old result, then commits and resumes with the existing sample history.

This is an explicitly parallel library datapath. It does not enable automatic scheduling, new implementation intents, or coefficient reload in other FIRs. This contract does not claim universal physical timing or one DSP per tap.

5.1 Explicit one-MAC variants

Import `std.dsp.fir_folded`. `RvFirFoldedExact` takes immutable `coefficients`; `RvFirFoldedReloadableExact` takes `initial_coefficients` and the same reload/ commit interface described above. Both admit N=1…32. Unstalled latency and II are N edges: acceptance performs tap 0, then one tap is accumulated per edge. N=1 is a one-stage FIR with no history array. The full accepted window is held while busy; samples from different windows never overlap. Input can retire/refill on the edge that consumes a pending result. Stalls hold that result; they do not advance sample history. Reload may proceed while busy, but commit waits for completion and retirement of the old result.

Output width/scaling is the same exact format as the parallel FIR. Optional `RvFirFoldedNearestEvenSaturate` and `RvFirFoldedReloadableNearestEvenSaturate` add `OW,OF` and one final conversion, without another register stage. The serial accumulator never rounds products. One logical multiplier does not establish a physical DSP count. Measure the actual specialization and target mapping; widths, controls and accumulator binding all affect synthesis. No generic one-DSP or routed-timing guarantee follows from this source architecture.

```
.venv/bin/zlang sim examples/fir_folded.zhl --top FoldedFir \
  --events examples/fir_folded.events.jsonl --json
```

6 Oscillators, CORDIC, multirate filters and coding

These source modules use the ordinary ready/valid contract. Input samples and all associated controls are consumed only on `input.transfer`. Except for the multi-stage CORDIC pipelines, two-stage interpolated NCO and interpolator phase batches described below, there is one pending result slot. A stalled result holds its value and stops new acceptance; same-edge retirement and acceptance are allowed. Reset discards pending work. No component below changes clock domains or inserts a hidden rate converter.

6.1 Phase accumulator and ROM sine/cosine

Import `std.dsp.nco`.

- `RvPhaseAccumulator<PW,INITIAL=0>` accepts `rv<uint<PW>>` increments and emits the phase **before** that increment, wrapping modulo 2^PW.
- `RvNcoSinCos<PW,AW,OW,OF,INITIAL=0>` emits `rv<SinCos<fixed<OW,OF>>>`, with fields `sine` and `cosine`. One full turn is 2^PW phase units. The top AW bits address a paired synchronous ROM of 2^AW entries. Table values use nearest-even rounding and saturation. No table interpolation or dither is performed. PW>=AW>=2; 2^AW<=4096; 0<=OF<OW.

Both have one-edge latency and II=1 when unstalled. With a constant increment `d`, the discrete frequency is `d/2^PW` turns per **accepted sample**. Only when samples are accepted every clock may that be converted to `d*f_clk/2^PW` Hz. Reset restores INITIAL; the first requested sample uses that phase.

```
.venv/bin/zlang sim examples/nco.zhl --top Nco \
  --events examples/nco.events.jsonl --trace build/nco.fst --json
```

6.2 Rotation-only CORDIC sine/cosine

Import `std.dsp.cordic`; instantiate `RvCordicSinCos<P=16,N=P-2>`. The input is `rv<sint<P>>`, not a phase increment. Signed raw angle `a` means `a/2^P` turns: at P=16, 0 is zero, 16384 is +90°, -16384 is -90°, and -32768 is -180°. The output is `rv<SinCos<fixed<16,14>>>`, with `sine` and `cosine` fields. In simulation JSON, fixed-point outputs are signed raw integers; divide by 16384 to recover their numerical values.

The supported profiles are P=8, 12 or 16, with 1<=N<=P-2. OW=16, OF=14 and G=4 are fixed for this first profile set; leave F/W/STAGES derived parameters at their defaults. Each accepted angle undergoes half-turn range reduction, N signed shift/add rotations, and a final nearest-even/saturating conversion. Intermediate coordinates retain four extra fractional bits; their signed 21-bit range is sufficient for every accepted angle. There is no early exit at a zero residual angle. Reviewed Q48 angle/inverse-gain constants elaborate to constants, not a runtime ROM or a new `atan` intrinsic.

Unstalled latency is **N+2 rising edges**, including the accepting edge; II=1. There are N+2 valid slots, not just one pending sample. When a valid output is stalled, the **whole** pipeline freezes and input.ready is zero; otherwise samples and bubbles advance together. Reset clears all pipeline data/valids and discards in-flight work. A producer must hold its payload while valid and not ready, as for other ready/valid components.

The output is a bounded approximation, not correctly rounded mathematical sin/cos. Conservative per-component absolute error bounds for default N are 0.07191 (P=8,N=6), 0.006778 (P=12,N=10), and 0.000697 (P=16,N=14), including angle-table rounding, residual rotation, coordinate shifts and final rounding. Smaller N increases angular error. This rotation component does not perform vectoring or arbitrary x/y rotation; iteration count is compile-time.

```
.venv/bin/zlang sim examples/cordic.zhl --top Cordic \
  --events examples/cordic.events.jsonl --trace build/cordic.fst --json
```

The supplied schedule requests zero, ±quarter-turn, negative half-turn and eighth-turn angles, then drains the pipeline. See the CORDIC numerical structure for background; that vendor implementation is not this component's contract.

6.3 Vectoring, atan2 and magnitude

Import `std.dsp.cordic_vectoring`. `RvCordicVectoring<SW=16,SF=SW-2,P=SW>` accepts `rv<CartesianFixed<SW,SF>> (x,y)` and produces `rv<PolarFixed<P,SW,SF>> (phase,magnitude,angle_defined)`. The admitted profiles are SW=P=8/12/16, SF=SW-2, N=P-2; leave derived parameters at defaults. `RvAtan2` has the same input and timing and returns only phase/angle_defined.

Phase is signed binary turns: raw phase / 2^P, in [-half-turn,half-turn). The negative x axis is exactly the negative half-turn code. Magnitude is unsigned `ufixed<SW,SF>`; simulator values are raw integers divided by 2^SF. At (0,0), phase and magnitude are zero and angle_defined=0. Every other input has angle_defined=1. Axis values are exact. Non-axis values are bounded approximations, not a claim of correct rounding.

Leading-bit normalization gives tiny nonzero inputs the same angular resolution as larger ones. Coordinates widen before quadrant negation, including the minimum signed input. Four guard fractional bits, finite-gain compensation and one final nearest-even/saturating magnitude conversion are explicit. Phase rounds then wraps modulo one turn; distance across the branch cut is circular, not a raw signed subtraction.

Conservative universal errors, rounded upward in raw output LSBs:

Input/profile	Circular phase error	Magnitude error
fixed<8,6>,P=8	3.870185	5.530681
fixed<12,10>,P=12	5.268837	6.158203
fixed<16,14>,P=16	6.667078	8.014771

Unstalled latency is N+3 rising edges including acceptance; II=1. A pending stalled output freezes the entire pipeline and stops input acceptance. Reset discards all in-flight work. No runtime iteration count, general rotation or hyperbolic CORDIC is provided.

6.4 CIC decimator

Import `std.dsp.cic`; instantiate `RvCicDecimator<SW,IW,R,N,M=1>`. Inputs are signed SW-bit integers, outputs signed IW-bit integers. R is the integer decimation rate, N the number of integrator/comb stages, and M the comb differential delay. All must be positive, and IW>=SW.

Every addition/subtraction explicitly wraps at IW bits. No internal saturation, rounding or gain compensation is applied. DC gain is `(R*M)^N`. For a bounded SW-bit input, `SW + ceil_log2((R*M)^N)` is a conservative full-output width; integrator wrap itself is normal CIC operation. Choosing less output headroom is an intentional modular numerical contract, not saturation protection.

The first result includes accepted samples 0 through R-1. Integrators advance on every acceptance; comb delays only on each R-th acceptance. Reset clears both histories and any incomplete decimation group. R=1 and M>1 are supported.

```
.venv/bin/zlang sim examples/multirate_filters.zhl --top CicDecimator \
  --events examples/multirate_filters.events.jsonl --trace build/cic.fst --json
```

6.5 Polyphase FIR decimator

Import `std.dsp.polyphase`. `RvPolyphaseFirDecimatorExact` takes named SW/SF/CW/CF/R/K and an immutable `coefficients` vector of length N=R*K. Leave derived N/H/PW parameters at their defaults. Phase p owns coefficients p+Rk.* After accepted sample t=R-1, 2R-1, ... the result is `sum(coefficients[j] * x[t-j])`, with zero history after reset. Coefficient zero multiplies the current accepted sample. Stalls do not advance history or phase.

Existing exact `dot` computes each phase; the final balanced sum retains full precision. Exact output type is `fixed<SW+CW+ceil_log2(N),SF+CF>`. Redundant tree guard bits are explicitly removed without dropping any representable sum. `RvPolyphaseFirDecimatorNearestEvenSaturate` additionally takes OW/OF and performs one final conversion; it adds no pipeline stage. This decimator has no coefficient reload, automatic symmetry or folded scheduling; interpolation is a

separate component described below. The bank is parallel (R*K products); phase decomposition alone is not a claim that synthesis uses fewer multipliers than a parallel FIR.

```
.venv/bin/zlang sim examples/multirate_filters.zhl --top PolyphaseFir \
  --events examples/multirate_filters.events.jsonl --trace build/polyphase.fst --json
```

6.6 Extracted 24-bit coding blocks

`std.coding.scrambler` owns the x^7+x^4+1 24-bit kernel previously local to the 802.11a example. `coding_scramble7_24(word,state,clear_mask)` returns data and next state. DATA bit zero is processed first. Clear-mask bits force the final scrambled output bits to zero, but do not stop state progression. `RvScrambler7_24<SEED=0x4b>` accepts `ScramblerWord24 { data, restart, clear_mask }`; restart selects SEED before processing that word. SEED is 1..127.

`std.coding.convolutional` owns the existing rate-1/2 K=7 kernel. `ieee_convolutional_encode24(word,history)` returns encoded bits and history. Source bit zero is first; the first transmitted pair occupies the two most significant output bits, G0 before G1. `RvConvolutionalK7_24` accepts `ConvolutionalWord24 { data, restart, finish }`. Restart zeros history before the word; finish clears history after it. Finish does **not** insert tail bits. No puncturing or rate policy is implied. Wi-Fi metadata/framing remains in the project, which imports these same authoritative pure kernels.

```
.venv/bin/zlang sim examples/coding_blocks.zhl --top Scrambler24 \
  --events examples/scrambler24.events.jsonl --trace build/scrambler.fst --json
.venv/bin/zlang sim examples/coding_blocks.zhl --top ConvolutionalEncoder24 \
  --events examples/convolutional24.events.jsonl --trace build/convolutional.fst --json
```

Background: phase-truncated DDS and CIC growth/decimation. These explain the numerical structures, not resource or timing guarantees for these ZLang components.

6.7 Polyphase FIR interpolation

`std.dsp.polyphase` also provides `RvPolyphaseFirInterpolatorExact` and `RvPolyphaseFirInterpolatorNearestEvenSaturate`. For R phases, K taps per phase, and N=R*K coefficients, output p for input n is `sum_k coefficients[p+R*k]*x[n-k]`, p=0…R−1. Coefficient 0 multiplies the current input in phase 0. This is an ordinary FIR applied to a zero-stuffed input, not a hidden gain of R. The caller supplies any desired gain through the coefficients. Padding is trailing zeros only, so prototype delay is kept.

All phase sums are stored in parallel at acceptance. The first is available after one edge; subsequent phases retire one per edge. Unstalled input II is R. A new input may replace the batch on retirement of its final phase. Stalls hold payload, phase and accepted-sample history. Reset discards the batch and clears history. Exact output is `fixed<SW+CW+ceil_log2(K),SF+CF>`; the optional OW/OF wrapper makes one final nearest-even/saturating conversion, with no extra stage. This component has immutable coefficients and no folded datapath.

Run `examples/fir_interpolate.zhl` (top `InterpolatingFir`) with `examples/fir_interpolate.events.jsonl` through `zlang sim --events`.

6.8 Fractionally interpolated NCO

`std.dsp.nco_linear` provides `RvNcoSinCosLinear<PW,AW,OW,OF,INITIAL=0>`. Each accepted phase increment samples the pre-increment phase. Its upper AW bits select a ROM segment; the remaining bits interpolate between adjacent sine/cosine endpoints, including the wrap to segment zero. The endpoint fraction has OF+4 bits. Only the final output conversion rounds nearest-even and saturates. The ordinary ROM NCO remains a separate, unchanged component.

Latency is two edges, II=1. Global backpressure holds both stages and phase; reset drops pending samples and restores INITIAL. PW=AW gives fraction zero but still has two-edge latency. AW must be at least two, PW at least AW, the table at most 4096 entries, and 0 <= OF < OW with

OW >= 2. Leave derived parameters at their defaults. Interpolation does not promise correct rounding of the mathematical sine/cosine, one-LSB accuracy, or measured SFDR.

```
.venv/bin/zlang sim examples/nco_linear.zhl --top LinearNco \
  --events examples/nco.events.jsonl --trace build/nco-linear.fst --json
```

6.9 CIC interpolation

`RvCicInterpolator<SW,IW,R,N,M=1>` in `std.dsp.cic` accepts signed SW-bit samples. Its low-rate combs feed zero insertion and high-rate integrators. Every addition/subtraction wraps explicitly at IW bits, with IW >= SW; no rounding, saturation or gain compensation is implicit. The nominal DC gain is `(R*M)^N/R`, including R=1 and M>1. Choose IW for the required headroom.

Each accepted input produces R ordered outputs. The first appears after one edge; unstalled input II is R. Retirement of the final phase may accept the next input on the same edge. Stall holds the output and all advancing state. Reset clears comb/integrator history and drops an unfinished group.

```
.venv/bin/zlang sim examples/cic_interpolate.zhl --top CicInterpolatorExample \
  --events examples/cic_interpolate.events.jsonl --trace build/cic-interpolate.fst --json
```

6.10 Halfband ×2

`std.dsp.halfband` provides `RvHalfbandDecimatorExact` and `RvHalfbandInterpolatorExact`, plus corresponding `NearestEvenSaturate` wrappers taking OW/OF. Both take SW/SF/CW/CF/N and immutable coefficients. N is odd and at least three. Specialization rejects asymmetric prototypes or nonzero coefficients at the center's parity, except the center itself. The caller supplies the center coefficient and gain explicitly.

Exact output is `fixed<SW+CW+ceil_log2(N),SF+CF>`. Symmetric sample pairs are added at SW+1 bits before multiplication; zero taps are omitted. There is no intermediate quantization. The optional wrapper makes one final nearest-even, saturating conversion with no additional stage.

The decimator emits the full FIR result after every second accepted input. The interpolator is the same prototype applied to zero-stuffed input and emits phase 0 then phase 1 per accepted input, with no hidden gain x2. Its first result appears after one edge and unstalled input II is two. Both hold pending results under stall and clear history/results on reset. Any padding for comparison with an even-length polyphase reference is trailing zero only.

Simulation JSONL payloads use raw integers: `8` represents `1.0` at `fixed<8,3>`. Divide an exact result's raw value by `2^(SF+CF)` to recover its numerical value; the OW/OF wrapper instead uses `2^OF`.

```
.venv/bin/zlang sim examples/halfband.zhl --top HalfbandDecimatorExample \
  --events examples/halfband.events.jsonl --trace build/halfband-decimate.fst --json
.venv/bin/zlang sim examples/halfband.zhl --top HalfbandInterpolatorExample \
  --events examples/halfband.events.jsonl --trace build/halfband-interpolate.fst --json
```

7 Extended-Hamming SECDED

Import `std.coding.secded` for `secded_encode<K>(data)`, `secded_decode<K>(received)`, `RvSecdedEncode<K>` and `RvSecdedDecode<K>`. The stream modules admit payload widths K=1…64. R is the smallest integer with `2^R >= K+R+1`; the codeword width is K+R+1. K=32 produces 39 bits, and K=64 produces 72 bits.

Hamming positions start at 1. Positions 1,2,4,8,… contain parity bits; remaining positions contain data in ascending order, least-significant data bit first. The most-significant codeword bit is overall even parity.

Decode returns `SecdedDecoded<K,R>` with `data`, `syndrome`, `status` and `data_valid`. Status is `SecdedStatus.clean`, `corrected` or `detected_uncorrectable`. A single corrupt bit is corrected, including a Hamming or overall parity bit. Every pair of corrupt bits is detected. On an uncorrectable result, `data_valid=0`: `data` is not trustworthy. The syndrome is a diagnostic position, not a safe arbitrary array index. Out-of-layout syndromes in shortened codes never trigger indexed correction. For three or more corrupt bits, miscorrection is possible; neither correction nor detection is guaranteed.

Both stream modules have one buffered result stage, one-edge unstalled latency and II=1. `input.valid && input.ready` accepts a word; `output.valid && output.ready` retires it. A stalled result is held stable. Retirement and replacement may occur on the same edge. Synchronous reset clears validity and discards the pending word.

7.1 Run the 39/32 example

After `make install`, run from the repository root:

```
.venv/bin/zlang sim examples/secded.zhl --top Secded32Decode \
  --events examples/secded32.events.jsonl \
  --trace build/secded32.fst --json
```

The event file sends codeword zero, a single parity-bit error (`1`), then two parity-bit errors (`3`), including a stalled output. Expected statuses are clean, corrected and detected_uncorrectable. Inspect `output.valid` before interpreting a result, and `output.payload.data_valid` before using its data. `Secded32Encode` in the same source accepts bits<32> and produces bits<39>; it can be connected to the decoder using ordinary ready/valid composition.

Native U/X remains diagnostic: unresolved accepted data and stateful controls are subject to the current strict-uninitialized observation contract. SECDED does not turn an unknown simulator value into a reliable binary codeword. There are no ECC RAM, scrubbing or Hsiao wrappers in this module.

8 Farrow fractional delay

`std.dsp.farrow` provides `RvFarrowFixedExact`, `RvFarrowFixedNearestEvenSaturate`, `RvFarrowLagrangeLinear` and `RvFarrowLagrangeCubic`. These are fractional-delay filters, not complete sample-rate converters: the caller supplies each sample and its delay value.

Input payload is `FarrowSample<SW,SF,MF>`:

- `sample:fixed<SW,SF>`;
- `mu_raw:uint<MF>`, representing μ=mu_raw/2^MF, with 0≤μ<1.

The exact core admits M=0...3, K=1...8 and MF≥1. The immutable matrix has M+1 rows of K `fixed<CW,CF>` coefficients. Row j computes `v_j = sum(c[j,k]*x[n-k])`; Horner evaluation computes `v_0 + μ*(v_1 + … + μ*v_M)`. Tap zero uses the current accepted sample. History changes only on input transfer.

With B=SW+CW+ceil_log2(K) and F=SF+CF, the exact output is `fixed<B+M*(MF+2),F+M*MF>`. Subfilter products/sums and every Horner stage retain their full typed widths. Scale alignment appends zero raw bits; it does not discard fractional information. Exactness refers to the stored binary coefficients, not an ideal rational filter.

There is one registered subfilter stage followed by M registered Horner stages: unstalled latency=M+1 edges, II=1. Samples, μ and validity move together. Output backpressure freezes the entire pipeline. Input is accepted when `input.valid && input.ready`; reset clears history and all in-flight validity. The final-quantized variant applies nearest-even/saturation once, after the last register, without another stage.

8.1 Lagrange profiles

The convenience wrappers use immutable `fixed<18,16>` coefficient matrices. `farrow_lagrange_linear_q16()` has exact coefficients for delay μ. `farrow_lagrange_cubic_q16()` stores nearest-even binary approximations of the cubic Lagrange coefficients for delay 1+μ. That one-sample algorithmic offset is separate from the four-edge pipeline latency.

`RvFarrowLagrangeLinear<SW,SF,MF>` returns `fixed<SW+MF+21,SF+MF+16>`; the cubic wrapper returns `fixed<SW+3*MF+26,SF+3*MF+16>`. For sample fixed<16,14>, MF=16, cubic exact output is fixed<90,78>. For a smaller output, pass the coefficient helper to `RvFarrowFixedNearestEvenSaturate` with explicit OW/OF, as in the example.

For arbitrary fixed<16,14> input samples and any representable μ, the cubic binary-coefficient approximation differs from the ideal cubic Lagrange polynomial by at most 1/24576 in sample units. This follows from the exact rational coefficient-error sum 1/49152 and |sample|≤2; it is a conservative uniform bound, not a correct-rounding claim. A final fixed<16,14> nearest-even/saturating conversion adds at most half an output LSB relative to the clipped exact result. Error relative to an *unclipped* ideal result also includes any intentional saturation. No all-band ideal-delay accuracy, SFDR or sample-rate-controller guarantee follows from this certificate.

8.2 Run

```
.venv/bin/zlang sim examples/farrow.zhl --top FarrowCubic \
  --events examples/farrow.events.jsonl --trace build/farrow.fst --json
```

The JSONL sample integers are raw fixed-point values: 16384 means 1.0 at SF=14. `mu_raw=32768` means μ=0.5 at MF=16. The example changes μ per accepted sample, stalls the first result, then drains the pipeline. Only output transfers count as delivered results. Coefficient reload, filter design and resampling control are outside this module.

9 DVB Reed–Solomon coding

`std.coding.rs_dvb` provides `RvRsDvb204188Encode`, `rs_dvb_parity_step` and `rs_dvb_syndrome_step`. `std.coding.rs_dvb_decode` provides `RvRsDvb204188Decode` with the same byte/status record types. The separate import keeps encoder-only designs independent of decoder arithmetic and control declarations. `std.coding.gf256` supplies polynomial-basis `gf256_mul` and `gf256_inverse`; inversion returns `Gf256Inverse {value,valid}`. Zero returns value=0, valid=0 because it has no multiplicative inverse.

The profile is systematic RS(204,188), shortened from RS(255,239) by 51 leading zero symbols. It uses GF(256), primitive polynomial 0x11D, primitive element 0x02, and generator roots $\alpha^0 \ldots \alpha^{15}$. This is the outer-code profile in ETSI EN 300 744 §4.3.2, not RS(255,223) or CCSDS. No energy dispersal, MPEG parsing or interleaving is performed.

Wire order is highest polynomial coefficient first. The first 188 output bytes exactly reproduce the input, followed by 16 parity bytes. For input bytes 0,1,…,187 the parity suffix is `31 1d 78 d6 c8 60 f8 78 b7 18 9f 1a 54 96 1d 5f`. The shortening zeros are never transmitted; appending zeros after the data would describe a different computation.

9.1 Encoder

Input and output payloads are `RsByteBeat {data:bits<8>, first:bit, last:bit}`. Only ready/valid transfers advance a frame. Set first on byte zero and last on byte 187. Both must be clear on intermediate input bytes. The entire 188-byte frame is stored and checked before any payload is emitted.

`status:rv<RsFrameStatus>` must retire before output begins. The record has `status:RsStatus` and `correction_count:uint<4>`. Encoder status is clean or malformed; correction_count is zero. Corrected/detected_uncorrectable are decoder statuses, not encoder error-detection guarantees. A malformed frame emits status only, never payload.

Missing first, an unexpected first, early last or a missing last at the 188-byte boundary is malformed. If the bad packet was unterminated, input is discarded after status retirement until last or a new first marker. The byte that triggered the error has already been consumed; it is not replayed as another frame. A new first marker encountered while discarding can start a fresh packet.

There is one frame in flight. Input is blocked during status wait and all 204 output bytes. Output first/last mark bytes zero/203, and stalled payload is stable. Reset clears validity, framing, parity and the pending packet. With continuous ready/valid and no reset, collection, status and drain take 188+1+204=393 edges, without overlap. Storage is an explicit 188-byte register shift buffer; this implementation does not promise inferred BRAM.

9.2 Decoder

Send exactly 204 received bytes with first on byte zero and last on byte203. The decoder stores the full codeword before solving or producing output. It corrects up to eight erroneous **symbols**, whether in data or parity; multiple erroneous bits in one byte count as one symbol. Erasures are not supported.

Status must transfer before any payload. A clean/corrected status is followed by exactly188 data bytes, with first/last on bytes0/187. The correction count is valid only for corrected status; other statuses return zero. A malformed or detected_uncorrectable result has no payload. Reset discards collection, solve and pending status/output. Output stalls preserve data and markers.

Malformed-marker recovery follows the encoder rules above, with the length boundary at204 rather than188. No next frame is accepted during solve, status wait or payload drain. Keep input valid/payload stable until ready accepts it.

The decoder uses Berlekamp–Massey, a complete255-position Chien search and Forney correction. It rejects invalid degree/root counts, virtual shortening positions and zero denominators, then recomputes all syndromes. Beyond eight symbol errors, miscorrection to another valid codeword remains possible: even zero final syndromes cannot guarantee the original message was recovered.

This implementation explicitly shares one variable GF multiplier across BM updates and polynomial evaluation. It also has a combinational inverse lookup and constant-coefficient syndrome arithmetic. It is not a one-cycle decoder. With continuous input valid and status/output ready, and no reset:

Successful frame	Edges from first input transfer through last output transfer
Clean	394
Corrected, e=1…8 symbols	6341 + 36·e

Stalls and input bubbles add edges; unsuccessful frames can stop earlier. Storage is204 packet bytes plus polynomial/control registers, not inferred BRAM. The module makes no routed Fmax guarantee.

9.3 Run

```
.venv/bin/zlang sim examples/rs_dvb.zhl --top DvbRsEncoder \
  --events examples/rs_dvb.events.jsonl --trace build/rs_dvb.fst --json
```

The example sends bytes 0…187, accepts clean status, stalls the first output byte, then drains the codeword. Consume only output transfers. This module does not decode or assess channel errors; a clean *encoder* status means the input framing was valid, not that a received packet passed a decoder.

The decoder example uses the same ramp codeword, with two corrupted symbols (the first data byte and last parity byte). It returns corrected status with count2 and the original bytes0…187:

```
.venv/bin/zlang sim examples/rs_dvb_decode.zhl --top DvbRsDecoder \
  --events examples/rs_dvb_decode.events.jsonl --trace build/rs_dvb_decode.fst --json
```

Observe status/output **transfers**, not only their valid values. The event file includes the complete solve/drain time; stopping just after the final received byte leaves the iterative decoder busy.

10 Framed K=7 Viterbi decoding

`std.coding.viterbi_k7` contains one rate-1/2 decoder engine: `RvViterbiK7Soft<MAX_BITS=4096>`. `RvViterbiK7Hard` converts each received bit to soft value0 or7 and uses that same engine. Capacity is1…4096 decoded bits; a frame may finish before capacity is reached.

10.1 Inputs and decision rule

Soft payload is `ViterbiSoftPair {q0:u3,q1:u3,first:bit,last:bit}`. Zero means strongest zero, seven strongest one. For an expected bit0 the cost is q; for bit1 it is7−q. Costs of the two received bits are added. Hard payload is `ViterbiHardPair {pair:bits<2>,first:bit,last:bit}`; pair[1] is G0 and pair[0] is G1.

The trellis matches `ieee_convolutional_encode24`: source bit0 first, seven-bit window `{new_bit,history}`, masks0x5b/0x79 and next history `{new_bit,history[5:1]}`. The initial state is zero. The final state is the minimum-metric reachable state, not necessarily zero. Encoder finish clears history after a word; it does not transmit tail bits.

Equal-cost paths select the smaller predecessor; equal final metrics select the smaller state number. Reachability is separate from the metric, whose width is `ceil_log2(14*MAX_BITS+1)`. There is no metric saturation or numerical sentinel. The result is a minimum-metric path, not a promise to detect an incorrectly decoded message.

10.2 Frames, storage and timing

Set first on the first accepted pair and last on the final accepted pair. `status:rv<ViterbiFrameStatus<MW,CW>>` reports malformed, decoded_bits and path_metric; `MW=ceil_log2(14*MAX_BITS+1)`, `CW=index_width(MAX_BITS+1)`. Malformed frames report zero length/metric and have no payload. Missing or unexpected first, or reaching capacity without last, is malformed. An unterminated bad frame is discarded until last or a new first marker after status retirement. Reset abandons the current frame and all pending output.

Successful status must retire before `output:rv<ViterbiDecodedBit>` begins. Its payload has data, first and last. The full survivor history is in RAM; after end-of-frame the decoder traces back completely, stores decoded bits, then emits them in the original order. No new frame overlaps traceback/drain. Payload and markers remain stable during output stall.

Input accepts one pair per edge while receiving. Traceback takes two edges per bit; drain takes two edges per bit without stalls. Ideal frame occupation is5·N+1 edges including N input transfers, traceback, status and drain. This is a framed implementation, not a continuous fixed-latency decoder. Neither puncturing, tail-biting nor tail insertion/removal is performed.

10.3 Existing 48-bit encoder words

`RvViterbiK7Word48` accepts `ViterbiK7Word48 {data:bits<48>,first:bit,last:bit}`. It sends bits47:46 first, then45:44, and so on. Markers belong to the word stream and must be supplied explicitly; they cannot be recovered from code bits. MAX_BITS is24…4096. Whole-word frame lengths are multiples of24, so the largest supported whole-word frame with default capacity is4080 bits.

10.4 Run a hard-decision example

```
.venv/bin/zlang sim examples/viterbi.zhl --top ViterbiFrame \
  --events examples/viterbi.events.jsonl --trace build/viterbi.fst --json
```

The stimulus carries24 source bits from word0x123456, bit0 first. Advance through input collection, traceback, status and the decoded bit stream in the debugger. A live waveform is event-based, not a hardware timing estimate.

11 Standard library recipes

Use the API inventory to find an import and exact signature, then read its component contract before choosing widths and buffering. The examples below are ordinary source programs, not compiler-recognized application names.

11.1 Stream composition and CRC

Start with the framed-CRC example. `RvFork2` delivers each accepted payload exactly once to each branch; `RvJoin2` pairs accepted elements by sequence, not by the cycle when they arrive. Connect one clock/reset domain and keep output payload stable under backpressure. The stream/CRC contract specifies simultaneous retire/refill and CRC initialization, polynomial and final XOR.

11.2 Choosing a FIR

Choose immutable parallel FIR for II=1; choose the explicit folded datapath when one logical MAC and II=N match the application. One logical multiplier is not a promise of one DSP for arbitrary widths. Reloadable wrappers use two banks and an explicit atomic commit; do not update live coefficients. See FIR contracts and the examples linked there.

Exact cores retain full product/reduction precision. Quantized wrappers apply one final nearest-even/saturating conversion. Do not insert conversions between products unless the application intentionally changes its numerical contract.

11.3 Changing the sample rate or delay

Use polyphase interpolation/decimation for a supplied FIR prototype and fixed integer rate. CIC uses an explicitly chosen modular internal width and no hidden gain correction. Halfband requires checked symmetric coefficients and alternating zeros. Multirate contracts define coefficient order and accepted-sample timing.

Farrow changes fractional delay, not the input sampling schedule. Each beat carries its own `mu_raw`; the Farrow contract distinguishes stored-coefficient exactness, ideal-filter approximation and final rounding.

11.4 Choosing an error-control component

- SECDED: correct one bit, detect every two-bit error.
- DVB RS: the specified shortened RS(204,188) byte order; the errors-only decoder corrects up to eight symbol errors, not erasures.
- Viterbi: minimum-metric framed K=7 decoding compatible with the existing encoder; a successful decode is not an error-detection guarantee.

Do not interchange the DVB polynomial/root/shortening conventions with CCSDS or another RS profile. Keep frame markers explicit; do not infer them from data.

11.5 Reproducing an example

From a prepared checkout, check a linked source, select the documented concrete top, generate Direct-SV and run the accompanying event schedule. The User Guide explains `make install`, JSONL event units, native simulation and waveform viewing. The Language Reference owns the common ready/valid, exact-width and clock/reset semantics; component chapters add only their own contracts.

12 Stdlib formal coverage

The nightly matrix uses the exact solver executables supplied by its pinned OSS CAD Suite; the current exact versions are recorded in `release/status.json`. The historical minimum for the minimized AXI4 read-endpoint query is Z3 4.13.4 because Z3 4.8.12 stalls. A successful formal RTL artifact build is not a solver result. The nightly job runs executable Yosys/SymbiYosys contracts and retains its JUnit, solver traces and the compiler-parser-owned declaration inventory. It is deliberately not a pull request gate.

The same job additionally publishes one immutable positive bundle and one deliberately failing bundle, then replays each through required Z3, Boolector and Bitwuzla routes plus corroborating Yices and cvc5 routes. Every route must produce the same property/status vector and the expected overall outcome. Results remain separate solver-execution records over the same compiler-owned bundle and lowering. Agreement is cross-solver corroboration, not a new proof status, and does not promote bounded evidence to `proven` or detect every shared front-end/lowering defect.

The nightly also qualifies distinct engine families against those same immutable bundles. ABC PDR supplies a separate unbounded safety route; Pono and btormc supply separate bounded BTOR routes. Their records are qualification evidence only: they are not accepted by the public `zlang verify --route` option and cannot satisfy required-formal selection. The Avy/AIGER route remains observational after a reproducible crash in the 2026-09-30 qualification. Updating the suite pin does not automatically promote that engine; incomplete observations never count as required gates. BTOR failures are mapped back through exact compiler bindings using route-owned rising-clock frames because BTOR witness VCDs do not carry the SMT route's `smt_step` marker.

Run locally with the repository's test environment:

```
mkdir -p build
.venv/bin/python -m pytest -q -o junit_family=legacy \
  tests/integration/test_stdlib_axi4_z3.py \
  tests/integration/test_stdlib_stream_z3.py \
  tests/integration/test_ztpu_axi_burst.py::
    test_root_ready_valid_safety_verification_executes_with_real_sby_z3 \
  --junitxml=build/stdlib-formal.xml
.venv/bin/python tools/stdlib_formal_coverage.py --junit build/stdlib-formal.xml
```

Use the exact release-status Z3 for acceptance. The test explicitly skips that case on versions older than the historical 4.13.4 minimum; the nightly no-skips gate rejects such an environment.

The inventory includes every parsed stdlib module, function and declarative type/protocol/target item. New executable declarations enter as `unverified`; they never inherit a proof from their containing file. `static-only` means that no RTL proof is applicable, **not** that a Z3 check ran. `partial` means that only the named contract and specialization have a result. The evidence table is checked against parsed declaration identities. Without a JUnit report all evidence entries say `evidence-unchecked`. With a report, the command rejects missing, failed or skipped evidence cases. The nightly job runs those tests in the same checkout before publishing the inventory. `blocked` records a reproduced solver obstacle; `unsupported` records a missing formal contract/route. Neither is a passing result.

Current executable contracts:

Stdlib entity	Contract and configuration	Solver level
`axi4_response_is_error`	Four `bits<2>` response codes; a wrong-code mutation fails	`PROVEN`
`axi4_address_valid`	Legal address implies `size == 0` for `AW=8,DW=8,IW=1`; a constant-legal mutation fails	`BOUNDED_PASS(4)` for this property only

Stdlib entity	Contract and configuration	Solver level
`AXI4ReadSubordinate`	Held R payload/valid under backpressure at `AW=8,DW=8,IW=1,D=2,BW=1`, with fixed legal request/backend beat; output-bypass mutation fails	`PROVEN` for this harness property only; module `partial`
`RvRegisterSlice`	Ready/valid stability with `T=u8`; an output-bypass mutation fails	`PROVEN` for this property only; module `partial`
`RvMux2`, `RvDemux2`	Selected payload/valid and selected-path backpressure at `T=u8`; inverted-select mutations fail	`BOUNDED_PASS(3)` for these routing properties only
`RvSkidBuffer`, `RvFifo`	Ready/valid stability at `T=u8, D=2` and `T=FrameBeat<u8,u2>, D=4`; output-bypass mutations fail	`BOUNDED_PASS(6)` only
`axi_burst` ZTPU reader/writer compositions	Generated ready/valid safety, depth 6	`BOUNDED_PASS`, not complete AXI4 compliance

The ordinary AXI4 read/write managers, remaining subordinate contracts, USER variants, pin adapters and exclusive path do **not** yet have complete endpoint Z3 proofs. Their compilation, simulation and formal-artifact tests must not be reported as proof. The same generated read-endpoint SMT2 query did not finish step 0 on Z3 4.8.12 within 5 seconds, while Z3 4.13.4 completed depth 2 in under one second; the source-authored read-endpoint harness then passed an inductive proof at depth 6 and a failing output-bypass mutation. This is a solver-version finding, not a ZLang semantic change. The fixed legal request and backend beat constrain the test, so it does not establish full AXI4 compliance. A reduced `AXI4ReadManager<8,8,1,2>` ready/valid harness timed out on the older solver; it still needs retesting and a checked-in proof contract.

Full-stdlib verification remains blocked by unverified executable entries and by formal routes that are currently unsupported, including multi-domain asynchronous reset goals. For generic entities, any future result applies only to its recorded finite parameter matrix and assumptions. Use `PROVEN` only for a successful inductive result; a finite-depth result remains `BOUNDED_PASS(depth)`. `UNKNOWN`, timeout and unsupported are never success.

12.1 New stream, DSP and coding sources

The source library includes buffered fork/join, CRC-32, immutable/reloadable/ folded FIR, rotation/vectoring CORDIC, ROM/interpolated NCO, CIC, polyphase and halfband filters, plus scrambler/convolutional coding. SECDED, Farrow, DVB RS(204,188) encode/decode and framed K=7 Viterbi are also present as source. Their numerical, latency and backpressure contracts are in the stream, FIR and multirate guides. The additional SECDED, Farrow, RS and Viterbi guides define their finite numerical, error-radius and framing guarantees.

Source availability is not a universal formal proof. The public source update does not add per-IP formal test suites or proof bundles: the existing nightly contracts above remain unchanged. Newly discovered executable declarations therefore enter the published inventory as `unverified` until their own checked, specialization-specific evidence is supplied. Previously recorded endpoint proofs do not qualify the new declarations.

13 Standard library API inventory

Generated by `tools/document_stdlib.py` from compiler declaration and symbol products. Do not edit signatures here. Edit the ordinary ZLang source and regenerate the catalog.

Each import below names a source namespace, not a qualification claim. Generic parameters and `where` clauses are part of the signature; a caller must select a legal specialization. Ports and record fields retain their declared width, sign and scale expressions. Pure helpers have no clock, latency or buffered transactions. Stateful modules have only the clocks, reset and ready/-valid ports listed; component chapters specify edge/II/stall and numerical behavior. Resource descriptions are capabilities, not physical timing evidence.

Source availability, successful simulation, synthesis and formal proof are distinct. No universal proof for all generic specializations is implied by this inventory. See the linked component contract and recipes for executable usage.

13.1 `std.arch.xilinx7_bilinear`

Import: `import std.arch.xilinx7_bilinear`.

Component contracts · Source

13.1.1 `Xilinx7ParallelProducts1` — class

Parallel independent full-width products; one optional joint output stage.

```
architecture Xilinx7ParallelProducts1 {
    operation parallel_bilinear
    require_resource DSP48E1 1
    latency 1
    ii 1
    use_pipeline unregistered
}
```

Exact declaration · Contract and restrictions

13.1.2 `Xilinx7ParallelProducts2` — class

```
architecture Xilinx7ParallelProducts2 {
    operation parallel_bilinear
    require_resource DSP48E1 2
    latency 1
    ii 1
    use_pipeline unregistered
}
```

Exact declaration · Contract and restrictions

13.1.3 `Xilinx7ParallelProducts3` — class

```
architecture Xilinx7ParallelProducts3 {
    operation parallel_bilinear
    require_resource DSP48E1 3
    latency 1
    ii 1
    use_pipeline unregistered
}
```

Exact declaration · Contract and restrictions

13.1.4 `Xilinx7ParallelProducts4` — class

```
architecture Xilinx7ParallelProducts4 {
    operation parallel_bilinear
    require_resource DSP48E1 4
    latency 1
    ii 1
    use_pipeline unregistered
}
```

Exact declaration · Contract and restrictions

13.1.5 `Xilinx7ParallelProducts5` — class

```
architecture Xilinx7ParallelProducts5 {
    operation parallel_bilinear
    require_resource DSP48E1 5
    latency 1
    ii 1
    use_pipeline unregistered
}
```

Exact declaration · Contract and restrictions

13.1.6 `Xilinx7ParallelProducts6` — class

```
architecture Xilinx7ParallelProducts6 {
    operation parallel_bilinear
    require_resource DSP48E1 6
    latency 1
    ii 1
    use_pipeline unregistered
}
```

Exact declaration · Contract and restrictions

13.1.7 `Xilinx7ParallelProducts7` — class

```
architecture Xilinx7ParallelProducts7 {
    operation parallel_bilinear
    require_resource DSP48E1 7
    latency 1
    ii 1
    use_pipeline unregistered
}
```

Exact declaration · Contract and restrictions

13.1.8 `Xilinx7ParallelProducts8` — class

```
architecture Xilinx7ParallelProducts8 {
    operation parallel_bilinear
    require_resource DSP48E1 8
    latency 1
    ii 1
    use_pipeline unregistered
}
```

Exact declaration · Contract and restrictions

13.1.9 `Xilinx7BilinearArchitectureLibrary` — module

```
module Xilinx7BilinearArchitectureLibrary
```

Exact declaration · Contract and restrictions

13.2 `std.arch.xilinx7_fir`

Import: `import std.arch.xilinx7_fir`.

Component contracts · Source

13.2.1 `Xilinx7SymmetricDSPCascade` — class

```
architecture Xilinx7SymmetricDSPCascade {
    operation symmetric_fir_cascade
    require_resource DSP48E1 4
    require_dedicated pcascade
    latency 1
    ii 1
    use_pipeline unregistered
}
```

Exact declaration · Contract and restrictions

13.2.2 `Xilinx7SymmetricDSPCascadeMREG` — class

```
architecture Xilinx7SymmetricDSPCascadeMREG {
    operation symmetric_fir_cascade
    require_resource DSP48E1 4
    require_dedicated pcascade
    latency 2
    ii 1
    use_pipeline multiply_registered
}
```

Exact declaration · Contract and restrictions

13.2.3 `Xilinx7SymmetricDSPCascadeMREGPREG` — class

```
architecture Xilinx7SymmetricDSPCascadeMREGPREG {
    operation symmetric_fir_cascade
    require_resource DSP48E1 4
    require_dedicated pcascade
    latency 3
    ii 1
    use_pipeline multiply_output_registered
}
```

Exact declaration · Contract and restrictions

13.2.4 `Xilinx7SymmetricDSPCascadeFullyPipelined` — class

```
architecture Xilinx7SymmetricDSPCascadeFullyPipelined {
    operation symmetric_fir_cascade
    require_resource DSP48E1 4
    require_dedicated pcascade
    latency 4
    ii 1
    use_pipeline fully_pipelined
}
```

Exact declaration · Contract and restrictions

13.2.5 `Xilinx7FirArchitectureLibrary` — module

```
module Xilinx7FirArchitectureLibrary
```

Exact declaration · Contract and restrictions

13.3 `std.arch.xilinx7_memory`

Import: `import std.arch.xilinx7_memory`.

Component contracts · Source

13.3.1 `Xilinx7BRAM36SimpleDualPort` — class

```
architecture Xilinx7BRAM36SimpleDualPort {
    operation synchronous_memory
    require_resource RAMB36E1 1
    latency 1
    ii 1
    use_pipeline core_registered
}
```

Exact declaration · Contract and restrictions

13.3.2 `Xilinx7AsyncFifoRAMB18` — class

FIFO-only route: typed async_mem read_latency=1 at the RAM core (DO_REG=0).

```
architecture Xilinx7AsyncFifoRAMB18 {
    operation async_fifo_memory
    require_resource RAMB18E1 1
    latency 1
    ii 1
    use_pipeline core_registered
}
```

Exact declaration · Contract and restrictions

13.3.3 `Xilinx7AsyncFifoRAMB36` — class

```
architecture Xilinx7AsyncFifoRAMB36 {
    operation async_fifo_memory
    require_resource RAMB36E1 1
    latency 1
    ii 1
    use_pipeline core_registered
}
```

Exact declaration · Contract and restrictions

13.3.4 `Xilinx7MemoryArchitectureLibrary` — module

```
module Xilinx7MemoryArchitectureLibrary
```

Exact declaration · Contract and restrictions

13.4 `std.arch.xilinx7_multiply`

Import: `import std.arch.xilinx7_multiply`.

Component contracts · Source

13.4.1 `Xilinx7Multiply` — class

One exact full-width scalar product. The mapper validates physical widths from the resource definition and chooses a declared DSP48E1 configuration.

```
architecture Xilinx7Multiply {
    operation multiply
    require_resource DSP48E1 1
    latency 1
    ii 1
    use_pipeline unregistered
}
```

Exact declaration · Contract and restrictions

13.4.2 `Xilinx7MultiplyMREG` — class

```
architecture Xilinx7MultiplyMREG {
    operation multiply
    require_resource DSP48E1 1
    latency 2
    ii 1
    use_pipeline multiply_registered
}
```

Exact declaration · Contract and restrictions

13.4.3 `Xilinx7MultiplyMREGPREG` — class

```
architecture Xilinx7MultiplyMREGPREG {
    operation multiply
    require_resource DSP48E1 1
    latency 3
    ii 1
    use_pipeline multiply_output_registered
}
```

Exact declaration · Contract and restrictions

13.4.4 `Xilinx7MultiplyFullyPipelined` — class

```
architecture Xilinx7MultiplyFullyPipelined {
    operation multiply
    require_resource DSP48E1 1
    latency 4
    ii 1
    use_pipeline fully_pipelined
}
```

Exact declaration · Contract and restrictions

13.4.5 `Xilinx7MultiplyArchitectureLibrary` — module

```
module Xilinx7MultiplyArchitectureLibrary
```

Exact declaration · Contract and restrictions

13.5 `std.arch.xilinx7_multiply_add`

Import: `import std.arch.xilinx7_multiply_add`.

Component contracts · Source

13.5.1 `Xilinx7MultiplyAdd` — class

One exact signed product combined with one same-scale addend. These are target-neutral architecture declarations; DSP48E1 legality and emission are selected from the resource definition and typed computation graph.

```
architecture Xilinx7MultiplyAdd {
    operation multiply_add
    require_resource DSP48E1 1
    latency 1
    ii 1
    use_pipeline unregistered
}
```

Exact declaration · Contract and restrictions

13.5.2 `Xilinx7MultiplyAddMREG` — class

```
architecture Xilinx7MultiplyAddMREG {
    operation multiply_add
    require_resource DSP48E1 1
    latency 2
    ii 1
    use_pipeline multiply_registered
}
```

Exact declaration · Contract and restrictions

13.5.3 `Xilinx7MultiplyAddMREGPREG` — class

```
architecture Xilinx7MultiplyAddMREGPREG {
    operation multiply_add
    require_resource DSP48E1 1
    latency 3
    ii 1
    use_pipeline multiply_output_registered
}
```

Exact declaration · Contract and restrictions

13.5.4 `Xilinx7MultiplyAddFullyPipelined` — class

```
architecture Xilinx7MultiplyAddFullyPipelined {
    operation multiply_add
    require_resource DSP48E1 1
    latency 4
    ii 1
    use_pipeline fully_pipelined
}
```

Exact declaration · Contract and restrictions

13.5.5 `Xilinx7MultiplyAddArchitectureLibrary` — module

```
module Xilinx7MultiplyAddArchitectureLibrary
```

Exact declaration · Contract and restrictions

13.6 `std.arch.xilinx7_signed_product`

Import: `import std.arch.xilinx7_signed_product`.

Component contracts · Source

13.6.1 `Xilinx7SignedProductCascade` — class

Ordered two-product accumulator cascades. The compiler matches the typed SignedProductReduction descriptor; these declarations contain only resource, timing and pipeline configuration intent.

```
architecture Xilinx7SignedProductCascade {
    operation signed_product_reduction
    require_resource DSP48E1 2
    require_dedicated pcascade
    latency 1
    ii 1
    use_pipeline unregistered
}
```

Exact declaration · Contract and restrictions

13.6.2 `Xilinx7SignedProductCascadeMREG` — class

```
architecture Xilinx7SignedProductCascadeMREG {
    operation signed_product_reduction
    require_resource DSP48E1 2
    require_dedicated pcascade
    latency 2
    ii 1
    use_pipeline multiply_registered
}
```

Exact declaration · Contract and restrictions

13.6.3 `Xilinx7SignedProductCascadeMREGPREG` — class

```
architecture Xilinx7SignedProductCascadeMREGPREG {
    operation signed_product_reduction
    require_resource DSP48E1 2
    require_dedicated pcascade
    latency 3
    ii 1
    use_pipeline multiply_output_registered
}
```

Exact declaration · Contract and restrictions

13.6.4 `Xilinx7SignedProductCascadeFullyPipelined` — class

```
architecture Xilinx7SignedProductCascadeFullyPipelined {
    operation signed_product_reduction
    require_resource DSP48E1 2
    require_dedicated pcascade
    latency 4
    ii 1
    use_pipeline fully_pipelined
}
```

Exact declaration · Contract and restrictions

13.6.5 `Xilinx7SignedProductArchitectureLibrary` — module

```
module Xilinx7SignedProductArchitectureLibrary
```

Exact declaration · Contract and restrictions

13.7 `std.bus.ahb_lite`

Import: `import std.bus.ahb_lite`.

Component contracts · Source

13.7.1 `AHBLite` — interface

```
protocol AHBLite<AW=32,DW=32> {
    role master
    role slave

    member HSEL      : bit       master -> slave
    member HADDR     : uint<AW>  master -> slave
    member HWRITE    : bit       master -> slave
    member HTRANS    : bits<2>   master -> slave
    member HSIZE     : bits<3>   master -> slave
    member HBURST    : bits<3>   master -> slave
    member HPROT     : bits<4>   master -> slave
    member HMASTLOCK : bit       master -> slave
    member HWDATA    : uint<DW>  master -> slave
    member HREADY    : bit       master -> slave

    member HRDATA    : uint<DW>  slave -> master
    member HREADYOUT : bit       slave -> master
    member HRESP     : bit       slave -> master
}
```

Exact declaration · Contract and restrictions

13.7.2 `AHBLiteBridgePhase` — enum

```
enum AHBLiteBridgePhase
Idle
Request
Response
ErrorFirst
ErrorSecond
```

Exact declaration · Contract and restrictions

13.7.3 `AHBLiteToRegBus` — module

```
module AHBLiteToRegBus<AW=32,DW=32>
where AW >= 1 && AW >= floor_log2(DW / 8) &&
      DW >= 8 && DW <= 1024 &&
      is_power_of_two(DW) && is_power_of_two(DW / 8)
interface ahb : AHBLite<AW,DW>.slave
interface regbus : RegBus<AW,DW>.requester
```

Exact declaration · Contract and restrictions

13.8 `std.bus.apb`

Import: `import std.bus.apb`.

Component contracts · Source

13.8.1 `APB` — interface

```
protocol APB<AW=32,DW=32> {
    role master
    role slave
    channel psel : wire<bit> master -> slave
    channel penable : wire<bit> master -> slave
    channel pwrite : wire<bit> master -> slave
    channel paddr : wire<uint<AW>> master -> slave
    channel pwdata : wire<uint<DW>> master -> slave
    channel pready : wire<bit> slave -> master
    channel prdata : wire<uint<DW>> slave -> master
    channel pslverr : wire<bit> slave -> master
}
```

Exact declaration · Contract and restrictions

13.8.2 `APBBridgePhase` — enum

```
enum APBBridgePhase
Idle
Access
Response
```

Exact declaration · Contract and restrictions

13.8.3 `APBToRegBus` — module

```
module APBToRegBus<AW=32,DW=32>
interface apb : APB<AW,DW>.slave
interface regbus : RegBus<AW,DW>.requester
```

Exact declaration · Contract and restrictions

13.9 `std.bus.axi4`

Import: `import std.bus.axi4`.

Component contracts · Source

13.9.1 `Axi4Address` — struct

```
struct Axi4Address<AW=32,IW=4>
id : bits<IW>
addr : uint<AW>
len : u8
size : u3
burst : bits<2>
lock : bit
cache : bits<4>
prot : bits<3>
qos : bits<4>
region : bits<4>
```

Exact declaration · Contract and restrictions

13.9.2 `Axi4WriteData` — struct

```
struct Axi4WriteData<DW=32>
data : bits<DW>
strb : bits<DW/8>
last : bit
```

Exact declaration · Contract and restrictions

13.9.3 `Axi4WriteResponse` — struct

```
struct Axi4WriteResponse<WIW=4>
id : bits<WIW>
resp : bits<2>
```

Exact declaration · Contract and restrictions

13.9.4 `Axi4ReadData` — struct

```
struct Axi4ReadData<DW=32,RIW=4>
id : bits<RIW>
data : bits<DW>
resp : bits<2>
last : bit
```

Exact declaration · Contract and restrictions

13.9.5 `AXI4` — interface

```
protocol AXI4<AW=32,DW=32,RIW=4,WIW=4> {
    role master
    role slave
    channel aw : rv<Axi4Address<AW,WIW>> master -> slave
    channel w : rv<Axi4WriteData<DW>> master -> slave
    channel b : rv<Axi4WriteResponse<WIW>> slave -> master
    channel ar : rv<Axi4Address<AW,RIW>> master -> slave
    channel r : rv<Axi4ReadData<DW,RIW>> slave -> master
}
```

Exact declaration · Contract and restrictions

13.9.6 `AXI4Read` — interface

```
protocol AXI4Read<AW=32,DW=32,RIW=4> {
    role master
    role slave
    channel ar : rv<Axi4Address<AW,RIW>> master -> slave
    channel r : rv<Axi4ReadData<DW,RIW>> slave -> master
}
```

Exact declaration · Contract and restrictions

13.9.7 `AXI4Write` — interface

```
protocol AXI4Write<AW=32,DW=32,WIW=4> {
    role master
    role slave
    channel aw : rv<Axi4Address<AW,WIW>> master -> slave
    channel w : rv<Axi4WriteData<DW>> master -> slave
    channel b : rv<Axi4WriteResponse<WIW>> slave -> master
}
```

Exact declaration · Contract and restrictions

13.9.8 `Axi4AddressUser` — struct

```
struct Axi4AddressUser<AW=32,IW=4,UW=1>
id : bits<IW>
addr : uint<AW>
len : u8
size : u3
burst : bits<2>
lock : bit
cache : bits<4>
prot : bits<3>
qos : bits<4>
region : bits<4>
user : bits<UW>
```

Exact declaration · Contract and restrictions

13.9.9 `Axi4WriteDataUser` — struct

```
struct Axi4WriteDataUser<DW=32,UW=1>
data : bits<DW>
strb : bits<DW/8>
last : bit
user : bits<UW>
```

Exact declaration · Contract and restrictions

13.9.10 `Axi4WriteResponseUser` — struct

```
struct Axi4WriteResponseUser<WIW=4,UW=1>
id : bits<WIW>
resp : bits<2>
user : bits<UW>
```

Exact declaration · Contract and restrictions

13.9.11 `Axi4ReadDataUser` — struct

```
struct Axi4ReadDataUser<DW=32,RIW=4,UW=1>
id : bits<RIW>
data : bits<DW>
resp : bits<2>
last : bit
user : bits<UW>
```

Exact declaration · Contract and restrictions

13.9.12 `AXI4WithUser` — interface

```
protocol AXI4WithUser<AW=32,DW=32,RIW=4,WIW=4,AWUW=1,WUW=1,BUW=1,ARUW=1,RUW=1> {
    role master
    role slave
    channel aw : rv<Axi4AddressUser<AW,WIW,AWUW>> master -> slave
    channel w : rv<Axi4WriteDataUser<DW,WUW>> master -> slave
    channel b : rv<Axi4WriteResponseUser<WIW,BUW>> slave -> master
    channel ar : rv<Axi4AddressUser<AW,RIW,ARUW>> master -> slave
    channel r : rv<Axi4ReadDataUser<DW,RIW,RUW>> slave -> master
}
```

Exact declaration · Contract and restrictions

13.9.13 `AXI4ReadWithUser` — interface

```
protocol AXI4ReadWithUser<AW=32,DW=32,RIW=4,ARUW=1,RUW=1> {
    role master
    role slave
    channel ar : rv<Axi4AddressUser<AW,RIW,ARUW>> master -> slave
    channel r : rv<Axi4ReadDataUser<DW,RIW,RUW>> slave -> master
}
```

Exact declaration · Contract and restrictions

13.9.14 `AXI4WriteWithUser` — interface

```
protocol AXI4WriteWithUser<AW=32,DW=32,WIW=4,AWUW=1,WUW=1,BUW=1> {
    role master
    role slave
    channel aw : rv<Axi4AddressUser<AW,WIW,AWUW>> master -> slave
    channel w : rv<Axi4WriteDataUser<DW,WUW>> master -> slave
    channel b : rv<Axi4WriteResponseUser<WIW,BUW>> slave -> master
}
```

Exact declaration · Contract and restrictions

13.9.15 `axi4_response_is_error` — function

The two response bits are transported without reinterpretation by the channel and the transaction adapters. EXOKAY is not interchangeable with OKAY on an exclusive read or a successful conditional write.

```
fn axi4_response_is_error(resp : bits<2>) -> bit
```

Exact declaration · Contract and restrictions

13.9.16 `axi4_transport_address` — function

Type-generic controllers retain the exact transport payload (including USER), while this projection supplies only the standard address fields to the common legality predicates. Missing/wrong fields fail specialization.

```
fn axi4_transport_address<type A,AW,IW>(request : A)
    -> Axi4Address<AW,IW>
```

Exact declaration · Contract and restrictions

13.9.17 `axi4_write_strobe_valid` — function

WSTRB may be sparse or entirely zero. Only lanes outside the transfer's legal byte window are forbidden; no helper silently rewrites WDATA/WSTRB.

```
fn axi4_write_strobe_valid<AW,DW>(address : uint<AW>, size : u3,
                                  strobe : bits<DW/8>) -> bit
```

Exact declaration · Contract and restrictions

13.9.18 `axi4_burst_shape_valid` — function

```
fn axi4_burst_shape_valid<DW>(length : u8, size : u3, burst : bits<2>) -> bit
```

Exact declaration · Contract and restrictions

13.9.19 `axi4_address_valid` — function

AW is 1..64. Widening to 80 bits prevents address and length overflow from making an illegal transaction appear to fit within the address bus or page. WRAP uses its aligned window; an unaligned INCR advances from the first aligned transfer boundary. Shifts (not variable division) calculate the legal power-of-two beat and wrap sizes.

```
fn axi4_address_valid<AW,DW,IW>(request : Axi4Address<AW,IW>) -> bit
```

Exact declaration · Contract and restrictions

13.9.20 `axi4_beat_address` — function

Beat zero may be unaligned. Subsequent INCR beats start on the transfer boundary; WRAP advances modulo its naturally aligned burst window.

```
fn axi4_beat_address<AW,IW>(request : Axi4Address<AW,IW>, index : u9)
    -> uint<AW>
```

Exact declaration · Contract and restrictions

13.9.21 `axi4_exclusive_address_valid` — function

A legal exclusive request also has an aligned, power-of-two transfer span of at most 128 bytes and at most 16 beats. Pair matching and reservation invalidation are stateful backend responsibilities, not this pure predicate.

```
fn axi4_exclusive_address_valid<AW,DW,IW>(request : Axi4Address<AW,IW>) -> bit
```

Exact declaration · Contract and restrictions

13.9.22 `AXI4ReadManagerCore` — module

A slot records the older outstanding slots with the same ID. This bounded predecessor relation permits repeated IDs without a wrapping sequence counter: only the oldest live matching slot may consume an R beat. Different IDs remain independently eligible for interleaved R beats.

```
module AXI4ReadManagerCore<type A,type R,AW=32,DW=32,RIW=4,D=4>
where AW >= 1 && AW <= 64 && RIW >= 1 && RIW <= 32 &&
      DW >= 8 && DW <= 1024 && is_power_of_two(DW) &&
      D >= 2 && is_power_of_two(D)
in command : rv<A>
out data : rv<R>
out protocol_error : bit
out axi_ar : rv<A>
in axi_r : rv<R>
```

Exact declaration · Contract and restrictions

13.9.23 `AXI4ReadManager` — module

```
module AXI4ReadManager<AW=32,DW=32,RIW=4,D=4>
where AW >= 1 && AW <= 64 && RIW >= 1 && RIW <= 32 &&
      DW >= 8 && DW <= 1024 && is_power_of_two(DW) &&
      D >= 2 && is_power_of_two(D)
in command : rv<Axi4Address<AW,RIW>>
out data : rv<Axi4ReadData<DW,RIW>>
out protocol_error : bit
interface axi : AXI4Read<AW,DW,RIW>.master
```

Exact declaration · Contract and restrictions

13.9.24 `AXI4ReadManagerWithUser` — module

```
module AXI4ReadManagerWithUser<AW=32,DW=32,RIW=4,ARUW=1,RUW=1,D=4>
where AW >= 1 && AW <= 64 && RIW >= 1 && RIW <= 32 &&
      DW >= 8 && DW <= 1024 && is_power_of_two(DW) &&
      ARUW >= 1 && RUW >= 1 && D >= 2 && is_power_of_two(D)
in command : rv<Axi4AddressUser<AW,RIW,ARUW>>
out data : rv<Axi4ReadDataUser<DW,RIW,RUW>>
out protocol_error : bit
interface axi : AXI4ReadWithUser<AW,DW,RIW,ARUW,RUW>.master
```

Exact declaration · Contract and restrictions

13.9.25 `AXI4WriteManager` — module

AW and W are independent: a stalled AW never suppresses WVALID. Only one write-data stream is active at a time (AXI4 has no WID/interleaving), while up to D completed streams may await B, including repeated IDs. The upstream write-beat producer owns payload stability under stall. A response with an unknown or unfinished ID is rejected and latched as a protocol error.

```
module AXI4WriteManager<AW=32,DW=32,WIW=4,D=4>
where AW >= 1 && AW <= 64 && WIW >= 1 && WIW <= 32 &&
      DW >= 8 && DW <= 1024 && is_power_of_two(DW) &&
      D >= 2 && is_power_of_two(D)
in command : rv<Axi4Address<AW,WIW>>
in write : rv<Axi4WriteData<DW>>
out completion : rv<Axi4WriteResponse<WIW>>
out protocol_error : bit
interface axi : AXI4Write<AW,DW,WIW>.master
```

Exact declaration · Contract and restrictions

13.9.26 `AXI4WriteManagerCore` — module

The USER write profile shares one payload-type-parametric transaction engine. PW is checked by every bitcast at specialization, so an incorrect caller cannot silently drop a USER bit. The established no-USER manager remains source-compatible with legacy cycle-simulator callers.

```
module AXI4WriteManagerCore<type A,type W,type B,AW,DW,WIW,PW,D=4>
where AW >= 1 && AW <= 64 && WIW >= 1 && WIW <= 32 &&
      DW >= 8 && DW <= 1024 && is_power_of_two(DW) &&
      D >= 2 && is_power_of_two(D)
in command : rv<A>
in write : rv<W>
out completion : rv<B>
out protocol_error : bit
out axi_aw : rv<A>
out axi_w : rv<W>
in axi_b : rv<B>
```

Exact declaration · Contract and restrictions

13.9.27 `AXI4WriteManagerWithUser` — module

```
module AXI4WriteManagerWithUser<AW=32,DW=32,WIW=4,
                                AWUW=1,WUW=1,BUW=1,D=4>
where AW >= 1 && AW <= 64 && WIW >= 1 && WIW <= 32 &&
      DW >= 8 && DW <= 1024 && is_power_of_two(DW) &&
      AWUW >= 1 && WUW >= 1 && BUW >= 1 &&
      D >= 2 && is_power_of_two(D)
in command : rv<Axi4AddressUser<AW,WIW,AWUW>>
in write : rv<Axi4WriteDataUser<DW,WUW>>
out completion : rv<Axi4WriteResponseUser<WIW,BUW>>
out protocol_error : bit
interface axi : AXI4WriteWithUser<AW,DW,WIW,AWUW,WUW,BUW>.master
```

Exact declaration · Contract and restrictions

13.9.28 `AXI4Manager` — module

A five-channel manager is composition of independently backpressured read and write engines. Its child signal bindings are ordinary static-protocol projections; no AXI-specific compiler route is involved.

```
module AXI4Manager<AW=32,DW=32,RIW=4,WIW=4,D=4>
where AW >= 1 && AW <= 64 && RIW >= 1 && RIW <= 32 &&
      WIW >= 1 && WIW <= 32 && DW >= 8 && DW <= 1024 &&
      is_power_of_two(DW) && D >= 2 && is_power_of_two(D)
in read_command : rv<Axi4Address<AW,RIW>>
out read_data : rv<Axi4ReadData<DW,RIW>>
in write_command : rv<Axi4Address<AW,WIW>>
in write_data : rv<Axi4WriteData<DW>>
out write_completion : rv<Axi4WriteResponse<WIW>>
out protocol_error : bit = reader.protocol_error | writer.protocol_error
interface axi : AXI4<AW,DW,RIW,WIW>.master
```

Exact declaration · Contract and restrictions

13.9.29 `AXI4ManagerWithUser` — module

```
module AXI4ManagerWithUser<AW=32,DW=32,RIW=4,WIW=4,
                           AWUW=1,WUW=1,BUW=1,ARUW=1,RUW=1,D=4>
where AW >= 1 && AW <= 64 && RIW >= 1 && RIW <= 32 &&
      WIW >= 1 && WIW <= 32 && DW >= 8 && DW <= 1024 &&
      is_power_of_two(DW) && AWUW >= 1 && WUW >= 1 &&
      BUW >= 1 && ARUW >= 1 && RUW >= 1 &&
      D >= 2 && is_power_of_two(D)
in read_command : rv<Axi4AddressUser<AW,RIW,ARUW>>
out read_data : rv<Axi4ReadDataUser<DW,RIW,RUW>>
in write_command : rv<Axi4AddressUser<AW,WIW,AWUW>>
in write_data : rv<Axi4WriteDataUser<DW,WUW>>
out write_completion : rv<Axi4WriteResponseUser<WIW,BUW>>
out protocol_error : bit = reader.protocol_error | writer.protocol_error
interface axi : AXI4WithUser<AW,DW,RIW,WIW,AWUW,WUW,BUW,ARUW,RUW>.master
```

Exact declaration · Contract and restrictions

13.10 `std.bus.axi4_exclusive`

Import: `import std.bus.axi4_exclusive`.

Component contracts · Source

13.10.1 `Axi4CommittedWrite` — struct

```
struct Axi4CommittedWrite<AW=32,DW=32>
beat_address : uint<AW>
strb : bits<DW/8>
```

Exact declaration · Contract and restrictions

13.10.2 `axi4_reservation_conflicts` — function

```
fn axi4_reservation_conflicts<AW,DW,IW>(
    reservation : Axi4Address<AW,IW>,
    write : Axi4CommittedWrite<AW,DW>
) -> bit
```

Exact declaration · Contract and restrictions

13.10.3 `AXI4ExclusiveMonitor` — module

```
module AXI4ExclusiveMonitor<AW=32,DW=32,IW=4,D=4,SW=2>
where AW >= 1 && AW <= 64 && IW >= 1 && IW <= 32 &&
      DW >= 8 && DW <= 1024 && is_power_of_two(DW) &&
      D >= 2 && is_power_of_two(D) && SW == floor_log2(D)
in reserve : rv<Axi4Address<AW,IW>>
in conditional : rv<Axi4Address<AW,IW>>
in committed_write : rv<Axi4CommittedWrite<AW,DW>>
out grant : bit
out reserve_can_accept : bit
out protocol_error : bit
```

Exact declaration · Contract and restrictions

13.10.4 `AXI4ExclusiveRamWitness` — module

One concrete single-port RAM integration of the atomic backend contract. A normal write wins the one physical port; the exclusive attempt waits for the next edge. This is a reference witness, not an AXI4 transaction adapter and not a promise about arbitrary external memories.

```
module AXI4ExclusiveRamWitness
clock clk
reset rst
in reserve : rv<Axi4Address<8,4>>
in conditional : rv<Axi4Address<8,4>>
in conditional_data : bits<32>
in normal_write : rv<Axi4CommittedWrite<8,32>>
in normal_data : bits<32>
in read_address : u4
out read_data : bits<32>
out exclusive_result : bits<2>
out protocol_error : bit
```

Exact declaration · Contract and restrictions

13.10.5 `AXI4ExclusiveSubordinate` — module

Five-channel exclusive-capable endpoint with a deliberately explicit backend commit contract. The backend presents a completed write result only on the edge on which it can atomically commit the collected W beats. For an exclusive result with resp=OKAY, it MUST gate that commit with exclusive_grant on the same edge; a failed grant discards the write. Every other actual write to the monitored memory is reported on committed_write, including writes originating from this endpoint. A backend which cannot satisfy those obligations must use AXI4Subordinate, not this variant.

```
module AXI4ExclusiveSubordinate<AW=32,DW=32,RIW=4,WIW=4,D=4,SW=2>
where AW >= 1 && AW <= 64 && RIW >= 1 && RIW <= 32 &&
      WIW >= 1 && WIW <= 32 && DW >= 8 && DW <= 1024 &&
      is_power_of_two(DW) && D >= 2 && is_power_of_two(D) &&
      SW == floor_log2(D)
out backend_read_request : rv<Axi4BackendReadRequest<AW,RIW,SW>>
in backend_read_beat : rv<Axi4BackendReadBeat<DW,SW>>
out backend_write_request : rv<Axi4BackendWriteRequest<AW,WIW,SW>>
out backend_write_beat : rv<Axi4BackendWriteBeat<DW,SW>>
in backend_write_result : rv<Axi4BackendWriteResult<SW>>
in committed_write : rv<Axi4CommittedWrite<AW,DW>>
out exclusive_grant : bit
out protocol_error : bit
interface axi : AXI4<AW,DW,RIW,WIW>.slave
```

Exact declaration · Contract and restrictions

13.11 `std.bus.axi4_pins`

Import: `import std.bus.axi4_pins`.

Component contracts · Source

13.11.1 `AXI4MasterPins` — module

```
module AXI4MasterPins<AW=32,DW=32,RIW=4,WIW=4>
where AW >= 1 && AW <= 64 && RIW >= 1 && RIW <= 32 &&
      WIW >= 1 && WIW <= 32 && DW >= 8 && DW <= 1024 &&
      is_power_of_two(DW)
clock ACLK
async reset ARESETn @ACLK { polarity active_low }
```

```
in request_aw : rv<Axi4Address<AW,WIW>>
in request_w : rv<Axi4WriteData<DW>>
out response_b : rv<Axi4WriteResponse<WIW>>
in request_ar : rv<Axi4Address<AW,RIW>>
out response_r : rv<Axi4ReadData<DW,RIW>>
out AWID : bits<WIW> = request_aw.payload.id
out AWADDR : uint<AW> = request_aw.payload.addr
out AWLEN : u8 = request_aw.payload.len
out AWSIZE : u3 = request_aw.payload.size
out AWBURST : bits<2> = request_aw.payload.burst
out AWLOCK : bit = request_aw.payload.lock
out AWCACHE : bits<4> = request_aw.payload.cache
out AWPROT : bits<3> = request_aw.payload.prot
out AWQOS : bits<4> = request_aw.payload.qos
out AWREGION : bits<4> = request_aw.payload.region
out AWVALID : bit = request_aw.valid
in AWREADY : bit
out WDATA : bits<DW> = request_w.payload.data
out WSTRB : bits<DW/8> = request_w.payload.strb
out WLAST : bit = request_w.payload.last
out WVALID : bit = request_w.valid
in WREADY : bit
in BID : bits<WIW>
in BRESP : bits<2>
in BVALID : bit
out BREADY : bit = response_b.ready
out ARID : bits<RIW> = request_ar.payload.id
out ARADDR : uint<AW> = request_ar.payload.addr
out ARLEN : u8 = request_ar.payload.len
out ARSIZE : u3 = request_ar.payload.size
out ARBURST : bits<2> = request_ar.payload.burst
out ARLOCK : bit = request_ar.payload.lock
out ARCACHE : bits<4> = request_ar.payload.cache
out ARPROT : bits<3> = request_ar.payload.prot
out ARQOS : bits<4> = request_ar.payload.qos
out ARREGION : bits<4> = request_ar.payload.region
out ARVALID : bit = request_ar.valid
in ARREADY : bit
in RID : bits<RIW>
in RDATA : bits<DW>
in RRESP : bits<2>
in RLAST : bit
in RVALID : bit
out RREADY : bit = response_r.ready
```

Exact declaration · Contract and restrictions

13.11.2 `AXI4SubordinatePins` — module

```
module AXI4SubordinatePins<AW=32,DW=32,RIW=4,WIW=4>
where AW >= 1 && AW <= 64 && RIW >= 1 && RIW <= 32 &&
      WIW >= 1 && WIW <= 32 && DW >= 8 && DW <= 1024 &&
      is_power_of_two(DW)
clock ACLK
async reset ARESETn @ACLK { polarity active_low }
out request_aw : rv<Axi4Address<AW,WIW>>
out request_w : rv<Axi4WriteData<DW>>
in response_b : rv<Axi4WriteResponse<WIW>>
out request_ar : rv<Axi4Address<AW,RIW>>
in response_r : rv<Axi4ReadData<DW,RIW>>
in AWID : bits<WIW>
```

```
in AWADDR : uint<AW>
in AWLEN : u8
in AWSIZE : u3
in AWBURST : bits<2>
in AWLOCK : bit
in AWCACHE : bits<4>
in AWPROT : bits<3>
in AWQOS : bits<4>
in AWREGION : bits<4>
in AWVALID : bit
out AWREADY : bit = request_aw.ready
in WDATA : bits<DW>
in WSTRB : bits<DW/8>
in WLAST : bit
in WVALID : bit
out WREADY : bit = request_w.ready
out BID : bits<WIW> = response_b.payload.id
out BRESP : bits<2> = response_b.payload.resp
out BVALID : bit = response_b.valid
in BREADY : bit
in ARID : bits<RIW>
in ARADDR : uint<AW>
in ARLEN : u8
in ARSIZE : u3
in ARBURST : bits<2>
in ARLOCK : bit
in ARCACHE : bits<4>
in ARPROT : bits<3>
in ARQOS : bits<4>
in ARREGION : bits<4>
in ARVALID : bit
out ARREADY : bit = request_ar.ready
out RID : bits<RIW> = response_r.payload.id
out RDATA : bits<DW> = response_r.payload.data
out RRESP : bits<2> = response_r.payload.resp
out RLAST : bit = response_r.payload.last
out RVALID : bit = response_r.valid
in RREADY : bit
```

Exact declaration · Contract and restrictions

13.11.3 `AXI4MasterPinsWithUser` — module

```
module AXI4MasterPinsWithUser<AW=32,DW=32,RIW=4,WIW=4,
                              AWUW=1,WUW=1,BUW=1,ARUW=1,RUW=1>
where AW >= 1 && AW <= 64 && RIW >= 1 && RIW <= 32 &&
      WIW >= 1 && WIW <= 32 && DW >= 8 && DW <= 1024 &&
      is_power_of_two(DW) && AWUW >= 1 && WUW >= 1 &&
      BUW >= 1 && ARUW >= 1 && RUW >= 1
clock ACLK
async reset ARESETn @ACLK { polarity active_low }
in request_aw : rv<Axi4AddressUser<AW,WIW,AWUW>>
in request_w : rv<Axi4WriteDataUser<DW,WUW>>
out response_b : rv<Axi4WriteResponseUser<WIW,BUW>>
in request_ar : rv<Axi4AddressUser<AW,RIW,ARUW>>
out response_r : rv<Axi4ReadDataUser<DW,RIW,RUW>>
out AWID : bits<WIW> = request_aw.payload.id
out AWADDR : uint<AW> = request_aw.payload.addr
out AWLEN : u8 = request_aw.payload.len
out AWSIZE : u3 = request_aw.payload.size
out AWBURST : bits<2> = request_aw.payload.burst
```

```
out AWLOCK : bit = request_aw.payload.lock
out AWCACHE : bits<4> = request_aw.payload.cache
out AWPROT : bits<3> = request_aw.payload.prot
out AWQOS : bits<4> = request_aw.payload.qos
out AWREGION : bits<4> = request_aw.payload.region
out AWUSER : bits<AWUW> = request_aw.payload.user
out AWVALID : bit = request_aw.valid
in AWREADY : bit
out WDATA : bits<DW> = request_w.payload.data
out WSTRB : bits<DW/8> = request_w.payload.strb
out WLAST : bit = request_w.payload.last
out WUSER : bits<WUW> = request_w.payload.user
out WVALID : bit = request_w.valid
in WREADY : bit
in BID : bits<WIW>
in BRESP : bits<2>
in BUSER : bits<BUW>
in BVALID : bit
out BREADY : bit = response_b.ready
out ARID : bits<RIW> = request_ar.payload.id
out ARADDR : uint<AW> = request_ar.payload.addr
out ARLEN : u8 = request_ar.payload.len
out ARSIZE : u3 = request_ar.payload.size
out ARBURST : bits<2> = request_ar.payload.burst
out ARLOCK : bit = request_ar.payload.lock
out ARCACHE : bits<4> = request_ar.payload.cache
out ARPROT : bits<3> = request_ar.payload.prot
out ARQOS : bits<4> = request_ar.payload.qos
out ARREGION : bits<4> = request_ar.payload.region
out ARUSER : bits<ARUW> = request_ar.payload.user
out ARVALID : bit = request_ar.valid
in ARREADY : bit
in RID : bits<RIW>
in RDATA : bits<DW>
in RRESP : bits<2>
in RLAST : bit
in RUSER : bits<RUW>
in RVALID : bit
out RREADY : bit = response_r.ready
```

Exact declaration · Contract and restrictions

13.11.4 `AXI4SubordinatePinsWithUser` — module

```
module AXI4SubordinatePinsWithUser<AW=32,DW=32,RIW=4,WIW=4,
                                   AWUW=1,WUW=1,BUW=1,ARUW=1,RUW=1>
where AW >= 1 && AW <= 64 && RIW >= 1 && RIW <= 32 &&
      WIW >= 1 && WIW <= 32 && DW >= 8 && DW <= 1024 &&
      is_power_of_two(DW) && AWUW >= 1 && WUW >= 1 &&
      BUW >= 1 && ARUW >= 1 && RUW >= 1
clock ACLK
async reset ARESETn @ACLK { polarity active_low }
out request_aw : rv<Axi4AddressUser<AW,WIW,AWUW>>
out request_w : rv<Axi4WriteDataUser<DW,WUW>>
in response_b : rv<Axi4WriteResponseUser<WIW,BUW>>
out request_ar : rv<Axi4AddressUser<AW,RIW,ARUW>>
in response_r : rv<Axi4ReadDataUser<DW,RIW,RUW>>
in AWID : bits<WIW>
in AWADDR : uint<AW>
in AWLEN : u8
in AWSIZE : u3
```

```
in AWBURST : bits<2>
in AWLOCK : bit
in AWCACHE : bits<4>
in AWPROT : bits<3>
in AWQOS : bits<4>
in AWREGION : bits<4>
in AWUSER : bits<AWUW>
in AWVALID : bit
out AWREADY : bit = request_aw.ready
in WDATA : bits<DW>
in WSTRB : bits<DW/8>
in WLAST : bit
in WUSER : bits<WUW>
in WVALID : bit
out WREADY : bit = request_w.ready
out BID : bits<WIW> = response_b.payload.id
out BRESP : bits<2> = response_b.payload.resp
out BUSER : bits<BUW> = response_b.payload.user
out BVALID : bit = response_b.valid
in BREADY : bit
in ARID : bits<RIW>
in ARADDR : uint<AW>
in ARLEN : u8
in ARSIZE : u3
in ARBURST : bits<2>
in ARLOCK : bit
in ARCACHE : bits<4>
in ARPROT : bits<3>
in ARQOS : bits<4>
in ARREGION : bits<4>
in ARUSER : bits<ARUW>
in ARVALID : bit
out ARREADY : bit = request_ar.ready
out RID : bits<RIW> = response_r.payload.id
out RDATA : bits<DW> = response_r.payload.data
out RRESP : bits<2> = response_r.payload.resp
out RLAST : bit = response_r.payload.last
out RUSER : bits<RUW> = response_r.payload.user
out RVALID : bit = response_r.valid
in RREADY : bit
```

Exact declaration · Contract and restrictions

13.12 `std.bus.axi4_subordinate`

Import: `import std.bus.axi4_subordinate`.

Component contracts · Source

13.12.1 `Axi4BackendReadRequest` — struct

```
struct Axi4BackendReadRequest<AW=32,RIW=4,SW=2>
slot : bits<SW>
address : Axi4Address<AW,RIW>
```

Exact declaration · Contract and restrictions

13.12.2 `Axi4BackendReadBeat` — struct

```
struct Axi4BackendReadBeat<DW=32,SW=2>
slot : bits<SW>
data : bits<DW>
resp : bits<2>
last : bit
```

Exact declaration · Contract and restrictions

13.12.3 `AXI4ReadSubordinate` — module

```
module AXI4ReadSubordinate<AW=32,DW=32,RIW=4,D=4,SW=2,EXCLUSIVE=0>
where AW >= 1 && AW <= 64 && RIW >= 1 && RIW <= 32 &&
      DW >= 8 && DW <= 1024 && is_power_of_two(DW) &&
      D >= 2 && is_power_of_two(D) && SW == floor_log2(D) &&
      EXCLUSIVE >= 0 && EXCLUSIVE <= 1
out backend_request : rv<Axi4BackendReadRequest<AW,RIW,SW>>
in backend_beat : rv<Axi4BackendReadBeat<DW,SW>>
out protocol_error : bit
interface axi : AXI4Read<AW,DW,RIW>.slave
```

Exact declaration · Contract and restrictions

13.12.4 `Axi4BackendWriteRequest` — struct

```
struct Axi4BackendWriteRequest<AW=32,WIW=4,SW=2>
slot : bits<SW>
address : Axi4Address<AW,WIW>
```

Exact declaration · Contract and restrictions

13.12.5 `Axi4BackendWriteBeat` — struct

```
struct Axi4BackendWriteBeat<DW=32,SW=2>
slot : bits<SW>
data : bits<DW>
strb : bits<DW/8>
last : bit
```

Exact declaration · Contract and restrictions

13.12.6 `Axi4BackendWriteResult` — struct

```
struct Axi4BackendWriteResult<SW=2>
slot : bits<SW>
resp : bits<2>
```

Exact declaration · Contract and restrictions

13.12.7 `AXI4WriteSubordinate` — module

```
module AXI4WriteSubordinate<AW=32,DW=32,WIW=4,D=4,SW=2,EXCLUSIVE=0>
where AW >= 1 && AW <= 64 && WIW >= 1 && WIW <= 32 &&
      DW >= 8 && DW <= 1024 && is_power_of_two(DW) &&
      D >= 2 && is_power_of_two(D) && SW == floor_log2(D) &&
      EXCLUSIVE >= 0 && EXCLUSIVE <= 1
out backend_request : rv<Axi4BackendWriteRequest<AW,WIW,SW>>
out backend_beat : rv<Axi4BackendWriteBeat<DW,SW>>
in backend_result : rv<Axi4BackendWriteResult<SW>>
out result_can_accept : bit
out response_slot : bits<SW>
out protocol_error : bit
interface axi : AXI4Write<AW,DW,WIW>.slave
```

Exact declaration · Contract and restrictions

13.12.8 `AXI4Subordinate` — module

The physical five-channel endpoint composes two independent bounded transaction engines. The connections below exercise the same generic static-child signal projection available to every protocol family.

```
module AXI4Subordinate<AW=32,DW=32,RIW=4,WIW=4,D=4,SW=2,EXCLUSIVE=0>
where AW >= 1 && AW <= 64 && RIW >= 1 && RIW <= 32 &&
      WIW >= 1 && WIW <= 32 && DW >= 8 && DW <= 1024 &&
      is_power_of_two(DW) && D >= 2 && is_power_of_two(D) &&
      SW == floor_log2(D) && EXCLUSIVE >= 0 && EXCLUSIVE <= 1
out backend_read_request : rv<Axi4BackendReadRequest<AW,RIW,SW>>
in backend_read_beat : rv<Axi4BackendReadBeat<DW,SW>>
out backend_write_request : rv<Axi4BackendWriteRequest<AW,WIW,SW>>
out backend_write_beat : rv<Axi4BackendWriteBeat<DW,SW>>
in backend_write_result : rv<Axi4BackendWriteResult<SW>>
out backend_write_result_can_accept : bit = writer.result_can_accept
out backend_write_response_slot : bits<SW> = writer.response_slot
out protocol_error : bit = reader.protocol_error | writer.protocol_error
interface axi : AXI4<AW,DW,RIW,WIW>.slave
```

Exact declaration · Contract and restrictions

13.13 `std.bus.axi4_user_subordinate`

Import: `import std.bus.axi4_user_subordinate`.

Component contracts · Source

13.13.1 `Axi4BackendReadRequestUser` — struct

```
struct Axi4BackendReadRequestUser<AW=32,RIW=4,ARUW=1,SW=2>
slot : bits<SW>
address : Axi4AddressUser<AW,RIW,ARUW>
```

Exact declaration · Contract and restrictions

13.13.2 `Axi4BackendReadBeatUser` — struct

```
struct Axi4BackendReadBeatUser<DW=32,RUW=1,SW=2>
slot : bits<SW>
data : bits<DW>
resp : bits<2>
last : bit
user : bits<RUW>
```

Exact declaration · Contract and restrictions

13.13.3 `AXI4ReadSubordinateWithUser` — module

```
module AXI4ReadSubordinateWithUser<AW=32,DW=32,RIW=4,
                                   ARUW=1,RUW=1,D=4,SW=2>
where AW >= 1 && AW <= 64 && RIW >= 1 && RIW <= 32 &&
      DW >= 8 && DW <= 1024 && is_power_of_two(DW) &&
      ARUW >= 1 && RUW >= 1 && D >= 2 && is_power_of_two(D) &&
      SW == floor_log2(D)
out backend_request : rv<Axi4BackendReadRequestUser<AW,RIW,ARUW,SW>>
in backend_beat : rv<Axi4BackendReadBeatUser<DW,RUW,SW>>
out protocol_error : bit
interface axi : AXI4ReadWithUser<AW,DW,RIW,ARUW,RUW>.slave
```

Exact declaration · Contract and restrictions

13.13.4 `Axi4BackendWriteRequestUser` — struct

```
struct Axi4BackendWriteRequestUser<AW=32,WIW=4,AWUW=1,SW=2>
slot : bits<SW>
address : Axi4AddressUser<AW,WIW,AWUW>
```

Exact declaration · Contract and restrictions

13.13.5 `Axi4BackendWriteBeatUser` — struct

```
struct Axi4BackendWriteBeatUser<DW=32,WUW=1,SW=2>
slot : bits<SW>
data : bits<DW>
strb : bits<DW/8>
last : bit
user : bits<WUW>
```

Exact declaration · Contract and restrictions

13.13.6 `Axi4BackendWriteResultUser` — struct

```
struct Axi4BackendWriteResultUser<BUW=1,SW=2>
slot : bits<SW>
resp : bits<2>
user : bits<BUW>
```

Exact declaration · Contract and restrictions

13.13.7 `AXI4WriteSubordinateWithUser` — module

```
module AXI4WriteSubordinateWithUser<AW=32,DW=32,WIW=4,
                                    AWUW=1,WUW=1,BUW=1,D=4,SW=2>
where AW >= 1 && AW <= 64 && WIW >= 1 && WIW <= 32 &&
      DW >= 8 && DW <= 1024 && is_power_of_two(DW) &&
      AWUW >= 1 && WUW >= 1 && BUW >= 1 &&
      D >= 2 && is_power_of_two(D) && SW == floor_log2(D)
out backend_request : rv<Axi4BackendWriteRequestUser<AW,WIW,AWUW,SW>>
out backend_beat : rv<Axi4BackendWriteBeatUser<DW,WUW,SW>>
in backend_result : rv<Axi4BackendWriteResultUser<BUW,SW>>
out protocol_error : bit
interface axi : AXI4WriteWithUser<AW,DW,WIW,AWUW,WUW,BUW>.slave
```

Exact declaration · Contract and restrictions

13.13.8 `AXI4SubordinateWithUser` — module

```
module AXI4SubordinateWithUser<AW=32,DW=32,RIW=4,WIW=4,
                               AWUW=1,WUW=1,BUW=1,ARUW=1,RUW=1,
                               D=4,SW=2>
where AW >= 1 && AW <= 64 && RIW >= 1 && RIW <= 32 &&
      WIW >= 1 && WIW <= 32 && DW >= 8 && DW <= 1024 &&
      is_power_of_two(DW) && AWUW >= 1 && WUW >= 1 &&
      BUW >= 1 && ARUW >= 1 && RUW >= 1 &&
      D >= 2 && is_power_of_two(D) && SW == floor_log2(D)
out backend_read_request : rv<Axi4BackendReadRequestUser<AW,RIW,ARUW,SW>>
in backend_read_beat : rv<Axi4BackendReadBeatUser<DW,RUW,SW>>
out backend_write_request : rv<Axi4BackendWriteRequestUser<AW,WIW,AWUW,SW>>
out backend_write_beat : rv<Axi4BackendWriteBeatUser<DW,WUW,SW>>
in backend_write_result : rv<Axi4BackendWriteResultUser<BUW,SW>>
out protocol_error : bit = reader.protocol_error | writer.protocol_error
interface axi : AXI4WithUser<AW,DW,RIW,WIW,AWUW,WUW,BUW,ARUW,RUW>.slave
```

Exact declaration · Contract and restrictions

13.14 `std.bus.axi_burst`

Import: `import std.bus.axi_burst`.

Component contracts · Source

13.14.1 `AxiBurstAddress` — struct

```
struct AxiBurstAddress<AW=32>
addr : uint<AW>
len : u8
size : u3
```

Exact declaration · Contract and restrictions

13.14.2 `AxiSingleBeatAddressResult` — struct

A full-width one-beat address. The payload is meaningful only when valid is set; unsupported geometry or an unaligned address must be rejected before asserting AR/AWVALID.

```
struct AxiSingleBeatAddressResult<AW=32>
payload : AxiBurstAddress<AW>
valid : bit
```

Exact declaration · Contract and restrictions

13.14.3 `axi_single_beat_address` — function

```
fn axi_single_beat_address<AW,DW>(address : uint<AW>)
    -> AxiSingleBeatAddressResult<AW>
```

Exact declaration · Contract and restrictions

13.14.4 `AxiBurstReadData` — struct

```
struct AxiBurstReadData<DW=32>
data : uint<DW>
last : bit
resp : bits<2>
```

Exact declaration · Contract and restrictions

13.14.5 `AxiBurstWriteData` — struct

```
struct AxiBurstWriteData<DW=32>
data : uint<DW>
last : bit
```

Exact declaration · Contract and restrictions

13.14.6 `AxiBurstWriteResponse` — struct

```
struct AxiBurstWriteResponse
resp : bits<2>
```

Exact declaration · Contract and restrictions

13.14.7 `AXI4BurstSubset` — interface

```
protocol AXI4BurstSubset<AW=32,DW=32> {
    role master
    role slave
    channel ar : rv<AxiBurstAddress<AW>> master -> slave
    channel r  : rv<AxiBurstReadData<DW>> slave -> master
    channel aw : rv<AxiBurstAddress<AW>> master -> slave
    channel w  : rv<AxiBurstWriteData<DW>> master -> slave
    channel b  : rv<AxiBurstWriteResponse> slave -> master
}
```

Exact declaration · Contract and restrictions

13.14.8 `AXI4BurstReadSubset` — interface

ZTPU exposes physically independent read and write masters. These views use the same payload types and ready/valid semantics as the combined profile.

```
protocol AXI4BurstReadSubset<AW=32,DW=32> {
    role master
    role slave
    channel ar : rv<AxiBurstAddress<AW>> master -> slave
    channel r  : rv<AxiBurstReadData<DW>> slave -> master
}
```

Exact declaration · Contract and restrictions

13.14.9 `AXI4BurstWriteSubset` — interface

```
protocol AXI4BurstWriteSubset<AW=32,DW=32> {
    role master
    role slave
    channel aw : rv<AxiBurstAddress<AW>> master -> slave
    channel w  : rv<AxiBurstWriteData<DW>> master -> slave
    channel b  : rv<AxiBurstWriteResponse> slave -> master
}
```

Exact declaration · Contract and restrictions

13.14.10 `AxiBurstReadPhase` — enum

```
enum AxiBurstReadPhase
Idle
Address
Data
Done
```

Exact declaration · Contract and restrictions

13.14.11 `AXI4BurstReader` — module

One outstanding, full-width incrementing read burst. Error is sticky for an epoch, but malformed RLAST or nonzero RRESP never prevents counted draining.

```
module AXI4BurstReader<AW=32,DW=32,LW=16>
where AW >= 1 && AW >= floor_log2(DW / 8) &&
      LW >= 9 && DW >= 8 && DW <= 1024 &&
      is_power_of_two(DW) && is_power_of_two(DW / 8)
in start : bit
in base_addr : uint<AW>
in length : uint<LW>
out busy : bit
out done : bit
out error : bit
out data : rv<uint<DW>>
interface axi : AXI4BurstReadSubset<AW,DW>.master
```

Exact declaration · Contract and restrictions

13.14.12 `AxiBurstWritePhase` — enum

```
enum AxiBurstWritePhase
Idle
Address
Data
Response
Done
```

Exact declaration · Contract and restrictions

13.14.13 `AXI4BurstWriter` — module

One outstanding write burst. AW is accepted before data is exposed to the bus; W and B retain independent backpressure and WLAST is locally counted.

```
module AXI4BurstWriter<AW=32,DW=32,LW=16>
where AW >= 1 && AW >= floor_log2(DW / 8) &&
      LW >= 9 && DW >= 8 && DW <= 1024 &&
      is_power_of_two(DW) && is_power_of_two(DW / 8)
in start : bit
in base_addr : uint<AW>
in length : uint<LW>
in data : rv<uint<DW>>
out busy : bit
out done : bit
out error : bit
interface axi : AXI4BurstWriteSubset<AW,DW>.master
```

Exact declaration · Contract and restrictions

13.14.14 `AXI4SingleBeatReader` — module

One full-width read, with the same counted RLAST/RRESP checks as the burst reader. The fixed length is a source-level specialization, not a new bus.

```
module AXI4SingleBeatReader<AW=32,DW=32>
where AW >= 1 && AW >= floor_log2(DW / 8) &&
      DW >= 8 && DW <= 1024 &&
      is_power_of_two(DW) && is_power_of_two(DW / 8)
in start : bit
in address : uint<AW>
out busy : bit
out done : bit
out error : bit
out data : rv<uint<DW>>
interface axi : AXI4BurstReadSubset<AW,DW>.master
```

Exact declaration · Contract and restrictions

13.14.15 `AxiSingleBeatWritePhase` — enum

```
enum AxiSingleBeatWritePhase
Idle
Send
Response
Done
```

Exact declaration · Contract and restrictions

13.14.16 `AXI4SingleBeatWriter` — module

AW and W may each be accepted first. Both payloads are snapshotted on start and held until their own handshakes; B is accepted only after both.

```
module AXI4SingleBeatWriter<AW=32,DW=32>
where AW >= 1 && AW >= floor_log2(DW / 8) &&
      DW >= 8 && DW <= 1024 &&
      is_power_of_two(DW) && is_power_of_two(DW / 8)
in start : bit
in address : uint<AW>
in data : uint<DW>
out busy : bit
out done : bit
out error : bit
interface axi : AXI4BurstWriteSubset<AW,DW>.master
```

Exact declaration · Contract and restrictions

13.15 `std.bus.axi_lite`

Import: `import std.bus.axi_lite`.

Component contracts · Source

13.15.1 `AxiLiteAw` — struct

```
struct AxiLiteAw<AW=32>
addr : uint<AW>
```

Exact declaration · Contract and restrictions

13.15.2 `AxiLiteW` — struct

```
struct AxiLiteW<DW=32>
data : uint<DW>
strb : bits<DW/8>
```

Exact declaration · Contract and restrictions

13.15.3 `AxiLiteB` — struct

```
struct AxiLiteB
resp : bits<2>
```

Exact declaration · Contract and restrictions

13.15.4 `AxiLiteR` — struct

```
struct AxiLiteR<DW=32>
data : uint<DW>
resp : bits<2>
```

Exact declaration · Contract and restrictions

13.15.5 `AXI4Lite` — interface

```
protocol AXI4Lite<AW=32,DW=32> {
    role master
    role slave
    channel aw : rv<AxiLiteAw<AW>> master -> slave
    channel w  : rv<AxiLiteW<DW>> master -> slave
    channel b  : rv<AxiLiteB> slave -> master
    channel ar : rv<AxiLiteAw<AW>> master -> slave
    channel r  : rv<AxiLiteR<DW>> slave -> master
}
```

Exact declaration · Contract and restrictions

13.15.6 `AXI4LiteToRegBus` — module

This complete source-authored bridge is compiled through generic aggregate endpoints, hierarchy, state and ready/valid lowering. Neither backend has an AXI-specific dispatcher. Error responses are retained as SLVERR while B/R channels are stalled.

```
module AXI4LiteToRegBus<AW=32,DW=32>
interface axi : AXI4Lite<AW,DW>.slave
interface regbus : RegBus<AW,DW>.requester
```

Exact declaration · Contract and restrictions

13.16 `std.bus.axi_stream`

Import: `import std.bus.axi_stream`.

Component contracts · Source

13.16.1 `AXIStreamBeat` — struct

AMBA AXI4-Stream source profile: byte lanes plus packet boundary.

```
struct AXIStreamBeat<DW=32>
data : uint<DW>
keep : bits<DW/8>
strb : bits<DW/8>
last : bit
```

Exact declaration · Contract and restrictions

13.16.2 `AXIStream` — interface

```
protocol AXIStream<DW=32> {
    role source
    role sink
    channel t : rv<AXIStreamBeat<DW>> source -> sink
}
```

Exact declaration · Contract and restrictions

13.16.3 `AXIStreamOf` — interface

Typed-payload profile. AXIStream above remains the byte-lane bus profile and is kept source-compatible. AXIStreamOf is useful when a library-level semantic payload (for example Complex<fixed<18,16») should cross the same ready/valid boundary without teaching the compiler that payload's meaning. A later physical bus wrapper may serialize T to TDATA.

```
protocol AXIStreamOf<type T> {
    role source
    role sink
    channel t : rv<T> source -> sink
}
```

Exact declaration · Contract and restrictions

13.16.4 `AXIStreamPipe` — module

A zero-latency, backpressure-preserving reusable library component.

```
module AXIStreamPipe<DW=32>
interface input : AXIStream<DW>.sink
interface output : AXIStream<DW>.source
```

Exact declaration · Contract and restrictions

13.17 `std.bus.reg`

Import: `import std.bus.reg`.

Component contracts · Source

13.17.1 `RegRequest` — struct

```
struct RegRequest<AW=32,DW=32>
addr : uint<AW>
write : bit
wdata : uint<DW>
wstrb : bits<DW/8>
```

Exact declaration · Contract and restrictions

13.17.2 `RegResponse` — struct

```
struct RegResponse<DW=32>
rdata : uint<DW>
error : bit
```

Exact declaration · Contract and restrictions

13.17.3 `RegBus` — interface

```
protocol RegBus<AW=32,DW=32> {
    role requester
    role responder
    channel request : rv<RegRequest<AW,DW>> requester -> responder
    channel response : rv<RegResponse<DW>> responder -> requester
}
```

Exact declaration · Contract and restrictions

13.17.4 `RegisteredCSRTarget` — module

One generic registered transport boundary for a canonical CSR access ABI. The bank remains an ordinary compiler-owned CSR module; this adapter owns only request capture and the exactly-one-response-phase protocol timing.

```
module RegisteredCSRTarget<AW=32,DW=32>
out csr_addr : uint<AW>
out csr_write : bit
out csr_wdata : uint<DW>
out csr_read : bit
in csr_rdata : uint<DW>
in csr_ready : bit
interface regbus : RegBus<AW,DW>.responder
```

Exact declaration · Contract and restrictions

13.17.5 `RegBusCSRBank` — module

The core CSR declaration is the sole authority for the fixture's address, reset, and access behavior. Its canonical access ports are supplied by the compiler's generic CSR composition ABI when this module is instantiated.

```
module RegBusCSRBank
```

Exact declaration · Contract and restrictions

13.17.6 `RegBusCSRTarget` — module

```
module RegBusCSRTarget<AW=32,DW=32>
out done : bit
out rw_value : bit
out w1c_value : bit
out pulse : bit
interface regbus : RegBus<AW,DW>.responder
```

Exact declaration · Contract and restrictions

13.18 `std.bus.wishbone`

Import: `import std.bus.wishbone`.

Component contracts · Source

13.18.1 `Wishbone` — interface

```
protocol Wishbone<AW=32,DW=32> {
    role master
    role slave
    member cyc   : bit master -> slave
    member stb   : bit master -> slave
    member we    : bit master -> slave
    member adr   : uint<AW> master -> slave
    member dat_w : uint<DW> master -> slave
    member sel   : bits<DW/8> master -> slave
    member ack   : bit slave -> master
    member err   : bit slave -> master
    member stall : bit slave -> master
    member dat_r : uint<DW> slave -> master
}
```

Exact declaration · Contract and restrictions

13.18.2 `WishboneToRegBus` — module

```
module WishboneToRegBus<AW=32,DW=32>
interface wb : Wishbone<AW,DW>.slave
interface regbus : RegBus<AW,DW>.requester
```

Exact declaration · Contract and restrictions

13.19 `std.coding.convolutional`

Import: `import std.coding.convolutional`.

Component contracts · Source

13.19.1 `ConvolutionalResult24` — struct

IEEE 802.11 K=7 rate-1/2 kernel, 24 source bits, bit zero first. Encoded pairs retain the project's transmission order at the MSB end.

```
struct ConvolutionalResult24
encoded : bits<48>
history : u6
```

Exact declaration · Contract and restrictions

13.19.2 `encoder_g0` — function

```
fn encoder_g0(stream : u30) -> bits<24>
```

Exact declaration · Contract and restrictions

13.19.3 `encoder_g1` — function

```
fn encoder_g1(stream : u30) -> bits<24>
```

Exact declaration · Contract and restrictions

13.19.4 `interleave24` — function

```
fn interleave24(g0 : bits<24>, g1 : bits<24>) -> bits<48>
```

Exact declaration · Contract and restrictions

13.19.5 `ieee_convolutional_encode24` — function

```
fn ieee_convolutional_encode24(word:bits<24>, history:u6) -> ConvolutionalResult24
```

Exact declaration · Contract and restrictions

13.19.6 `ConvolutionalWord24` — struct

```
struct ConvolutionalWord24
data : bits<24>
restart : bit
finish : bit
```

Exact declaration · Contract and restrictions

13.19.7 `RvConvolutionalK7_24` — module

No implicit tail insertion, puncturing or rate selection. Finish clears the history AFTER this word; restart clears it BEFORE this word.

```
module RvConvolutionalK7_24
clock clk
reset rst
in input : rv<ConvolutionalWord24>
out output : rv<bits<48>>
```

Exact declaration · Contract and restrictions

13.20 `std.coding.core`

Import: `import std.coding.core`.

Component contracts · Source

13.20.1 `coding_parity` — function

Pure bit-level coding helpers. They are intentionally independent of any packet protocol, polynomial convention, or Wi-Fi profile.

```
fn coding_parity<N>(value : bits<N>) -> bit
```

Exact declaration · Contract and restrictions

13.20.2 `coding_reverse_bits` — function

```
fn coding_reverse_bits<N>(value : bits<N>)
```

Exact declaration · Contract and restrictions

13.20.3 `coding_lfsr_step` — function

Fibonacci LFSR step: bit zero of the returned raw sequence is the feedback bit and the old sequence shifts toward the MSB. Functions do not yet accept declaration constraints, so the public module wrapper below is the checked hardware boundary while this pure helper retains source compatibility.

```
fn coding_lfsr_step<N>(state : bits<N>, taps : bits<N>)
```

Exact declaration · Contract and restrictions

13.20.4 `CodingLfsrStep` — module

```
module CodingLfsrStep<N=8> where N >= 2
in state : bits<N>
in polynomial_taps : bits<N>
out next_state : bits<N>
```

Exact declaration · Contract and restrictions

13.20.5 `coding_convolution_dot` — function

Exact convolution/dot helper. The normal numeric type rules determine the widened result; this helper performs no narrowing or quantization.

```
fn coding_convolution_dot<type T,N>(
    samples : vec<N,T>, coefficients : vec<N,T>
)
```

Exact declaration · Contract and restrictions

13.20.6 `table_gather` — function

Runtime table gather with a statically proven selector range. This helper preserves source order and deliberately does not claim that `order` is a bijective permutation: duplicate and omitted source elements are legal. Each concrete specialization is accepted only when uint's complete range fits 0..N-1.

```
fn table_gather<type T,N,IW>(
    values : vec<N,T>, order : vec<N,uint<IW>>
)
```

Exact declaration · Contract and restrictions

13.21 `std.coding.crc`

Import: `import std.coding.crc`.

Component contracts · Source

13.21.1 `crc_step_reflected` — function

One reflected polynomial step, exact W-bit raw state. Initialization, bit ordering across words, and final conditioning belong to the caller.

```
fn crc_step_reflected<W>(state : bits<W>, data_bit : bit,
                         polynomial : bits<W>) -> bits<W>
```

Exact declaration · Contract and restrictions

13.21.2 `CrcByteBeat` — struct

```
struct CrcByteBeat
data : bits<8>
last : bit
```

Exact declaration · Contract and restrictions

13.21.3 `RvCrc32Byte` — module

CRC-32/ISO-HDLC: reflected 0xEDB88320, raw init/final XOR 0xFFFFFFFF. A last byte publishes exactly one checksum. The next frame waits for the result slot, with same-edge result retirement / first-byte acceptance.

```
module RvCrc32Byte
clock clk
reset rst
in input : rv<CrcByteBeat>
out output : rv<bits<32>>
```

Exact declaration · Contract and restrictions

13.22 `std.coding.gf256`

Import: `import std.coding.gf256`.

Component contracts · Source

13.22.1 `gf256_mul` — function

Polynomial-basis GF(256), primitive polynomial x^8+x^4+x^3+x^2+1. Values are raw polynomial coefficients, least-significant term first.

```
fn gf256_mul(a:bits<8>,b:bits<8>) -> bits<8>
```

Exact declaration · Contract and restrictions

13.22.2 `Gf256Inverse` — struct

```
struct Gf256Inverse
value:bits<8>
valid:bit
```

Exact declaration · Contract and restrictions

13.22.3 `gf256_inverse` — function

Zero has no multiplicative inverse. Its returned value is zero and invalid.

```
fn gf256_inverse(a:bits<8>) -> Gf256Inverse
```

Exact declaration · Contract and restrictions

13.23 `std.coding.rs_dvb`

Import: `import std.coding.rs_dvb`.

Component contracts · Source

Import-only namespace; its source imports the component declarations listed in this catalog.

13.24 `std.coding.rs_dvb_arithmetic`

Import: `import std.coding.rs_dvb_arithmetic`.

Component contracts · Source

13.24.1 `rs_dvb_parity_coefficients` — function

Highest-degree first, omitting the monic coefficient of the degree-16 generator product (x+alpha^0)...(x+alpha^15), alpha=0x02.

```
fn rs_dvb_parity_coefficients() -> vec<16,bits<8>>
```

Exact declaration · Contract and restrictions

13.24.2 `rs_dvb_roots` — function

```
fn rs_dvb_roots() -> vec<16,bits<8>>
```

Exact declaration · Contract and restrictions

13.24.3 `rs_dvb_parity_step` — function

```
fn rs_dvb_parity_step(state:vec<16,bits<8>>,data:bits<8>) -> vec<16,bits<8>>
```

Exact declaration · Contract and restrictions

13.24.4 `rs_dvb_syndrome_step` — function

```
fn rs_dvb_syndrome_step(state:vec<16,bits<8>>,data:bits<8>) -> vec<16,bits<8>>
```

Exact declaration · Contract and restrictions

13.25 `std.coding.rs_dvb_decode`

Import: `import std.coding.rs_dvb_decode`.

Component contracts · Source

13.25.1 `RsDvbDecodePhase` — enum

```
enum RsDvbDecodePhase
receive
bm_start
bm_delta
bm_ratio
bm_update
bm_finish
locator_check
omega_start
omega_accum
omega_store
scan_start
scan_poly
scan_ratio
scan_correct
scan_advance
roots_check
recheck
status_wait
drain
```

Exact declaration · Contract and restrictions

13.25.2 `RvRsDvb204188Decode` — module

One frame, one shared variable GF multiplier. Polynomial evaluation and BM updates are explicit sequential datapaths, not automatic scheduling. Packet storage is registers; no BRAM or fixed decode latency is promised.

```
module RvRsDvb204188Decode
clock clk
reset rst
in input : rv<RsByteBeat>
out status : rv<RsFrameStatus>
out output : rv<RsByteBeat>
```

Exact declaration · Contract and restrictions

13.26 `std.coding.rs_dvb_decoder_inverse`

Import: `import std.coding.rs_dvb_decoder_inverse`.

Component contracts · Source

13.26.1 `rs_dvb_inverse_image` — function

The fixed GF(256)/0x11D inverse image. Zero maps to zero; the caller reports it invalid. All256 entries are independently checked against polynomial arithmetic by the actual-RTL kernel qualification.

```
fn rs_dvb_inverse_image() -> vec<256,bits<8>>
```

Exact declaration · Contract and restrictions

13.26.2 `rs_dvb_inverse_value` — function

```
fn rs_dvb_inverse_value(a:bits<8>) -> bits<8>
```

Exact declaration · Contract and restrictions

13.27 `std.coding.rs_dvb_decoder_kernels`

Import: `import std.coding.rs_dvb_decoder_kernels`.

Component contracts · Source

13.27.1 `RsDvbMultiplyKernel` — module

```
module RsDvbMultiplyKernel
in a,b:bits<8>
out product:bits<8>
```

Exact declaration · Contract and restrictions

13.27.2 `RsDvbBmKernel` — module

Pure combinational boundaries, with no hidden registers or scheduling.

```
module RsDvbBmKernel
in state:RsDvbBmState
in syndromes:vec<16,bits<8>>
in step:u5
in delta:bits<8>
out updated:RsDvbBmState
out discrepancy,omega:bits<8>
```

Exact declaration · Contract and restrictions

13.27.3 `RsDvbPolyKernel` — module

```
module RsDvbPolyKernel
in coefficients:vec<17,bits<8>>
in x:bits<8>
out value:bits<8>
```

Exact declaration · Contract and restrictions

13.27.4 `RsDvbInverseKernel` — module

```
module RsDvbInverseKernel
in value:bits<8>
out result:Gf256Inverse
```

Exact declaration · Contract and restrictions

13.27.5 `RsDvbSyndromeKernel` — module

```
module RsDvbSyndromeKernel
in active:bit
in state:vec<16,bits<8>>
in data:bits<8>
out value:vec<16,bits<8>>
```

Exact declaration · Contract and restrictions

13.27.6 `RsDvbSelectKernel` — module

```
module RsDvbSelectKernel
in packet:vec<204,bits<8>>
in index:u8
out value:bits<8>
```

Exact declaration · Contract and restrictions

13.28 `std.coding.rs_dvb_decoder_math`

Import: `import std.coding.rs_dvb_decoder_math`.

Component contracts · Source

13.28.1 `rs_dvb_select` — function

Total constant-index selection, including out-of-range zero. Callers own bounded indices; no illegal dynamic vector access is masked after the fact.

```
fn rs_dvb_select<N,IW>(values:vec<N,bits<8>>,index:uint<IW>) -> bits<8>
```

Exact declaration · Contract and restrictions

13.28.2 `RsDvbBmState` — struct

```
struct RsDvbBmState
locator:vec<17,bits<8>>
previous_locator:vec<17,bits<8>>
degree:u5
distance:u5
previous:bits<8>
```

Exact declaration · Contract and restrictions

13.28.3 `rs_dvb_discrepancy` — function

```
fn rs_dvb_discrepancy(state:RsDvbBmState,syndromes:vec<16,bits<8>>,step:u5) -> bits<8>
```

Exact declaration · Contract and restrictions

13.28.4 `rs_dvb_bm_update` — function

The controller initializes previous=1 and only replaces it with a nonzero discrepancy. It checks that invariant before invoking this update.

```
fn rs_dvb_bm_update(state:RsDvbBmState,step:u5,delta:bits<8>) -> RsDvbBmState
```

Exact declaration · Contract and restrictions

13.28.5 `rs_dvb_omega_coefficient` — function

```
fn rs_dvb_omega_coefficient(locator:vec<17,bits<8>>,syndromes:vec<16,bits<8>>,index:u5) ->
     bits<8>
```

Exact declaration · Contract and restrictions

13.28.6 `rs_dvb_poly_value` — function

Ascending coefficients, degree at most16. Callers explicitly zero-pad lower degrees; the fixed input type cannot silently discard coefficients.

```
fn rs_dvb_poly_value(coefficients:vec<17,bits<8>>,x:bits<8>) -> bits<8>
```

Exact declaration · Contract and restrictions

13.29 `std.coding.rs_dvb_encode`

Import: `import std.coding.rs_dvb_encode`.

Component contracts · Source

13.29.1 `RvRsDvb204188Encode` — module

Complete-frame, non-overlapped systematic RS(204,188). The explicit shift buffer is register storage, not a claim of inferred BRAM or hidden memory.

```
module RvRsDvb204188Encode
clock clk
reset rst
in input : rv<RsByteBeat>
out status : rv<RsFrameStatus>
out output : rv<RsByteBeat>
```

Exact declaration · Contract and restrictions

13.30 `std.coding.rs_dvb_framing`

Import: `import std.coding.rs_dvb_framing`.

Component contracts · Source

13.30.1 `RsByteBeat` — struct

```
struct RsByteBeat
data:bits<8>
first:bit
last:bit
```

Exact declaration · Contract and restrictions

13.30.2 `RsStatus` — enum

```
enum RsStatus
clean
corrected
detected_uncorrectable
malformed
```

Exact declaration · Contract and restrictions

13.30.3 `RsFrameStatus` — struct

```
struct RsFrameStatus
status:RsStatus
correction_count:uint<4>
```

Exact declaration · Contract and restrictions

13.30.4 `RsDvbFraming` — module

Marker/count ownership only. Storage and coding arithmetic belong to the caller. After a malformed unterminated frame, discard until last or first.

```
module RsDvbFraming<N> where N == 188 || N == 204
clock clk
reset rst
in valid,first,last,release : bit
out ready,take,complete,malformed : bit
```

Exact declaration · Contract and restrictions

13.31 `std.coding.scrambler`

Import: `import std.coding.scrambler`.

Component contracts · Source

13.31.1 `ScramblerResult24` — struct

x^7+x^4+1, DATA bit zero first. Tap masks are the 24-step linear expansion of feedback = state[0] ^ state[3], state' = {feedback,state[6:1]}.

```
struct ScramblerResult24
data : bits<24>
state : u7
```

Exact declaration · Contract and restrictions

13.31.2 `scrambler7_taps24` — function

```
fn scrambler7_taps24() -> vec<24,u7>
```

Exact declaration · Contract and restrictions

13.31.3 `coding_scramble7_24` — function

```
fn coding_scramble7_24(word:bits<24>, state:u7, clear_mask:bits<24>) -> ScramblerResult24
```

Exact declaration · Contract and restrictions

13.31.4 `ScramblerWord24` — struct

```
struct ScramblerWord24
data : bits<24>
restart : bit
clear_mask : bits<24>
```

Exact declaration · Contract and restrictions

13.31.5 `RvScrambler7_24` — module

Explicit per-word restart and tail clearing. No packet metadata/rate policy.

```
module RvScrambler7_24<SEED=0x4b> where SEED >= 1 && SEED < 128
clock clk
reset rst
in input : rv<ScramblerWord24>
out output : rv<bits<24>>
```

Exact declaration · Contract and restrictions

13.32 `std.coding.secded`

Import: `import std.coding.secded`.

Component contracts · Source

13.32.1 `SecdedStatus` — enum

Extended Hamming: data LSB first in increasing non-power-of-two positions; Hamming parity in positions 1,2,4,... and overall parity in the final bit.

```
enum SecdedStatus
clean
corrected
detected_uncorrectable
```

Exact declaration · Contract and restrictions

13.32.2 `SecdedDecoded` — struct

```
struct SecdedDecoded<K,R>
data : bits<K>
syndrome : uint<R>
status : SecdedStatus
data_valid : bit
```

Exact declaration · Contract and restrictions

13.32.3 `secded_column_mask` — function

Compile-time column mask; bit i belongs to parity equation J iff the one-based Hamming position i+1 has bit J set.

```
fn secded_column_mask<H,J>() -> bits<H>
```

Exact declaration · Contract and restrictions

13.32.4 `secded_encode` — function

```
fn secded_encode<K,Q=ceil_log2(K+1),R=ceil_log2(K+Q+1),H=K+R>(
    data:bits<K>) -> bits<H+1>
```

Exact declaration · Contract and restrictions

13.32.5 `secded_decode` — function

```
fn secded_decode<K,Q=ceil_log2(K+1),R=ceil_log2(K+Q+1),H=K+R>(
    received:bits<H+1>) -> SecdedDecoded<K,R>
```

Exact declaration · Contract and restrictions

13.32.6 `RvSecdedEncode` — module

```
module RvSecdedEncode<K,Q=ceil_log2(K+1),R=ceil_log2(K+Q+1),H=K+R>
where K >= 1 && K <= 64 && Q == ceil_log2(K+1) && H == K+R
   && (1 << R) >= K+R+1 && (1 << (R-1)) < K+R
clock clk
reset rst
in input : rv<bits<K>>
out output : rv<bits<H+1>>
```

Exact declaration · Contract and restrictions

13.32.7 `RvSecdedDecode` — module

```
module RvSecdedDecode<K,Q=ceil_log2(K+1),R=ceil_log2(K+Q+1),H=K+R>
where K >= 1 && K <= 64 && Q == ceil_log2(K+1) && H == K+R
   && (1 << R) >= K+R+1 && (1 << (R-1)) < K+R
clock clk
reset rst
in input : rv<bits<H+1>>
out output : rv<SecdedDecoded<K,R>>
```

Exact declaration · Contract and restrictions

13.33 `std.coding.viterbi_k7`

Import: `import std.coding.viterbi_k7`.

Component contracts · Source

Import-only namespace; its source imports the component declarations listed in this catalog.

13.34 `std.coding.viterbi_k7_core`

Import: `import std.coding.viterbi_k7_core`.

Component contracts · Source

13.34.1 `ViterbiSoftPair` — struct

```
struct ViterbiSoftPair
q0:u3
q1:u3
first:bit
last:bit
```

Exact declaration · Contract and restrictions

13.34.2 `ViterbiHardPair` — struct

```
struct ViterbiHardPair
pair:bits<2>
first:bit
last:bit
```

Exact declaration · Contract and restrictions

13.34.3 `ViterbiDecodedBit` — struct

```
struct ViterbiDecodedBit
data:bit
first:bit
last:bit
```

Exact declaration · Contract and restrictions

13.34.4 `ViterbiFrameStatus` — struct

```
struct ViterbiFrameStatus<MW,CW>
malformed:bit
decoded_bits:uint<CW>
path_metric:uint<MW>
```

Exact declaration · Contract and restrictions

13.34.5 `ViterbiK7Phase` — enum

```
enum ViterbiK7Phase
receive
trace_request
trace_apply
status_wait
output_request
output_hold
```

Exact declaration · Contract and restrictions

13.34.6 `RvViterbiK7Soft` — module

One frame, 64 parallel ACS lanes, full survivor RAM and complete traceback. No tail bits or final-state-zero assumption. Drain uses one bit per two unstalled edges; input ACS accepts one received pair per edge.

```
module RvViterbiK7Soft<MAX_BITS=4096,MW=ceil_log2(14*MAX_BITS+1),
                       AW=index_width(MAX_BITS),D=1<<AW,CW=index_width(MAX_BITS+1)>
where MAX_BITS >= 1 && MAX_BITS <= 4096 && MW == ceil_log2(14*MAX_BITS+1)
   && AW == index_width(MAX_BITS) && D == (1<<AW) && CW == index_width(MAX_BITS+1)
clock clk
reset rst
in input : rv<ViterbiSoftPair>
out status : rv<ViterbiFrameStatus<MW,CW>>
out output : rv<ViterbiDecodedBit>
```

Exact declaration · Contract and restrictions

13.34.7 `RvViterbiK7Hard` — module

```
module RvViterbiK7Hard<MAX_BITS=4096,MW=ceil_log2(14*MAX_BITS+1),CW=index_width(MAX_BITS
    +1)>
where MAX_BITS >= 1 && MAX_BITS <= 4096 && MW == ceil_log2(14*MAX_BITS+1)
   && CW == index_width(MAX_BITS+1)
clock clk
reset rst
in input : rv<ViterbiHardPair>
out status : rv<ViterbiFrameStatus<MW,CW>>
out output : rv<ViterbiDecodedBit>
```

Exact declaration · Contract and restrictions

13.35 `std.coding.viterbi_k7_math`

Import: `import std.coding.viterbi_k7_math`.

Component contracts · Source

13.35.1 `ViterbiK7Trellis` — struct

Matches the existing encoder's bit-zero-first history convention.

```
struct ViterbiK7Trellis
pair:bits<2>
next:u6
```

Exact declaration · Contract and restrictions

13.35.2 `viterbi_k7_trellis` — function

```
fn viterbi_k7_trellis(history:u6,value:bit) -> ViterbiK7Trellis
```

Exact declaration · Contract and restrictions

13.35.3 `ViterbiK7Decision` — struct

```
struct ViterbiK7Decision<MW>
metric:uint<MW>
valid:bit
predecessor_lsb:bit
```

Exact declaration · Contract and restrictions

13.35.4 `viterbi_k7_branch_metric` — function

```
fn viterbi_k7_branch_metric(expected:bits<2>,q0:u3,q1:u3) -> u4
```

Exact declaration · Contract and restrictions

13.35.5 `viterbi_k7_select` — function

```
fn viterbi_k7_select<MW>(expected0:bits<2>,expected1:bits<2>,
    m0:uint<MW>,m1:uint<MW>,v0:bit,v1:bit,q0:u3,q1:u3) -> ViterbiK7Decision<MW>
```

Exact declaration · Contract and restrictions

13.35.6 `viterbi_k7_acs` — function

Callers supply an even predecessor P and input U=0/1. Metrics are exact while the frame-bound range certificate holds; the full sums drive choice.

```
fn viterbi_k7_acs<MW,P,U>(m0:uint<MW>,m1:uint<MW>,v0:bit,v1:bit,q0:u3,q1:u3)
    -> ViterbiK7Decision<MW>
```

Exact declaration · Contract and restrictions

13.35.7 `ViterbiK7Minimum` — struct

```
struct ViterbiK7Minimum<MW>
metric:uint<MW>
node:u6
valid:bit
```

Exact declaration · Contract and restrictions

13.35.8 `+` — operator

A closed min-selection algebra for the source-defined balanced sum tree. The node number makes ties independent of tree shape; invalid is separate.

```
operator +<MW>(a:ViterbiK7Minimum<MW>,b:ViterbiK7Minimum<MW>) -> ViterbiK7Minimum<MW>
```

Exact declaration · Contract and restrictions

13.35.9 `viterbi_k7_decisions` — function

```
fn viterbi_k7_decisions<MW>(metrics:vec<64,uint<MW>>,reachable:bits<64>,q0:u3,q1:u3)
    -> vec<64,ViterbiK7Decision<MW>>
```

Exact declaration · Contract and restrictions

13.35.10 `viterbi_k7_minimum_entry` — function

```
fn viterbi_k7_minimum_entry<MW>(value:ViterbiK7Decision<MW>,node:u6) -> ViterbiK7Minimum<
    MW>
```

Exact declaration · Contract and restrictions

13.35.11 `viterbi_k7_minimum` — function

```
fn viterbi_k7_minimum<MW>(decisions:vec<64,ViterbiK7Decision<MW>>) -> ViterbiK7Minimum<MW>
```

Exact declaration · Contract and restrictions

13.36 `std.coding.viterbi_k7_storage`

Import: `import std.coding.viterbi_k7_storage`.

Component contracts · Source

13.36.1 `ViterbiK7Storage` — module

Isolated source-owned RAM: the controller owns state/rules in its parent. This uses the existing 1R1W primitive, not a new memory mapping/provider.

```
module ViterbiK7Storage<type T,N,AW>
where N >= 2 && is_power_of_two(N) && AW == floor_log2(N)
clock clk
reset rst
in write,read : bit
in write_address,read_address : uint<AW>
in write_data : T
out read_data : T
```

Exact declaration · Contract and restrictions

13.37 `std.coding.viterbi_k7_word`

Import: `import std.coding.viterbi_k7_word`.

Component contracts · Source

13.37.1 `ViterbiK7Word48` — struct

```
struct ViterbiK7Word48
data:bits<48>
first:bit
last:bit
```

Exact declaration · Contract and restrictions

13.37.2 `RvViterbiK7Word48` — module

The existing encoder emits its first G0/G1 pair in bits 47:46. Frame markers are explicit; finish on the encoder is not a transmitted tail.

```
module RvViterbiK7Word48<MAX_BITS=4096,MW=ceil_log2(14*MAX_BITS+1),CW=index_width(MAX_BITS
    +1)>
where MAX_BITS >= 24 && MAX_BITS <= 4096 && MW == ceil_log2(14*MAX_BITS+1)
   && CW == index_width(MAX_BITS+1)
clock clk
reset rst
in input : rv<ViterbiK7Word48>
out status : rv<ViterbiFrameStatus<MW,CW>>
out output : rv<ViterbiDecodedBit>
```

Exact declaration · Contract and restrictions

13.38 `std.coding`

Import: `import std.coding`.

Component contracts · Source

Import-only namespace; its source imports the component declarations listed in this catalog.

13.39 `std.dsp.cic`

Import: `import std.dsp.cic`.

Component contracts · Source

13.39.1 `CicIntegrator` — module

Modular arithmetic is intentional: all state and differences wrap at W bits.

```
module CicIntegrator<W> where W >= 1
clock clk
reset rst
in input : sint<W>
in advance : bit
out integrated : sint<W>
```

Exact declaration · Contract and restrictions

13.39.2 `CicComb` — module

```
module CicComb<W,M> where W >= 1 && M >= 1
clock clk
reset rst
in input : sint<W>
in advance : bit
out difference : sint<W>
```

Exact declaration · Contract and restrictions

13.39.3 `RvCicDecimator` — module

No gain compensation: DC gain is (R*M)^N. IW is caller-owned headroom.

```
module RvCicDecimator<SW,IW,R,N,M=1,CW=index_width(R)>
where SW >= 1 && IW >= SW && R >= 1 && N >= 1 && M >= 1
   && CW == index_width(R)
clock clk
reset rst
in input : rv<sint<SW>>
out output : rv<sint<IW>>
```

Exact declaration · Contract and restrictions

13.39.4 `RvCicInterpolator` — module

Low-rate combs, zero insertion, then high-rate integrators. All IW-bit operations wrap explicitly. No gain compensation: DC gain is (R*M)^N/R.

```
module RvCicInterpolator<SW,IW,R,N,M=1,CW=index_width(R)>
where SW >= 1 && IW >= SW && R >= 1 && N >= 1 && M >= 1
   && CW == index_width(R)
clock clk
reset rst
in input : rv<sint<SW>>
out output : rv<sint<IW>>
```

Exact declaration · Contract and restrictions

13.40 `std.dsp.cordic`

Import: `import std.dsp.cordic`.

Component contracts · Source

13.40.1 `CordicState` — struct

```
struct CordicState<P,F>
x : sint<F+3>
y : sint<F+3>
z : sint<P+1>
```

Exact declaration · Contract and restrictions

13.40.2 `cordic_rotation_step` — function

Simultaneous old-x/old-y updates; signed shifts round toward -infinity.

```
fn cordic_rotation_step<P,F,I>(x:sint<F+3>, y:sint<F+3>, z:sint<P+1>) -> CordicState<P,F>
```

Exact declaration · Contract and restrictions

13.40.3 `RvCordicSinCos` — module

Fixed rotation-only profiles. Full pipeline freezes under output backpressure. N+2 unstalled rising edges, II=1; output rounding is the only final conversion.

```
module RvCordicSinCos<P=16,N=P-2,OW=16,OF=14,G=4,F=OF+G,W=F+3,STAGES=N+1>
where (P == 8 || P == 12 || P == 16) && N >= 1 && N <= P-2
   && OW == 16 && OF == 14 && G == 4 && F == OF+G && W == F+3
   && STAGES == N+1
clock clk
reset rst
in input : rv<sint<P>>
out output : rv<SinCos<fixed<OW,OF>>>
```

Exact declaration · Contract and restrictions

13.41 `std.dsp.cordic_constants`

Import: `import std.dsp.cordic_constants`.

Component contracts · Source

13.41.1 `cordic_angles_q48` — function

Immutable Q48 tables, schema cordic-rotation-q48-v1, i=0..13 / N=1..14. Rounded nearest-even from rational atan/pi and inverse-square-root intervals. Coarser admitted P/F rounding cells are independently certified in tests.

```
fn cordic_angles_q48() -> vec<14,u49>
```

Exact declaration · Contract and restrictions

13.41.2 `cordic_inverse_gains_q48` — function

```
fn cordic_inverse_gains_q48() -> vec<14,u49>
```

Exact declaration · Contract and restrictions

13.41.3 `cordic_angle` — function

```
fn cordic_angle<P,I>() -> sint<P+1>
```

Exact declaration · Contract and restrictions

13.41.4 `cordic_inverse_gain` — function

```
fn cordic_inverse_gain<F,N>() -> sint<F+3>
```

Exact declaration · Contract and restrictions

13.42 `std.dsp.cordic_vectoring`

Import: `import std.dsp.cordic_vectoring`.

Component contracts · Source

13.42.1 `CartesianFixed` — struct

```
struct CartesianFixed<SW,SF>
x : fixed<SW,SF>
y : fixed<SW,SF>
```

Exact declaration · Contract and restrictions

13.42.2 `PolarFixed` — struct

```
struct PolarFixed<P,SW,SF>
phase : sint<P>
magnitude : ufixed<SW,SF>
angle_defined : bit
```

Exact declaration · Contract and restrictions

13.42.3 `VectoringState` — struct

```
struct VectoringState<W,Z>
x : sint<W>
y : sint<W>
z : sint<Z>
```

Exact declaration · Contract and restrictions

13.42.4 `cordic_vector_step` — function

```
fn cordic_vector_step<W,Z,A,I>(x:sint<W>, y:sint<W>, z:sint<Z>) -> VectoringState<W,Z>
```

Exact declaration · Contract and restrictions

13.42.5 `RvCordicVectoring` — module

Three bounded profiles. Phase is binary turns; magnitude is gain-corrected. Pre-normalization, N rotations, gain and conversion: N+3 edges, II=1.

```
module RvCordicVectoring<SW=16,SF=SW-2,P=SW,N=P-2,F=SF+4,W=F+4,
                         A=P+4,Z=A+2,S=N+1,B=index_width(SW+1),MW=W+F+3+SW>
where (SW == 8 || SW == 12 || SW == 16) && SF == SW-2 && P == SW
   && N == P-2 && F == SF+4 && W == F+4 && A == P+4 && Z == A+2
   && S == N+1 && B == index_width(SW+1) && MW == W+F+3+SW
clock clk
reset rst
in input : rv<CartesianFixed<SW,SF>>
out output : rv<PolarFixed<P,SW,SF>>
```

Exact declaration · Contract and restrictions

13.42.6 `Atan2Result` — struct

```
struct Atan2Result<P>
phase : sint<P>
angle_defined : bit
```

Exact declaration · Contract and restrictions

13.42.7 `RvAtan2` — module

```
module RvAtan2<SW=16,SF=SW-2,P=SW>
clock clk
reset rst
in input : rv<CartesianFixed<SW,SF>>
out output : rv<Atan2Result<P>>
```

Exact declaration · Contract and restrictions

13.43 `std.dsp.farrow`

Import: `import std.dsp.farrow`.

Component contracts · Source

13.43.1 `FarrowSample` — struct

```
struct FarrowSample<SW,SF,MF>
sample : fixed<SW,SF>
mu_raw : uint<MF>
```

Exact declaration · Contract and restrictions

13.43.2 `farrow_horner_step` — function

The extra zero bit makes mu a nonnegative signed fixed-point operand. Product and addition widths are exact; there is no intermediate conversion.

```
fn farrow_horner_step<B,F,MF,I>(
    value:fixed<B+I*(MF+2),F+I*MF>, addend:fixed<B,F>, mu_raw:uint<MF>)
```

Exact declaration · Contract and restrictions

13.43.3 `RvFarrowFixedExact` — module

```
module RvFarrowFixedExact<SW,SF,CW,CF,M,K,MF,J=M+1,
                          B=SW+CW+ceil_log2(K),F=SF+CF,
                          coefficients:vec<J,vec<K,fixed<CW,CF>>>>
where M >= 0 && M <= 3 && K >= 1 && K <= 8 && MF >= 1
   && J == M+1 && B == SW+CW+ceil_log2(K) && F == SF+CF
clock clk
reset rst
in input : rv<FarrowSample<SW,SF,MF>>
out output : rv<fixed<B+M*(MF+2),F+M*MF>>
```

Exact declaration · Contract and restrictions

13.43.4 `RvFarrowFixedNearestEvenSaturate` — module

```
module RvFarrowFixedNearestEvenSaturate<SW,SF,CW,CF,M,K,MF,OW,OF,J=M+1,
                                        coefficients:vec<J,vec<K,fixed<CW,CF>>>>
where M >= 0 && M <= 3 && K >= 1 && K <= 8 && MF >= 1 && J == M+1
clock clk
reset rst
in input : rv<FarrowSample<SW,SF,MF>>
out output : rv<fixed<OW,OF>>
```

Exact declaration · Contract and restrictions

13.43.5 `farrow_lagrange_linear_q16` — function

Stored binary coefficients, nearest-even Q2.16 approximations. The cubic polynomial approximates Lagrange delay 1+mu; thirds are not exact rationals.

```
fn farrow_lagrange_linear_q16() -> vec<2,vec<2,fixed<18,16>>>
```

Exact declaration · Contract and restrictions

13.43.6 `farrow_lagrange_cubic_q16` — function

```
fn farrow_lagrange_cubic_q16() -> vec<4,vec<4,fixed<18,16>>>
```

Exact declaration · Contract and restrictions

13.43.7 `RvFarrowLagrangeLinear` — module

```
module RvFarrowLagrangeLinear<SW,SF,MF> where MF >= 1
clock clk
reset rst
in input : rv<FarrowSample<SW,SF,MF>>
out output : rv<fixed<SW+MF+21,SF+MF+16>>
```

Exact declaration · Contract and restrictions

13.43.8 `RvFarrowLagrangeCubic` — module

```
module RvFarrowLagrangeCubic<SW,SF,MF> where MF >= 1
clock clk
reset rst
in input : rv<FarrowSample<SW,SF,MF>>
out output : rv<fixed<SW+3*MF+26,SF+3*MF+16>>
```

Exact declaration · Contract and restrictions

13.44 `std.dsp.fft`

Import: `import std.dsp.fft`.

Component contracts · Source

13.44.1 `FFTButterfly` — struct

```
struct FFTButterfly<type S, type D>
sum : S
difference : D
```

Exact declaration · Contract and restrictions

13.44.2 `fft_butterfly_exact` — function

```
fn fft_butterfly_exact<type A, type B, type W>(
    a : Complex<A>, b : Complex<B>, twiddle : Complex<W>
)
```

Exact declaration · Contract and restrictions

13.44.3 `fft_bit_reverse` — function

```
fn fft_bit_reverse<N>(value : bits<N>)
```

Exact declaration · Contract and restrictions

13.44.4 `fft_forward_twiddles` — function

```
fn fft_forward_twiddles<type T,N>()
```

Exact declaration · Contract and restrictions

13.44.5 `fft_inverse_twiddles` — function

```
fn fft_inverse_twiddles<type T,N>()
```

Exact declaration · Contract and restrictions

13.45 `std.dsp.fir`

Import: `import std.dsp.fir`.

Component contracts · Source

13.45.1 `RvFirFixedExact` — module

Immutable coefficient FIR. Exact products and the language's balanced dot reduction precede the only output register. No implementation intent or target-specific architecture is needed to use this parallel II=1 component. H is a checked derived dimension because vec lengths use literal/parameter names; callers leave it at N-1. It does not select a different architecture.

```
module RvFirFixedExact<SW,SF,CW,CF,N,H=N-1,coefficients:vec<N,fixed<CW,CF>>>
where N >= 1 && H == N-1
clock clk
reset rst
in input : rv<fixed<SW,SF>>
out output : rv<fixed<SW+CW+ceil_log2(N),SF+CF>>
```

Exact declaration · Contract and restrictions

13.45.2 `RvFirFixedNearestEvenSaturate` — module

One explicit numerical boundary after full-precision accumulation. This wrapper is combinational after the core's output register: no extra cycle.

```
module RvFirFixedNearestEvenSaturate<SW,SF,CW,CF,N,OW,OF,
                                     coefficients:vec<N,fixed<CW,CF>>>
where N >= 1
clock clk
reset rst
in input : rv<fixed<SW,SF>>
out output : rv<fixed<OW,OF>>
```

Exact declaration · Contract and restrictions

13.46 `std.dsp.fir_coefficients`

Import: `import std.dsp.fir_coefficients`.

Component contracts · Source

13.46.1 `FirCoefficientBanks` — module

One owner of coefficient epochs. Loading never changes the active bank.

```
module FirCoefficientBanks<CW,CF,N,IW=index_width(N+1),
                           initial_coefficients:vec<N,fixed<CW,CF>>>
where N >= 1 && IW == index_width(N+1)
clock clk
reset rst
in reload : rv<fixed<CW,CF>>
in commit : bit
in idle : bit
out commit_ready : bit
out block_input : bit
out coefficients : vec<N,fixed<CW,CF>>
```

Exact declaration · Contract and restrictions

13.47 `std.dsp.fir_folded`

Import: `import std.dsp.fir_folded`.

Component contracts · Source

13.47.1 `fir_folded_select` — function

Constant indices keep selection valid for every N, including non-powers of two. This is a bit-exact selector, not a numeric conversion or a table load.

```
fn fir_folded_select<W,F,N,IW>(values:vec<N,fixed<W,F>>, index:uint<IW>) -> fixed<W,F>
```

Exact declaration · Contract and restrictions

13.47.2 `FirFoldedRuntimeExact` — module

The external bank owner holds coefficients while this datapath is busy. One accepted window, one logical product, one full-precision accumulator.

```
module FirFoldedRuntimeExact<SW,SF,CW,CF,N,IW=index_width(N),PW=SW+CW,
                             AW=PW+ceil_log2(N)>
where N >= 1 && N <= 32 && IW == index_width(N)
   && PW == SW+CW && AW == PW+ceil_log2(N)
clock clk
reset rst
in input : rv<fixed<SW,SF>>
in coefficients : vec<N,fixed<CW,CF>>
in block_input : bit
out output : rv<fixed<AW,SF+CF>>
out idle : bit
```

Exact declaration · Contract and restrictions

13.47.3 `RvFirFoldedExact` — module

```
module RvFirFoldedExact<SW,SF,CW,CF,N,coefficients:vec<N,fixed<CW,CF>>>
where N >= 1 && N <= 32
clock clk
reset rst
in input : rv<fixed<SW,SF>>
out output : rv<fixed<SW+CW+ceil_log2(N),SF+CF>>
```

Exact declaration · Contract and restrictions

13.47.4 `RvFirFoldedReloadableExact` — module

```
module RvFirFoldedReloadableExact<SW,SF,CW,CF,N,
                                  initial_coefficients:vec<N,fixed<CW,CF>>>
where N >= 1 && N <= 32
clock clk
reset rst
in input : rv<fixed<SW,SF>>
in reload : rv<fixed<CW,CF>>
in commit : bit
out commit_ready : bit
out output : rv<fixed<SW+CW+ceil_log2(N),SF+CF>>
```

Exact declaration · Contract and restrictions

13.47.5 `RvFirFoldedNearestEvenSaturate` — module

The only quantization is after the exact core's pending-result register.

```
module RvFirFoldedNearestEvenSaturate<SW,SF,CW,CF,N,OW,OF,
                                      coefficients:vec<N,fixed<CW,CF>>>
where N >= 1 && N <= 32
clock clk
reset rst
in input : rv<fixed<SW,SF>>
out output : rv<fixed<OW,OF>>
```

Exact declaration · Contract and restrictions

13.47.6 `RvFirFoldedReloadableNearestEvenSaturate` — module

```
module RvFirFoldedReloadableNearestEvenSaturate<SW,SF,CW,CF,N,OW,OF,
                                                initial_coefficients:vec<N,fixed<CW,CF>>>
where N >= 1 && N <= 32
clock clk
reset rst
in input : rv<fixed<SW,SF>>
in reload : rv<fixed<CW,CF>>
in commit : bit
out commit_ready : bit
out output : rv<fixed<OW,OF>>
```

Exact declaration · Contract and restrictions

13.48 `std.dsp.fir_reloadable`

Import: `import std.dsp.fir_reloadable`.

Component contracts · Source

13.48.1 `FirRuntimeExact` — module

Exact parallel datapath. External bank owner freezes coefficients on commit.

```
module FirRuntimeExact<SW,SF,CW,CF,N,H=N-1>where N >= 1 && H == N-1
clock clk
reset rst
in input : rv<fixed<SW,SF>>
in coefficients : vec<N,fixed<CW,CF>>
in block_input : bit
out output : rv<fixed<SW+CW+ceil_log2(N),SF+CF>>
out idle : bit
```

Exact declaration · Contract and restrictions

13.48.2 `RvFirReloadableExact` — module

```
module RvFirReloadableExact<SW,SF,CW,CF,N,initial_coefficients:vec<N,fixed<CW,CF>>>
where N >= 1
clock clk
reset rst
in input : rv<fixed<SW,SF>>
in reload : rv<fixed<CW,CF>>
in commit : bit
out commit_ready : bit
out output : rv<fixed<SW+CW+ceil_log2(N),SF+CF>>
```

Exact declaration · Contract and restrictions

13.48.3 `RvFirReloadableNearestEvenSaturate` — module

```
module RvFirReloadableNearestEvenSaturate<SW,SF,CW,CF,N,OW,OF,
                                          initial_coefficients:vec<N,fixed<CW,CF>>>
where N >= 1
clock clk
reset rst
in input : rv<fixed<SW,SF>>
in reload : rv<fixed<CW,CF>>
in commit : bit
out commit_ready : bit
out output : rv<fixed<OW,OF>>
```

Exact declaration · Contract and restrictions

13.49 `std.dsp.halfband`

Import: `import std.dsp.halfband`.

Component contracts · Source

13.49.1 `HalfbandPrototypeContract` — module

Pure compile-time validation. The empty contract instance creates no state or datapath; an invalid immutable prototype rejects at specialization.

```
module HalfbandPrototypeContract<valid:bit> where valid == 1
```

Exact declaration · Contract and restrictions

13.49.2 `halfband_prototype_valid` — function

```
fn halfband_prototype_valid<N,CW,CF>(coefficients:vec<N,fixed<CW,CF>>) -> bit
```

Exact declaration · Contract and restrictions

13.49.3 `halfband_exact` — function

Symmetric pairs use SW+1 exact pre-adders. Structurally zero taps have no multiplier. Only redundant accumulation guards are removed at the end.

```
fn halfband_exact<SW,SF,CW,CF,N,C=(N-1)/2,Q=(C+(C%2))/2,AW=SW+CW+ceil_log2(N)>(
    samples:vec<N,fixed<SW,SF>>, coefficients:vec<N,fixed<CW,CF>>) -> fixed<AW,SF+CF>
```

Exact declaration · Contract and restrictions

13.49.4 `RvHalfbandDecimatorExact` — module

```
module RvHalfbandDecimatorExact<SW,SF,CW,CF,N,H=N-1,AW=SW+CW+ceil_log2(N),
                                coefficients:vec<N,fixed<CW,CF>>>
where N >= 3 && N%2 == 1 && H == N-1 && AW == SW+CW+ceil_log2(N)
clock clk
reset rst
in input : rv<fixed<SW,SF>>
out output : rv<fixed<AW,SF+CF>>
```

Exact declaration · Contract and restrictions

13.49.5 `RvHalfbandInterpolatorExact` — module

```
module RvHalfbandInterpolatorExact<SW,SF,CW,CF,N,K=(N+1)/2,H=K-1,
                                   AW=SW+CW+ceil_log2(N),coefficients:vec<N,fixed<CW,CF>>>
where N >= 3 && N%2 == 1 && K == (N+1)/2 && H == K-1 && AW == SW+CW+ceil_log2(N)
clock clk
reset rst
in input : rv<fixed<SW,SF>>
out output : rv<fixed<AW,SF+CF>>
```

Exact declaration · Contract and restrictions

13.49.6 `RvHalfbandDecimatorNearestEvenSaturate` — module

```
module RvHalfbandDecimatorNearestEvenSaturate<SW,SF,CW,CF,N,OW,OF,
                                              coefficients:vec<N,fixed<CW,CF>>>
where N >= 3 && N%2 == 1
clock clk
reset rst
in input : rv<fixed<SW,SF>>
out output : rv<fixed<OW,OF>>
```

Exact declaration · Contract and restrictions

13.49.7 `RvHalfbandInterpolatorNearestEvenSaturate` — module

```
module RvHalfbandInterpolatorNearestEvenSaturate<SW,SF,CW,CF,N,OW,OF,
                                                 coefficients:vec<N,fixed<CW,CF>>>
where N >= 3 && N%2 == 1
clock clk
reset rst
in input : rv<fixed<SW,SF>>
out output : rv<fixed<OW,OF>>
```

Exact declaration · Contract and restrictions

13.50 `std.dsp.nco`

Import: `import std.dsp.nco`.

Component contracts · Source

13.50.1 `phase_advance` — function

Phase is measured in binary turns: 2^PW units are one full revolution. Phase advances on accepted tuning words, not on idle clocks.

```
fn phase_advance<PW>(phase:uint<PW>, increment:uint<PW>) -> uint<PW>
```

Exact declaration · Contract and restrictions

13.50.2 `SinCos` — struct

```
struct SinCos<type T>
sine : T
cosine : T
```

Exact declaration · Contract and restrictions

13.50.3 `nco_sincos_table` — function

```
fn nco_sincos_table<OW,OF,D>()
```

Exact declaration · Contract and restrictions

13.50.4 `RvPhaseAccumulator` — module

```
module RvPhaseAccumulator<PW,INITIAL=0>
where PW >= 1 && INITIAL >= 0 && INITIAL < (1 << PW)
clock clk
reset rst
in input : rv<uint<PW>>
out output : rv<uint<PW>>
```

Exact declaration · Contract and restrictions

13.50.5 `RvNcoSinCos` — module

One paired synchronous ROM; holding its address holds a backpressured sample.

```
module RvNcoSinCos<PW,AW,OW,OF,INITIAL=0,D=1<<AW>
where PW >= AW && AW >= 2 && D == (1 << AW) && D <= 4096
   && OW >= 2 && OF >= 0 && OF < OW
   && INITIAL >= 0 && INITIAL < (1 << PW)
clock clk
reset rst
in input : rv<uint<PW>>
out output : rv<SinCos<fixed<OW,OF>>>
```

Exact declaration · Contract and restrictions

13.51 `std.dsp.nco_linear`

Import: `import std.dsp.nco_linear`.

Component contracts · Source

13.51.1 `NcoSegment` — struct

```
struct NcoSegment<type T>
left : SinCos<T>
right : SinCos<T>
```

Exact declaration · Contract and restrictions

13.51.2 `nco_linear_endpoint` — function

Build one entry, not a complete table nested inside each segment entry.

```
fn nco_linear_endpoint<EW,EF,D,I>()
```

Exact declaration · Contract and restrictions

13.51.3 `nco_segment_table` — function

```
fn nco_segment_table<EW,EF,D>()
```

Exact declaration · Contract and restrictions

13.51.4 `nco_linear_component` — function

Caller supplies the checked full-product/sum widths from the module below. Raw shifts align binary points exactly; only the final conversion rounds.

```
fn nco_linear_component<BW,FW,EW,EF,OW,OF,MW,RW>(
    left:fixed<EW,EF>, right:fixed<EW,EF>, fraction:uint<BW>) -> fixed<OW,OF>
```

Exact declaration · Contract and restrictions

13.51.5 `RvNcoSinCosLinear` — module

Two-edge global-stall pipeline. The paired endpoint ROM is synchronous.

```
module RvNcoSinCosLinear<PW,AW,OW,OF,INITIAL=0,D=1<<AW,FW=PW-AW,
                         BW=index_width(1<<FW),EF=OF+4,EW=EF+2,MW=EW+BW+2,RW=MW+1>
where PW >= AW && AW >= 2 && D == (1 << AW) && D <= 4096
   && OW >= 2 && OF >= 0 && OF < OW
   && INITIAL >= 0 && INITIAL < (1 << PW) && FW == PW-AW
   && BW == index_width(1<<FW) && EF == OF+4 && EW == EF+2
   && MW == EW+BW+2 && RW == MW+1
clock clk
reset rst
in input : rv<uint<PW>>
out output : rv<SinCos<fixed<OW,OF>>>
```

Exact declaration · Contract and restrictions

13.52 `std.dsp.polyphase`

Import: `import std.dsp.polyphase`.

Component contracts · Source

13.52.1 `RvPolyphaseFirDecimatorExact` — module

Parallel decimation bank: phase p owns coefficients p, p+R, p+2R, … . Full precision is retained across the phase dots and final reduction.

```
module RvPolyphaseFirDecimatorExact<SW,SF,CW,CF,R,K,N=R*K,H=N-1,
                                    PW=index_width(R),coefficients:vec<N,fixed<CW,CF>>>
where R >= 1 && K >= 1 && N == R*K && H == N-1 && PW == index_width(R)
clock clk
reset rst
in input : rv<fixed<SW,SF>>
out output : rv<fixed<SW+CW+ceil_log2(N),SF+CF>>
```

Exact declaration · Contract and restrictions

13.52.2 `RvPolyphaseFirDecimatorNearestEvenSaturate` — module

```
module RvPolyphaseFirDecimatorNearestEvenSaturate<SW,SF,CW,CF,R,K,OW,OF,
                                                  N=R*K,coefficients:vec<N,fixed<CW,CF>>>
where R >= 1 && K >= 1 && N == R*K
clock clk
reset rst
in input : rv<fixed<SW,SF>>
out output : rv<fixed<OW,OF>>
```

Exact declaration · Contract and restrictions

13.52.3 `fir_interpolation_batch` — function

Every accepted sample produces one stored batch of R exact phase sums. Coefficient p+R*k multiplies the kth accepted sample in that phase.

```
fn fir_interpolation_batch<SW,SF,CW,CF,R,K,N=R*K>(
    samples:vec<K,fixed<SW,SF>>, coefficients:vec<N,fixed<CW,CF>>)
```

Exact declaration · Contract and restrictions

13.52.4 `RvPolyphaseFirInterpolatorExact` — module

```
module RvPolyphaseFirInterpolatorExact<SW,SF,CW,CF,R,K,N=R*K,H=K-1,
                                       PW=index_width(R),AW=SW+CW+ceil_log2(K),
                                       coefficients:vec<N,fixed<CW,CF>>>
where R >= 1 && K >= 1 && N == R*K && H == K-1
   && PW == index_width(R) && AW == SW+CW+ceil_log2(K)
clock clk
reset rst
in input : rv<fixed<SW,SF>>
out output : rv<fixed<AW,SF+CF>>
```

Exact declaration · Contract and restrictions

13.52.5 `RvPolyphaseFirInterpolatorNearestEvenSaturate` — module

```
module RvPolyphaseFirInterpolatorNearestEvenSaturate<SW,SF,CW,CF,R,K,OW,OF,
                                                     N=R*K,coefficients:vec<N,fixed<CW,CF>>>
where R >= 1 && K >= 1 && N == R*K
clock clk
reset rst
in input : rv<fixed<SW,SF>>
out output : rv<fixed<OW,OF>>
```

Exact declaration · Contract and restrictions

13.53 `std.math.complex`

Import: `import std.math.complex`.

Component contracts · Source

13.53.1 `Complex` — struct

Ordinary generic ZLang source; the compiler and backends have no Complex special case.

```
struct Complex<type T>
re : T
im : T
```

Exact declaration · Contract and restrictions

13.53.2 `Butterfly` — struct

```
struct Butterfly<type S, type D>
sum : S
difference : D
```

Exact declaration · Contract and restrictions

13.53.3 `+` — operator

```
operator +<type A, type B>(a : Complex<A>, b : Complex<B>)
```

Exact declaration · Contract and restrictions

13.53.4 `-` — operator

```
operator -<type A, type B>(a : Complex<A>, b : Complex<B>)
```

Exact declaration · Contract and restrictions

13.53.5 `*` — operator

```
operator *<type A, type B>(a : Complex<A>, b : Complex<B>)
```

Exact declaration · Contract and restrictions

13.53.6 `-` — operator

```
operator -<type T>(value : Complex<T>)
```

Exact declaration · Contract and restrictions

13.53.7 `complex_sum` — function

Discoverable stdlib spelling for the language-level exact nominal sum. `sum(values)` and `complex_sum(values)` normalize to the same typed Reduce with one frozen, overload-resolved balanced tree. No quantization occurs.

```
fn complex_sum<type T,N>(values : vec<N,Complex<T>>)
```

Exact declaration · Contract and restrictions

13.53.8 `complex_quantize_nearest_even_saturate` — function

```
fn complex_quantize_nearest_even_saturate<type T, type A>(
    value : Complex<A>
) -> Complex<T>
```

Exact declaration · Contract and restrictions

13.53.9 `butterfly_quantized` — function

Profile-neutral quantized butterfly. T is always explicit: the caller, rather than this library, owns the numerical quantization boundary.

```
fn butterfly_quantized<type T, type A, type B, type W>(
    a : Complex<A>, b : Complex<B>, twiddle : Complex<W>
)
```

Exact declaration · Contract and restrictions

13.54 `std.math.complex_fixed_18_16`

Import: `import std.math.complex_fixed_18_16`.

Component contracts · Source

13.54.1 `ComplexMul18_16` — module

```
module ComplexMul18_16
in a : Complex<fixed<18,16>>
in b : Complex<fixed<18,16>>
out result : Complex<fixed<18,16>>
```

Exact declaration · Contract and restrictions

13.55 `std.math.fixed`

Import: `import std.math.fixed`.

Component contracts · Source

13.55.1 `fixed_min` — function

These pure helpers retain the exact input type. They therefore work for any comparable fixed/u-fixed specialization without choosing a numerical profile.

```
fn fixed_min<type T>(a : T, b : T) -> T
```

Exact declaration · Contract and restrictions

13.55.2 `fixed_max` — function

```
fn fixed_max<type T>(a : T, b : T) -> T
```

Exact declaration · Contract and restrictions

13.55.3 `fixed_clamp` — function

```
fn fixed_clamp<type T>(value : T, low : T, high : T) -> T
```

Exact declaration · Contract and restrictions

13.55.4 `fixed_abs` — function

Abs follows the language's exact unary-negation width rule. Its inferred result is widened when required to represent the magnitude of the most- negative signed input; this helper never introduces an implicit narrowing or saturation boundary.

```
fn fixed_abs<type T>(value : T)
```

Exact declaration · Contract and restrictions

13.55.5 `fixed_mac` — function

MAC retains the language's exact product and addition widths. Its inferred result type is therefore visible to optimization and may be quantized once by the caller at the chosen numerical boundary.

```
fn fixed_mac<type A, type B, type C>(a : A, b : B, accumulator : C)
```

Exact declaration · Contract and restrictions

13.55.6 `fixed_quantize_floor_wrap` — function

Explicit conversion helpers keep rounding and overflow visible at the call site through the function name and retain FixedConvert in typed IR.

```
fn fixed_quantize_floor_wrap<type T, type A>(value : A) -> T
```

Exact declaration · Contract and restrictions

13.55.7 `fixed_quantize_toward_zero_wrap` — function

```
fn fixed_quantize_toward_zero_wrap<type T, type A>(value : A) -> T
```

Exact declaration · Contract and restrictions

13.55.8 `fixed_quantize_away_zero_saturate` — function

```
fn fixed_quantize_away_zero_saturate<type T, type A>(value : A) -> T
```

Exact declaration · Contract and restrictions

13.55.9 `fixed_quantize_nearest_even_saturate` — function

```
fn fixed_quantize_nearest_even_saturate<type T, type A>(value : A) -> T
```

Exact declaration · Contract and restrictions

13.55.10 `FixedAbs` — module

```
module FixedAbs<W=16,F=8>
in x : fixed<W,F>
out y : fixed<W,F>
```

Exact declaration · Contract and restrictions

13.55.11 `FixedClamp` — module

```
module FixedClamp<W=16,F=8>
in x : fixed<W,F>
in low : fixed<W,F>
in high : fixed<W,F>
out y : fixed<W,F>
```

Exact declaration · Contract and restrictions

13.55.12 `FixedMinMax` — module

```
module FixedMinMax<W=16,F=8>
in a : fixed<W,F>
in b : fixed<W,F>
out minimum : fixed<W,F>
out maximum : fixed<W,F>
```

Exact declaration · Contract and restrictions

13.55.13 `FixedMAC` — module

```
module FixedMAC<W=16,F=8>
in a : fixed<W,F>
in b : fixed<W,F>
in accumulator : fixed<W*2,F*2>
out result : fixed<W*2+1,F*2>
```

Exact declaration · Contract and restrictions

13.55.14 `FixedSaturatingAdd` — module

```
module FixedSaturatingAdd<W=16,F=8>
in a : fixed_sat<W,F>
in b : fixed_sat<W,F>
out result : fixed_sat<W,F>
```

Exact declaration · Contract and restrictions

13.55.15 `UFixedSaturatingAdd` — module

```
module UFixedSaturatingAdd<W=16,F=8>
in a : ufixed_sat<W,F>
in b : ufixed_sat<W,F>
out result : ufixed_sat<W,F>
```

Exact declaration · Contract and restrictions

13.56 `std.math.fp`

Import: `import std.math.fp`.

Component contracts · Source

13.56.1 `fp_compute` — function

Complete non-iterative operation selection; div/sqrt finite datapaths belong to RvBinaryFloat below. This function also owns their exceptional results.

```
fn fp_compute<E,F,W=1+E+F>(request:FpRequest<W>) -> FpResult
```

Exact declaration · Contract and restrictions

13.56.2 `BinaryFloatFast` — module

Keep combinational arithmetic outside the sequential control process. This is an ordinary reusable source boundary, not a vendor/black-box primitive.

```
module BinaryFloatFast<E=8,F=23,W=1+E+F>
in request:FpRequest<W>
out result:FpResult
```

Exact declaration · Contract and restrictions

13.56.3 `RvBinaryFloat` — module

One in-flight request, one buffered result. Simple operations take one edge; finite division takes F+6 edges and sqrt F+5 edges. No overlap, no hidden CSR. Output stalls preserve all bits/flags. Cancellation discards work/result. Only the two IEEE interchange profiles are admitted by this public wrapper.

```
module RvBinaryFloat<E=8,F=23,W=1+E+F,X=E+5,D=F+5,S=F+4,R=F*2+8,H=F+6>
where ((E == 8 && F == 23) || (E == 11 && F == 52)) && W == 1+E+F
   && X == E+5 && D == F+5 && S == F+4 && R == F*2+8 && H == F+6
clock clk
reset rst
in input:rv<FpRequest<W>>
out output:rv<FpResult>
in cancel:bit
```

Exact declaration · Contract and restrictions

13.57 `std.math.fp_arithmetic`

Import: `import std.math.fp_arithmetic`.

Component contracts · Source

13.57.1 `fp_add` — function

```
fn fp_add<E,F,W=1+E+F,M=F+6>(a:bits<W>,b:bits<W>,subtract:bit,rounding:u3) -> FpResult
```

Exact declaration · Contract and restrictions

13.57.2 `fp_multiply` — function

```
fn fp_multiply<E,F,W=1+E+F,X=E+5>(a:bits<W>,b:bits<W>,rounding:u3) -> FpResult
```

Exact declaration · Contract and restrictions

13.57.3 `fp_fused` — function

```
fn fp_fused<E,F,W=1+E+F,X=E+5,M=2*F+6>(a:bits<W>,b:bits<W>,c:bits<W>,negative_product:bit,
    negative_addend:bit,rounding:u3) -> FpResult
```

Exact declaration · Contract and restrictions

13.57.4 `fp_compare` — function

```
fn fp_compare<E,F,W=1+E+F>(a:bits<W>,b:bits<W>,operation:u5) -> FpResult
```

Exact declaration · Contract and restrictions

13.57.5 `fp_minmax` — function

```
fn fp_minmax<E,F,W=1+E+F>(a:bits<W>,b:bits<W>,maximum:bit) -> FpResult
```

Exact declaration · Contract and restrictions

13.58 `std.math.fp_convert`

Import: `import std.math.fp_convert`.

Component contracts · Source

13.58.1 `fp_from_integer` — function

```
fn fp_from_integer<E,F,X=E+5>(value:bits<64>,word:bit,unsigned_integer:bit,rounding:u3) ->
     FpResult
```

Exact declaration · Contract and restrictions

13.58.2 `fp_to_integer` — function

```
fn fp_to_integer<E,F,W=1+E+F,X=E+5>(value:bits<W>,word:bit,unsigned_integer:bit,rounding:
    u3) -> FpResult
```

Exact declaration · Contract and restrictions

13.58.3 `fp_convert` — function

```
fn fp_convert<SE,SF,DE,DF,SW=1+SE+SF,X=DE+5,Z=SE+DE+5>(value:bits<SW>,rounding:u3) ->
    FpResult
```

Exact declaration · Contract and restrictions

13.59 `std.math.fp_format`

Import: `import std.math.fp_format`.

Component contracts · Source

13.59.1 `FpDecoded` — struct

CPU-independent IEEE binary interchange encodings. No language float type, host floating-point operation, architectural CSR or flush-to-zero behavior.

```
struct FpDecoded<E,F>
sign:bit
exponent:sint<E+5>
significand:bits<F+1>
is_zero:bit
subnormal:bit
infinity:bit
nan:bit
signaling:bit
```

Exact declaration · Contract and restrictions

13.59.2 `FpResult` — struct

```
struct FpResult
value:bits<64>
flags:bits<5>
illegal:bit
```

Exact declaration · Contract and restrictions

13.59.3 `FpRequest` — struct

```
struct FpRequest<W>
a:bits<W>
b:bits<W>
c:bits<W>
integer:bits<64>
operation:u5
rounding:u3
word:bit
unsigned_integer:bit
```

Exact declaration · Contract and restrictions

13.59.4 `fp_bit_length` — function

```
fn fp_bit_length<M>(value:bits<M>) -> u8
```

Exact declaration · Contract and restrictions

13.59.5 `fp_shift_jam` — function

```
fn fp_shift_jam<M>(value:bits<M>,distance:u16) -> bits<M>
```

Exact declaration · Contract and restrictions

13.59.6 `fp_decode` — function

```
fn fp_decode<E,F,W=1+E+F,X=E+5>(value:bits<W>) -> FpDecoded<E,F>
```

Exact declaration · Contract and restrictions

13.59.7 `fp_nan` — function

```
fn fp_nan<E,F,W=1+E+F>() -> bits<W>
```

Exact declaration · Contract and restrictions

13.59.8 `fp_infinity` — function

```
fn fp_infinity<E,F,W=1+E+F>(sign:bit) -> bits<W>
```

Exact declaration · Contract and restrictions

13.59.9 `fp_classify` — function

```
fn fp_classify<E,F,W=1+E+F>(value:bits<W>) -> bits<10>
```

Exact declaration · Contract and restrictions

13.59.10 `fp_round_increment` — function

```
fn fp_round_increment(sign:bit,rounding:u3,low:bits<3>,odd:bit) -> bit
```

Exact declaration · Contract and restrictions

13.60 `std.math.fp_round`

Import: `import std.math.fp_round`.

Component contracts · Source

13.60.1 `fp_round_pack` — function

The sole finite rounding/packing owner. Exact input is magnitude*2^scale; bit zero may carry a right-shift sticky bit. Tininess is after rounding.

```
fn fp_round_pack<E,F,M,W=1+E+F,X=E+5,P=F+4,Q=F+2,L=M+F+4>(sign:bit,magnitude:bits<M>,scale
    :sint<E+5>,rounding:u3) -> FpResult
```

Exact declaration · Contract and restrictions

13.60.2 `fp_sum` — function

Aligned signed magnitude addition. K is the exact binary point of both input magnitudes relative to their leading unbiased exponent.

```
fn fp_sum<E,F,M,K,X=E+5>(a:bits<M>,b:bits<M>,ea:sint<E+5>,eb:sint<E+5>,
    sa:bit,sb:bit,rounding:u3) -> FpResult
```

Exact declaration · Contract and restrictions

13.61 `std.peripheral.capture`

Import: `import std.peripheral.capture`.

Component contracts · Source

13.61.1 `RisingCapture32` — module

Rising edges only. One slot; a full slot drops the new edge. Reset clears loss.

```
module RisingCapture32
clock clk
reset rst
in reset_active,enable,level : bit
in timestamp : u32
out output : rv<u32>
out overflow : bit
```

Exact declaration · Contract and restrictions

13.61.2 `Capture32BasicRegisters` — module

Register declarations own read data, reset storage and command events.

```
module Capture32BasicRegisters
clock clk
reset rst
in status,captured:u32
out pop:bits<32>
```

Exact declaration · Contract and restrictions

13.61.3 `RisingCapture32RegBus32` — module

```
module RisingCapture32RegBus32
clock clk
reset rst
in reset_active : bit
in level:bit
in timestamp:u32
interface regbus : RegBus<32,32>.responder
```

Exact declaration · Contract and restrictions

13.62 `std.peripheral.gpio`

Import: `import std.peripheral.gpio`.

Component contracts · Source

13.62.1 `Gpio8` — module

Eight combinational GPIO bits. Pad sampling and reset ownership are explicit.

```
module Gpio8
in reset_active : bit
in pins_in,data,oe : bits<8>
out sampled,pins_out,pins_oe : bits<8>
```

Exact declaration · Contract and restrictions

13.62.2 `Gpio8BasicRegisters` — module

Register declarations own read data, reset storage and command events.

```
module Gpio8BasicRegisters
clock clk
reset rst
in sampled:u32
```

Exact declaration · Contract and restrictions

13.62.3 `Gpio8RegBus32` — module

```
module Gpio8RegBus32
clock clk
reset rst
in reset_active : bit
in pins_in:bits<8>
out pins_out,pins_oe:bits<8>
interface regbus : RegBus<32,32>.responder
```

Exact declaration · Contract and restrictions

13.63 `std.peripheral.i2c`

Import: `import std.peripheral.i2c`.

Component contracts · Source

13.63.1 `I2cWriteRequest` — struct

```
struct I2cWriteRequest
address:u7
data:bits<8>
```

Exact declaration · Contract and restrictions

13.63.2 `I2cWriteByte` — module

Atomic START + 7-bit write address + one byte + STOP. No read/repeated START. Open-drain low-drive enables; pin sampling and pull-ups are caller-owned. Status: 0 success, 1 address NACK, 2 data NACK, 3 stretch timeout, 4 bus fault.

```
module I2cWriteByte<HALF_PERIOD=4>
where HALF_PERIOD >= 1 && HALF_PERIOD <= 65535
clock clk
reset rst
in reset_active,scl_sampled,sda_sampled : bit
in input : rv<I2cWriteRequest>
out output : rv<u3>
out scl_low,sda_low,busy : bit
```

Exact declaration · Contract and restrictions

13.63.3 `I2cWriteBasicRegisters` — module

Register declarations own read data, reset storage and command events.

```
module I2cWriteBasicRegisters<HALF_PERIOD=4>
clock clk
reset rst
in status,result:u32
in cfg_half_period:u32
out request_word,pop:bits<32>
```

Exact declaration · Contract and restrictions

13.63.4 `I2cWriteByteRegBus32` — module

```
module I2cWriteByteRegBus32<HALF_PERIOD=4>
clock clk
reset rst
in reset_active : bit
in scl_sampled,sda_sampled:bit
out scl_low,sda_low:bit
interface regbus : RegBus<32,32>.responder
```

Exact declaration · Contract and restrictions

13.64 `std.peripheral.pwm`

Import: `import std.peripheral.pwm`.

Component contracts · Source

13.64.1 `Pwm16` — module

One edge-aligned channel; period and duty are immutable specialization values.

```
module Pwm16<PERIOD=256,DUTY=128>
where PERIOD >= 1 && PERIOD <= 65535 && DUTY >= 0 && DUTY <= PERIOD
clock clk
reset rst
in reset_active,enable : bit
out pin : bit
out phase : u16
```

Exact declaration · Contract and restrictions

13.64.2 `Pwm16BasicRegisters` — module

Register declarations own read data, reset storage and command events.

```
module Pwm16BasicRegisters<PERIOD=256,DUTY=128>
clock clk
reset rst
in phase:u32
in cfg_duty:u32
in cfg_period:u32
```

Exact declaration · Contract and restrictions

13.64.3 `Pwm16RegBus32` — module

```
module Pwm16RegBus32<PERIOD=256,DUTY=128>
clock clk
reset rst
in reset_active : bit
out pin:bit
interface regbus : RegBus<32,32>.responder
```

Exact declaration · Contract and restrictions

13.65 `std.peripheral.regbus`

Import: `import std.peripheral.regbus`.

Component contracts · Source

13.65.1 `BasicPeripheralTarget32` — module

One request, one captured response. Effects occur only at request acceptance. A stalled response never repeats a command or changes its read snapshot.

```
module BasicPeripheralTarget32
clock clk
reset rst
in reset_active : bit
in request : rv<RegRequest<32,32>>
out response : rv<RegResponse<32>>
in read_value : u32
in error : bit
out accept : bit
```

Exact declaration · Contract and restrictions

13.65.2 `basic_peripheral_merge` — function

The CSR access ABI has no WSTRB; merge bytes at this ordinary stdlib boundary.

```
fn basic_peripheral_merge(old:bits<32>,data:bits<32>,strobes:bits<4>) -> bits<32>
```

Exact declaration · Contract and restrictions

13.66 `std.peripheral.spi`

Import: `import std.peripheral.spi`.

Component contracts · Source

13.66.1 `SpiMasterMode0` — module

One CS, eight bits, full duplex, MSB first, mode 0 only. No runtime config.

```
module SpiMasterMode0<HALF_PERIOD=2>
where HALF_PERIOD >= 1 && HALF_PERIOD <= 65535
clock clk
reset rst
in reset_active,miso_sampled : bit
in input : rv<bits<8>>
out output : rv<bits<8>>
out sck,mosi,cs_n,busy : bit
```

Exact declaration · Contract and restrictions

13.66.2 `SpiMode0BasicRegisters` — module

Register declarations own read data, reset storage and command events.

```
module SpiMode0BasicRegisters<HALF_PERIOD=2>
clock clk
reset rst
in status,received:u32
in cfg_half_period:u32
out transmit,pop:bits<32>
```

Exact declaration · Contract and restrictions

13.66.3 `SpiMode0RegBus32` — module

```
module SpiMode0RegBus32<HALF_PERIOD=2>
clock clk
reset rst
in reset_active : bit
in miso_sampled:bit
out sck,mosi,cs_n:bit
interface regbus : RegBus<32,32>.responder
```

Exact declaration · Contract and restrictions

13.67 `std.peripheral.timer`

Import: `import std.peripheral.timer`.

Component contracts · Source

13.67.1 `PeriodicTimer32` — module

Fixed 32-bit periodic counter. Disable pauses; reset restarts at zero.

```
module PeriodicTimer32<PERIOD=1000>
where PERIOD >= 1 && PERIOD <= 0xffffffff
clock clk
reset rst
in reset_active,enable : bit
out count : u32
out tick : bit
```

Exact declaration · Contract and restrictions

13.67.2 `Timer32BasicRegisters` — module

Register declarations own read data, reset storage and command events.

```
module Timer32BasicRegisters<PERIOD=1000>
clock clk
reset rst
in count:u32
in cfg_period:u32
```

Exact declaration · Contract and restrictions

13.67.3 `PeriodicTimer32RegBus32` — module

```
module PeriodicTimer32RegBus32<PERIOD=1000>
clock clk
reset rst
in reset_active : bit
out tick:bit
interface regbus : RegBus<32,32>.responder
```

Exact declaration · Contract and restrictions

13.68 `std.peripheral.uart`

Import: `import std.peripheral.uart`.

Component contracts · Source

13.68.1 `UartRxBeat` — struct

```
struct UartRxBeat
data:bits<8>
framing_error:bit
```

Exact declaration · Contract and restrictions

13.68.2 `UartTx8N1` — module

One 8N1 transmitter, LSB first, with an immutable sample-clock divisor.

```
module UartTx8N1<DIVISOR=16> where DIVISOR >= 4 && DIVISOR <= 65535
clock clk
reset rst
in reset_active : bit
in input : rv<bits<8>>
out tx,busy : bit
```

Exact declaration · Contract and restrictions

13.68.3 `UartRx8N1` — module

One receive slot. Framing errors are explicit; overflow is sticky until reset. rx_sampled must already belong to clk's sampling domain.

```
module UartRx8N1<DIVISOR=16> where DIVISOR >= 4 && DIVISOR <= 65535
clock clk
reset rst
in reset_active,rx_sampled : bit
out output : rv<UartRxBeat>
out busy,overrun : bit
```

Exact declaration · Contract and restrictions

13.68.4 `Uart8N1BasicRegisters` — module

Register declarations own read data, reset storage and command events.

```
module Uart8N1BasicRegisters<DIVISOR=16>
clock clk
reset rst
in status,received:u32
in cfg_divisor:u32
out transmit,pop:bits<32>
```

Exact declaration · Contract and restrictions

13.68.5 `Uart8N1RegBus32` — module

```
module Uart8N1RegBus32<DIVISOR=16>
clock clk
reset rst
in reset_active : bit
in rx_sampled:bit
out tx:bit
interface regbus : RegBus<32,32>.responder
```

Exact declaration · Contract and restrictions

13.69 `std.storage.core`

Import: `import std.storage.core`.

Component contracts · Source

13.69.1 `StorageDelay1` — module

Source-authored storage wrappers over the existing semantic primitives. `delay<N>` currently requires a literal N, so bounded common depths are separate ordinary modules rather than a misleading generic wrapper.

```
module StorageDelay1<type T>
clock clk
reset rst
in input : T
out output : T
```

Exact declaration · Contract and restrictions

13.69.2 `StorageDelay2` — module

```
module StorageDelay2<type T>
clock clk
reset rst
in input : T
out output : T
```

Exact declaration · Contract and restrictions

13.69.3 `StorageSnapshot` — module

A protocol-neutral atomic capture primitive. CSR command snapshots are ordinary typed state: connect a compiler-qualified bank event to `capture` and a structural state view to `input`; no CSR-specific scheduler exists.

```
module StorageSnapshot<type T,reset_value:fn()->T>
clock clk
reset rst
in input : T
in capture : bit
out output : T
```

Exact declaration · Contract and restrictions

13.69.4 `StorageQueue` — module

```
module StorageQueue<type T,D=2,CW=2>
clock clk
reset rst
in push : bit
in pop : bit
in data : T
out front : T
out valid : bit
out ready : bit
out count : uint<CW>
```

Exact declaration · Contract and restrictions

13.69.5 `StorageRom` — module

Immutable images are compile-time specialization arguments, never runtime ports. The concrete image is hashed into the child specialization and the existing ROM companion artifact.

```
module StorageRom<type T,N,IW,image:vec<N,T>>
where N >= 2 && is_power_of_two(N) && IW == floor_log2(N)
clock clk
reset rst
in address : uint<IW>
out data : T
```

Exact declaration · Contract and restrictions

13.69.6 `StorageGeneratedRom` — module

A statically selected zero-argument producer is specialized and evaluated during elaboration. It is not a runtime function pointer or a backend callback.

```
module StorageGeneratedRom<type T,N,IW,producer:fn()->vec<N,T>>
where N >= 2 && is_power_of_two(N) && IW == floor_log2(N)
clock clk
reset rst
in address : uint<IW>
out data : T
```

Exact declaration · Contract and restrictions

13.69.7 `StorageReorderBits` — module

A generic raw-bit reorder bank. The caller owns the permutation and occupancy policy; this wrapper supplies deterministic range-checked indexed storage with one write per cycle. A typed caller may use explicit bitcast at this representation boundary.

```
module StorageReorderBits<W=8,N=8>
where N >= 2 && is_power_of_two(N)
clock clk
reset rst
in write : bit
in write_index : uint<floor_log2(N)>
in write_data : bits<W>
in read_index : uint<floor_log2(N)>
out read_data : bits<W>
```

Exact declaration · Contract and restrictions

13.69.8 `StoragePingPongStatus` — struct

Two raw-bit banks permit one completed block to be read while the other is filled. `commit` publishes the current write bank and changes banks; `retire` releases the current read bank. The explicit valid bits prevent a writer from overwriting an unconsumed block.

```
struct StoragePingPongStatus<W=8>
write_ready : bit
commit_ready : bit
read_valid : bit
read_data : bits<W>
```

Exact declaration · Contract and restrictions

13.69.9 `StoragePingPongBits` — module

```
module StoragePingPongBits<W=8,N=8>
where N >= 2 && is_power_of_two(N)
clock clk
reset rst
in write : bit
in write_index : uint<floor_log2(N)>
in write_data : bits<W>
in commit : bit
in retire : bit
in read_index : uint<floor_log2(N)>
out status : StoragePingPongStatus<W>
```

Exact declaration · Contract and restrictions

13.69.10 `StorageAsyncFifo` — module

Ordinary source wrapper around the existing explicit asynchronous FIFO crossing. It introduces no second FIFO primitive and no implicit CDC.

```
module StorageAsyncFifo<type T,D=4>
clock write_clk
reset write_rst @write_clk
clock read_clk
reset read_rst @read_clk
in source : rv<T> @write_clk
out sink : rv<T> @read_clk
```

Exact declaration · Contract and restrictions

13.69.11 `StorageDualPortMemory` — module

Same-clock true dual-port logical memory. Physical selection may use a matching native resource or the bounded generic register/mux realization.

```
module StorageDualPortMemory<type T,N,AW>
where N >= 2 && is_power_of_two(N) && AW == floor_log2(N)
clock clk
reset rst
in a_read : bit
in a_write : bit
in a_address : uint<AW>
in a_write_data : T
out a_read_data : T
in b_read : bit
in b_write : bit
in b_address : uint<AW>
in b_write_data : T
out b_read_data : T
```

Exact declaration · Contract and restrictions

13.69.12 `Storage2R1WMemory` — module

A one-writer/two-reader shape is deterministically implemented as coherent replicated 1R1W memories when no exact native resource is selected.

```
module Storage2R1WMemory<type T,N,AW>
where N >= 2 && is_power_of_two(N) && AW == floor_log2(N)
clock clk
reset rst
in write : bit
in write_address : uint<AW>
in write_data : T
in read0 : bit
in read0_address : uint<AW>
out read0_data : T
in read1 : bit
in read1_address : uint<AW>
out read1_data : T
```

Exact declaration · Contract and restrictions

13.69.13 `StorageAsyncMemory1W1R` — module

The bounded independent-clock writable-memory shape: one write-only source domain and one read-only destination domain.

```
module StorageAsyncMemory1W1R<type T,N,AW>
where N >= 2 && is_power_of_two(N) && AW == floor_log2(N)
clock write_clk
reset write_rst @write_clk
clock read_clk
reset read_rst @read_clk
in write : bit @write_clk
in write_address : uint<AW> @write_clk
in write_data : T @write_clk
in read_address : uint<AW> @read_clk
out read_data : T @read_clk
```

Exact declaration · Contract and restrictions

13.70 `std.storage`

Import: `import std.storage`.

Component contracts · Source

Import-only namespace; its source imports the component declarations listed in this catalog.

13.71 `std.stream.complex_fixed`

Import: `import std.stream.complex_fixed`.

Component contracts · Source

13.71.1 `ComplexQuantizeStream37_32To18_16` — module

```
module ComplexQuantizeStream37_32To18_16
clock clk
reset rst
in input : rv<Complex<fixed<37,32>>>
out output : rv<Complex<fixed<18,16>>>
```

Exact declaration · Contract and restrictions

13.72 `std.stream.composition`

Import: `import std.stream.composition`.

Component contracts · Source

13.72.1 `Tee` — module

Combinational observation. Both outputs retain the exact input type/value; the tap has no handshake, storage, reset or clock-domain conversion.

```
module Tee<type T>
in input : T
out output : T
out tap : T
```

Exact declaration · Contract and restrictions

13.72.2 `RvTee` — module

Passive ready/valid observation. Only the functional output drives ready; the tap cannot stall traffic and does not buffer or retain accepted items.

```
module RvTee<type T>
in input : rv<T>
out output : rv<T>
out tap_payload : T
out tap_valid, tap_ready, tap_transfer : bit
```

Exact declaration · Contract and restrictions

13.72.3 `RvFork2` — module

Buffered broadcast. A register slice owns the generic payload; flags track which consumers still need this item, independently of their ready signals.

```
module RvFork2<type T>
clock clk
reset rst
in input : rv<T>
out output0 : rv<T>
out output1 : rv<T>
```

Exact declaration · Contract and restrictions

13.72.4 `StreamPair` — struct

```
struct StreamPair<type A, type B>
left : A
right : B
```

Exact declaration · Contract and restrictions

13.72.5 `RvJoin2` — module

Independent one-entry lanes zip by acceptance order, not arrival cycle.

```
module RvJoin2<type A, type B>
clock clk
reset rst
in input0 : rv<A>
in input1 : rv<B>
out output : rv<StreamPair<A,B>>
```

Exact declaration · Contract and restrictions

13.73 `std.stream.core`

Import: `import std.stream.core`.

Component contracts · Source

13.73.1 `FrameBeat` — struct

Generic ready/valid building blocks. These are ordinary ZLang modules; state, transfer, and backpressure use the language's existing FIFO model.

```
struct FrameBeat<type T, type M>
data : T
meta : M
first : bit
last : bit
```

Exact declaration · Contract and restrictions

13.73.2 `RvRegisterSlice` — module

One-entry elastic register slice. A FIFO supplies the generic payload's reset value without inventing a compiler-level default(T) operation.

```
module RvRegisterSlice<type T>
clock clk
reset rst
in input : rv<T>
out output : rv<T>
```

Exact declaration · Contract and restrictions

13.73.3 `RvSkidBuffer` — module

A two-entry elastic skid buffer. The second entry absorbs one transfer after downstream ready deasserts; ordering and payload stability are the ordinary FIFO guarantees.

```
module RvSkidBuffer<type T>
clock clk
reset rst
in input : rv<T>
out output : rv<T>
```

Exact declaration · Contract and restrictions

13.73.4 `RvFifo` — module

```
module RvFifo<type T,D=2>
clock clk
reset rst
in input : rv<T>
out output : rv<T>
```

Exact declaration · Contract and restrictions

13.73.5 `RvMux2` — module

Explicit two-way ready/valid routing for runtime-selected physical clients. These modules route transactions; they do not create dynamic instances or infer clock gating. Wider fan-in/fan-out is built as a bounded tree so each admission and backpressure edge remains visible in source.

```
module RvMux2<type T>
in select : bit
in input0 : rv<T>
in input1 : rv<T>
out output : rv<T>
```

Exact declaration · Contract and restrictions

13.73.6 `RvDemux2` — module

```
module RvDemux2<type T>
in select : bit
in input : rv<T>
out output0 : rv<T>
out output1 : rv<T>
```

Exact declaration · Contract and restrictions

13.74 `std.stream.serialization`

Import: `import std.stream.serialization`.

Component contracts · Source

13.74.1 `RvVectorSerializer` — module

Bounded vector-to-stream serialization using existing ready/valid and FIFO semantics. N must be a power of two and at least two in this first source implementation. The declaration rejects incoherent specializations before any storage or backend lowering is attempted.

```
module RvVectorSerializer<type T,N=8,IW=3>
where N >= 2 && is_power_of_two(N) && IW == floor_log2(N)
clock clk
reset rst
in input : rv<vec<N,T>>
out output : rv<T>
```

Exact declaration · Contract and restrictions

13.74.2 `RvBitCollector` — module

A bit collector is useful at raw serialization boundaries and does not need an implicit default value for an arbitrary generic T.

```
module RvBitCollector<N=8,IW=3>
where N >= 2 && is_power_of_two(N) && IW == floor_log2(N)
clock clk
reset rst
in input : rv<bit>
out output : rv<bits<N>>
```

Exact declaration · Contract and restrictions

13.75 `std.target.asic.generic`

Import: `import std.target.asic.generic`.

Component contracts · Source

13.75.1 `AsicMultiplierMacro` — class

ASIC compatibility fixtures use exactly the same vendor-neutral resource IR.

```
resource AsicMultiplierMacro { class multiplier operation multiply capability hard_macro
    true limit multiplier_a 32 limit multiplier_b 32 binding systemverilog unsupported
    physical_primitive project_multiplier }
```

Exact declaration · Contract and restrictions

13.75.2 `AsicSRAMMacro` — class

```
resource AsicSRAMMacro { class block_memory operation synchronous_ram capability
    hard_macro true capability port_modes single.simple_dual capability synchronous_read
    true binding systemverilog unsupported physical_primitive project_sram }
```

Exact declaration · Contract and restrictions

13.75.3 `AsicPLLMacro` — class

```
resource AsicPLLMacro { class clock_generator operation clock_multiply_divide capability
    hard_macro true capability outputs 4 capability lock true binding systemverilog
    unsupported physical_primitive project_pll }
```

Exact declaration · Contract and restrictions

13.75.4 `AsicStandardCellLogic` — class

```
resource AsicStandardCellLogic { class standard_cell_logic operation combinational_logic
    capability characterized_library true binding systemverilog generic_rtl }
```

Exact declaration · Contract and restrictions

13.75.5 `GenericAsicFamily` — class

```
target GenericAsicFamily {
    provides AsicMultiplierMacro
    provides AsicSRAMMacro
    provides AsicPLLMacro
    provides AsicStandardCellLogic
}
```

Exact declaration · Contract and restrictions

13.75.6 `generic_asic` — object

```
device generic_asic : GenericAsicFamily {
    part generic-asic
    inventory AsicMultiplierMacro 16
    inventory AsicSRAMMacro 8
    inventory AsicPLLMacro 2
}
```

Exact declaration · Contract and restrictions

13.75.7 `GenericAsicResourceLibrary` — module

```
module GenericAsicResourceLibrary
```

Exact declaration · Contract and restrictions

13.76 `std.target.asic.sky130`

Import: `import std.target.asic.sky130`.

Component contracts · Source

13.76.1 `Sky130HdStandardCellLogic` — class

SKY130 high-density standard-cell synthesis profile. Ordinary Direct-SV remains generic RTL and is mapped by the downstream ASIC flow against the sky130_fd_sc_hd Liberty/LEF views. Only cells documented by that library are named here. The standard-cell library does not provide a hard multiplier, SRAM or PLL, so those resources must come from an explicit project macro library rather than being invented by this target.

```
resource Sky130HdStandardCellLogic {
    class standard_cell_logic
    operation combinational_logic
    capability characterized_library true
    capability library sky130_fd_sc_hd
    capability implementation liberty_mapped
    binding systemverilog generic_rtl
}
```

Exact declaration · Contract and restrictions

13.76.2 `Sky130HdFlipFlop` — class

```
resource Sky130HdFlipFlop {
    port clk input unsigned 1
    port d input unsigned 1
    port q output unsigned 1
    class register
    operation register
    capability library sky130_fd_sc_hd
    capability cell sky130_fd_sc_hd__dfxtp_1
    capability edge rising
    capability reset none
    pipeline_site storage location output latency 1 ii 1 local true delay_ps 0
    pipeline_config registered latency 1 ii 1 enable storage
    binding systemverilog generic_rtl
    physical_primitive sky130_fd_sc_hd__dfxtp_1
    physical_site storage Q
}
```

Exact declaration · Contract and restrictions

13.76.3 `Sky130HdResetFlipFlop` — class

```
resource Sky130HdResetFlipFlop {
    port clk input unsigned 1
    port reset_b input unsigned 1
    port d input unsigned 1
    port q output unsigned 1
    class register
    operation register
    capability library sky130_fd_sc_hd
    capability cell sky130_fd_sc_hd__dfrtp_1
    capability edge rising
    capability reset asynchronous_active_low
    pipeline_site storage location output latency 1 ii 1 local true delay_ps 0
    pipeline_config registered latency 1 ii 1 enable storage
    binding systemverilog generic_rtl
    physical_primitive sky130_fd_sc_hd__dfrtp_1
    physical_site storage Q
}
```

Exact declaration · Contract and restrictions

13.76.4 `Sky130HdClockGate` — class

```
resource Sky130HdClockGate {
    class clock_control
    operation clock_enable
    capability library sky130_fd_sc_hd
    capability cell sky130_fd_sc_hd__dlclkp_1
    capability glitch_free true
    binding systemverilog unsupported
    physical_primitive sky130_fd_sc_hd__dlclkp_1
}
```

Exact declaration · Contract and restrictions

13.76.5 `Sky130FdScHd` — class

```
target Sky130FdScHd {
    provides Sky130HdStandardCellLogic
    provides Sky130HdFlipFlop
    provides Sky130HdResetFlipFlop
    provides Sky130HdClockGate
}
```

Exact declaration · Contract and restrictions

13.76.6 `sky130_fd_sc_hd` — object

A standard-cell target has no fixed cell inventory: die area and project constraints bound utilization in synthesis/place-and-route.

```
device sky130_fd_sc_hd : Sky130FdScHd {
    part sky130_fd_sc_hd
}
```

Exact declaration · Contract and restrictions

13.76.7 `Sky130FdScHdTargetLibrary` — module

```
module Sky130FdScHdTargetLibrary
```

Exact declaration · Contract and restrictions

13.77 `std.target.generic`

Import: `import std.target.generic`.

Component contracts · Source

13.77.1 `GenericLUT` — class

Technology-independent resources. Ordinary RTL remains the implementation.

```
resource GenericLUT { class logic operation combinational_logic capability implementation
    rtl limit inputs 6 binding systemverilog generic_rtl }
```

Exact declaration · Contract and restrictions

13.77.2 `GenericRegister` — class

```
resource GenericRegister {
    port d input unsigned 1
    port q output unsigned 1
    class register
    operation register
    capability reset synchronous
    pipeline_site storage location output latency 1 ii 1 local true delay_ps 0
    pipeline_config registered latency 1 ii 1 enable storage
    binding systemverilog generic_rtl
}
```

Exact declaration · Contract and restrictions

13.77.3 `GenericMultiplier` — class

```
resource GenericMultiplier { class multiplier operation multiply capability signed true
    capability unsigned true binding systemverilog generic_rtl }
```

Exact declaration · Contract and restrictions

13.77.4 `GenericAdder` — class

```
resource GenericAdder { class adder operation add capability signed true capability
    unsigned true binding systemverilog generic_rtl }
```

Exact declaration · Contract and restrictions

13.77.5 `GenericMAC` — class

```
resource GenericMAC {
    class dsp_mac
    operation multiply_accumulate
    capability signed true
    capability unsigned true
    pipeline_site multiply location multiply latency 1 ii 1 local false delay_ps 0
    pipeline_site accumulate_output location accumulate_output latency 1 ii 1 local false
    delay_ps 0
    pipeline_config unregistered latency 0 ii 1
    pipeline_config multiply_registered latency 1 ii 1 enable multiply
    pipeline_config multiply_output_registered latency 2 ii 1 enable multiply enable
    accumulate_output
    binding systemverilog generic_rtl
}
```

Exact declaration · Contract and restrictions

13.77.6 `GenericFIFOStorage` — class

```
resource GenericFIFOStorage { class fifo_storage operation fifo_storage capability
    implementation registers_or_memory capability simultaneous_push_pop true binding
    systemverilog generic_rtl }
```

Exact declaration · Contract and restrictions

13.77.7 `GenericSyncRAM` — class

```
resource GenericSyncRAM { class block_memory operation synchronous_ram capability
    port_modes single.simple_dual capability synchronous_read true capability
    output_register optional capability initialization true capability read_during_write
    read_first.write_first binding systemverilog generic_rtl }
```

Exact declaration · Contract and restrictions

13.77.8 `GenericDualPortRAM` — class

```
resource GenericDualPortRAM { class block_memory operation dual_port_ram capability
    port_modes simple_dual.true_dual capability synchronous_read true binding systemverilog
     generic_rtl }
```

Exact declaration · Contract and restrictions

13.77.9 `GenericROM` — class

```
resource GenericROM { class block_memory operation synchronous_rom capability
    synchronous_read true capability initialization true binding systemverilog generic_rtl
    }
```

Exact declaration · Contract and restrictions

13.77.10 `GenericClockSource` — class

```
resource GenericClockSource { class clock_source operation clock_source capability outputs
     1 binding systemverilog generic_rtl }
```

Exact declaration · Contract and restrictions

13.77.11 `GenericPLL` — class

```
resource GenericPLL { class clock_generator operation clock_multiply_divide capability
    phase configurable capability lock true binding systemverilog unsupported }
```

Exact declaration · Contract and restrictions

13.77.12 `GenericClockEnable` — class

```
resource GenericClockEnable { class clock_control operation clock_enable capability
    glitch_free_required true binding systemverilog generic_rtl }
```

Exact declaration · Contract and restrictions

13.77.13 `GenericCarry` — class

```
resource GenericCarry { class arithmetic_chain operation carry dedicated carry carry_out
    -> carry_in width 1 relation logical_chain fallback true binding systemverilog
    generic_rtl }
```

Exact declaration · Contract and restrictions

13.77.14 `GenericFamily` — class

```
target GenericFamily {
    provides GenericLUT
    provides GenericRegister
    provides GenericMultiplier
    provides GenericAdder
    provides GenericMAC
    provides GenericFIFOStorage
    provides GenericSyncRAM
    provides GenericDualPortRAM
    provides GenericROM
    provides GenericClockSource
    provides GenericPLL
    provides GenericClockEnable
    provides GenericCarry
}
```

Exact declaration · Contract and restrictions

13.77.15 `generic` — object

```
device generic : GenericFamily { part generic }
```

Exact declaration · Contract and restrictions

13.77.16 `GenericTargetLibrary` — module

```
module GenericTargetLibrary
```

Exact declaration · Contract and restrictions

13.78 `std.target.intel.cyclone_v`

Import: `import std.target.intel.cyclone_v`.

Component contracts · Source

13.78.1 `CycloneVVariableDSP` — class

Intel Cyclone V variable-precision DSP, M10K, ALM and clock resources.

```
resource CycloneVVariableDSP {
    class dsp_mac
    operation multiply_accumulate
    capability signed true
    capability preadder true
    capability accumulator_width 64
    capability modes 3x9x9.2x18x18.1x27x27
    capability cascade true
    limit multiplier_a 27
    limit multiplier_b 27
    limit accumulator 64
    pipeline_site input_preadd location input_preadd latency 1 ii 1 local true delay_ps 0
    pipeline_site multiply location multiply latency 1 ii 1 local true delay_ps 0
    pipeline_site accumulate_output location accumulate_output latency 1 ii 1 local true
    delay_ps 0
    pipeline_config unregistered latency 0 ii 1
    pipeline_config multiply_registered latency 1 ii 1 enable multiply setting
    multiplier_register 1
    pipeline_config fully_pipelined latency 3 ii 1 enable input_preadd enable multiply
    enable accumulate_output setting input_register 1 setting multiplier_register 1 setting
     output_register 1
    dedicated chain chain_out -> chain_in width 64 relation adjacent_dsp
    binding systemverilog unsupported
    physical_primitive cyclonev_mac
    physical_site input_preadd input_register
    physical_site multiply multiplier_register
    physical_site accumulate_output output_register
    physical_edge chain chainout chainin
}
```

Exact declaration · Contract and restrictions

13.78.2 `CycloneVM10K` — class

```
resource CycloneVM10K {
    class block_memory
    operation synchronous_ram
    capability capacity_bits 10240
    capability port_modes single.simple_dual.true_dual
    capability widths 1.2.4.5.8.10.16.20.32.40
    capability synchronous_read true
    capability output_register optional
    capability byte_write_enable true
    capability initialization mif.hex
    pipeline_site output location memory_output latency 1 ii 1 local true delay_ps 0
    pipeline_config core_registered latency 0 ii 1
    pipeline_config output_registered latency 1 ii 1 enable output setting output_register
     1
    binding systemverilog unsupported
    physical_primitive altsyncram
    physical_site output outdata_reg
}
```

Exact declaration · Contract and restrictions

13.78.3 `CycloneVALM` — class

```
resource CycloneVALM { class logic operation combinational_logic capability inputs 8
    capability arithmetic_chain true capability mlab_bits 640 binding systemverilog
    generic_rtl }
```

Exact declaration · Contract and restrictions

13.78.4 `CycloneVFF` — class

```
resource CycloneVFF { class register operation register capability packed_with_alm true
    binding systemverilog generic_rtl }
```

Exact declaration · Contract and restrictions

13.78.5 `CycloneVFPLL` — class

```
resource CycloneVFPLL { class clock_generator operation clock_multiply_divide capability
    outputs 9 capability phase configurable capability lock true binding systemverilog
    unsupported physical_primitive altera_pll }
```

Exact declaration · Contract and restrictions

13.78.6 `IntelCycloneV` — class

```
target IntelCycloneV {
    provides CycloneVVariableDSP
    provides CycloneVM10K
    provides CycloneVALM
    provides CycloneVFF
    provides CycloneVFPLL
}
```

Exact declaration · Contract and restrictions

13.78.7 `cyclone_v_5csema5f31c6` — object

```
device cyclone_v_5csema5f31c6 : IntelCycloneV {
    part 5CSEMA5F31C6
    inventory CycloneVVariableDSP 87
    inventory CycloneVM10K 397
    inventory CycloneVFPLL 6
    dedicated_capacity CycloneVVariableDSP chain 86
}
```

Exact declaration · Contract and restrictions

13.78.8 `IntelCycloneVTargetLibrary` — module

```
module IntelCycloneVTargetLibrary
```

Exact declaration · Contract and restrictions

13.79 `std.target.xilinx.series7`

Import: `import std.target.xilinx.series7`.

Component contracts · Source

13.79.1 `DSP48E1` — class

AMD 7-series resources: DSP48E1 (UG479), block RAM (UG473), clocks (UG472).

```
resource DSP48E1 {
    port a input signed 25
    port d input signed 25
    port b input signed 18
    port c input signed 48
    port pcin input signed 48
    port p output signed 48
    port pcout output signed 48
    class dsp_mac
    operation preadd_multiply_accumulate
    capability signed true
    capability preadder true
    capability accumulator true
    capability accumulator_modes accumulator_plus_product.accumulator_minus_product
    // The explicit binding covers both the existing symmetric FIR cascade
    // and the ordered signed-product cascade used by FFT real/imag paths.
    // The semantic mapper still validates the exact accumulator mode and
    // finite-width shape before a physical graph is offered.
    capability physical_architectures symmetric_fir_cascade.signed_product_reduction
    capability c_path true
    capability output_width 48
    limit preadder 25
    limit multiplier_a 25
    limit multiplier_b 18
    limit product 43
    limit accumulator 48
    pipeline_site input_preadd location input_preadd latency 1 ii 1 local true delay_ps 0
    pipeline_site multiply location multiply latency 1 ii 1 local true delay_ps 0
    pipeline_site accumulate_output location accumulate_output latency 1 ii 1 local true
    delay_ps 0
    pipeline_config unregistered latency 0 ii 1
    pipeline_config multiply_registered latency 1 ii 1 enable multiply setting mreg 1
    pipeline_config multiply_output_registered latency 2 ii 1 enable multiply enable
    accumulate_output setting mreg 1 setting terminal_preg 1
    pipeline_config fully_pipelined latency 3 ii 1 enable input_preadd enable multiply
    enable accumulate_output setting areg 1 setting breg 1 setting dreg 1 setting mreg 1
    setting terminal_preg 1
    dedicated pcascade pcout -> pcin width 48 relation adjacent_same_column
    binding systemverilog dsp48e1_explicit
    physical_primitive DSP48E1
    physical_site input_preadd AREG
    physical_site input_preadd BREG
    physical_site input_preadd DREG
    physical_site multiply MREG
    physical_site accumulate_output PREG
    physical_edge pcascade PCOUT PCIN
}
```

Exact declaration · Contract and restrictions

13.79.2 `RAMB18E1` — class

```
resource RAMB18E1 {
    class block_memory
    operation synchronous_ram
    capability capacity_bits 18432
    capability port_modes single.simple_dual.true_dual
    capability widths 1.2.4.9.18.36
    capability synchronous_read true
    capability output_register optional
    capability byte_write_enable true
    capability initialization true
    capability read_during_write read_first.write_first.no_change
    pipeline_site output location memory_output latency 1 ii 1 local true delay_ps 0
    pipeline_config core_registered latency 0 ii 1
    pipeline_config output_registered latency 1 ii 1 enable output setting dorega 1
    binding systemverilog xilinx_bram_inference
    physical_primitive RAMB18E1
    physical_site output DOA_REG
}
```

Exact declaration · Contract and restrictions

13.79.3 `RAMB36E1` — class

```
resource RAMB36E1 {
    class block_memory
    operation synchronous_ram
    capability capacity_bits 36864
    capability port_modes single.simple_dual.true_dual
    capability widths 1.2.4.9.18.36.72
    capability synchronous_read true
    capability output_register optional
    capability byte_write_enable true
    capability initialization true
    capability read_during_write read_first.write_first.no_change
    capability cascade true
    pipeline_site output location memory_output latency 1 ii 1 local true delay_ps 0
    pipeline_config core_registered latency 0 ii 1
    pipeline_config output_registered latency 1 ii 1 enable output setting dorega 1
    dedicated memory_cascade cascade_out -> cascade_in width 1 relation
    adjacent_same_column
    binding systemverilog xilinx_bram_inference
    physical_primitive RAMB36E1
    physical_site output DOA_REG
    physical_edge memory_cascade CASCADEOUTA CASCADEINA
}
```

Exact declaration · Contract and restrictions

13.79.4 `LUT6` — class

```
resource LUT6 { class logic operation combinational_logic capability inputs 6 binding
    systemverilog generic_rtl physical_primitive LUT6 }
```

Exact declaration · Contract and restrictions

13.79.5 `FDRE` — class

```
resource FDRE { class register operation register capability reset synchronous binding
    systemverilog generic_rtl physical_primitive FDRE }
```

Exact declaration · Contract and restrictions

13.79.6 `CARRY4` — class

```
resource CARRY4 { class arithmetic_chain operation carry dedicated carry carry_out ->
    carry_in width 1 relation adjacent_slice binding systemverilog generic_rtl
    physical_primitive CARRY4 physical_edge carry CO CI }
```

Exact declaration · Contract and restrictions

13.79.7 `LUTRAM` — class

```
resource LUTRAM { class distributed_memory operation synchronous_ram capability
    capacity_bits 64 capability widths 1.2.4 capability synchronous_read false binding
    systemverilog generic_rtl }
```

Exact declaration · Contract and restrictions

13.79.8 `SRL32` — class

```
resource SRL32 { class shift_register operation delay capability depth_max 32 binding
    systemverilog generic_rtl physical_primitive SRLC32E }
```

Exact declaration · Contract and restrictions

13.79.9 `BUFG` — class

```
resource BUFG { class clock_buffer operation clock_buffer capability glitch_free true
    capability outputs 1 binding systemverilog unsupported physical_primitive BUFG }
```

Exact declaration · Contract and restrictions

13.79.10 `PLLE2_ADV` — class

```
resource PLLE2_ADV { class clock_generator operation clock_multiply_divide capability
    input_frequency_min_mhz 19 capability input_frequency_max_mhz 800 capability
    output_frequency_min_mhz 6 capability output_frequency_max_mhz 800 capability outputs 6
     capability phase configurable capability lock true binding systemverilog unsupported
    physical_primitive PLLE2_ADV }
```

Exact declaration · Contract and restrictions

13.79.11 `MMCME2_ADV` — class

```
resource MMCME2_ADV { class clock_generator operation clock_multiply_divide capability
    input_frequency_min_mhz 10 capability input_frequency_max_mhz 800 capability
    output_frequency_min_mhz 4 capability output_frequency_max_mhz 800 capability outputs 7
     capability phase configurable capability dynamic_phase true capability lock true
    binding systemverilog unsupported physical_primitive MMCME2_ADV }
```

Exact declaration · Contract and restrictions

13.79.12 `Xilinx7Series` — class

```
target Xilinx7Series {
    provides DSP48E1
    provides RAMB18E1
    provides RAMB36E1
    provides LUT6
    provides FDRE
    provides CARRY4
    provides LUTRAM
    provides SRL32
    provides BUFG
    provides PLLE2_ADV
    provides MMCME2_ADV
}
```

Exact declaration · Contract and restrictions

13.79.13 `Xilinx7TargetLibrary` — module

```
module Xilinx7TargetLibrary
```

Exact declaration · Contract and restrictions

13.80 `std.target.xilinx.xc7z030`

Import: `import std.target.xilinx.xc7z030`.

Component contracts · Source

13.80.1 `xc7z030ffg676_1` — object

```
device xc7z030ffg676_1 : Xilinx7Series {
    part xc7z030ffg676-1
    inventory DSP48E1 400
    inventory RAMB36E1 265
    inventory RAMB18E1 530
    inventory LUT6 78600
    inventory FDRE 157200
    inventory CARRY4 19650
    inventory BUFG 32
    inventory PLLE2_ADV 8
    inventory MMCME2_ADV 8
    dedicated_capacity DSP48E1 pcascade 4
    dedicated_capacity RAMB36E1 memory_cascade 2
    dedicated_capacity CARRY4 carry 19649
}
```

Exact declaration · Contract and restrictions

13.80.2 `Xc7z030TargetLibrary` — module

```
module Xc7z030TargetLibrary
```

Exact declaration · Contract and restrictions

Inventory: 80 source namespaces; 423 top-level declarations.

Source links target the public release tag `v0.1.0a24`. For a candidate, these web links become available when that tag is published. Links between the three manuals work offline.

Font license notice

Bitstream Vera / DejaVu fonts

Copyright (c) 2003 by Bitstream, Inc. All Rights Reserved. Bitstream Vera is a trademark of Bitstream, Inc. DejaVu changes are in public domain.

Permission is hereby granted, free of charge, to any person obtaining a copy of the fonts accompanying this license (”Fonts”) and associated documentation files (the ”Font Software”), to reproduce and distribute the Font Software, including without limitation the rights to use, copy, merge, publish, distribute, and/or sell copies of the Font Software, and to permit persons to whom the Font Software is furnished to do so, subject to the following conditions:

The above copyright and trademark notices and this permission notice shall be included in all copies of one or more of the Font Software typefaces.

The Font Software may be modified, altered, or added to, and in particular the designs of glyphs or characters in the Fonts may be modified and additional glyphs or characters may be added to the Fonts, only if the fonts are renamed to names not containing either the words ”Bitstream” or the word ”Vera”.

This License becomes null and void to the extent applicable to Fonts or Font Software that has been modified and is distributed under the ”Bitstream Vera” names.

The Font Software may be sold as part of a larger software package but no copy of one or more of the Font Software typefaces may be sold by itself.

THE FONT SOFTWARE IS PROVIDED ”AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT OF COPYRIGHT, PATENT, TRADEMARK, OR OTHER RIGHT. IN NO EVENT SHALL BITSTREAM OR THE GNOME FOUNDATION BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, INCLUDING ANY GENERAL, SPECIAL, INDIRECT, INCIDENTAL, OR CONSEQUENTIAL DAMAGES, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF THE USE OR INABILITY TO USE THE FONT SOFTWARE OR FROM OTHER DEALINGS IN THE FONT SOFTWARE.

Except as contained in this notice, the names of Gnome, the Gnome Foundation, and Bitstream Inc., shall not be used in advertising or otherwise to promote the sale, use or other dealings in this Font Software without prior written authorization from the Gnome Foundation or Bitstream Inc., respectively. For further information, contact: fonts at gnome dot org.